



National University of Computer and Emerging Sciences, Lahore



StockSense

Muhammad Usman 23L-0693 BS(CS)

Muhammad Rizwan Arshad 23L-0755 BS(CS)

Hayyan Haider 23L-0727 BS(CS)

Supervisor: Mr. Saifullah Tanvir

Final Year Project

November 11, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: StockSense

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: .....

Name: Muhammad Usman

Signature: .....

Name: Muhammad Rizwan Arshad

Signature: .....

Name: Hayyan Haider

Signature: .....

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

Retailers lose money daily when a product, like clothing, footwear or any other physical product with variants, sizes or stores, is out of stock or dead stock. For large chains, the solution is enterprise forecasting systems such as SAP and Blue Yonder, which small businesses can't afford to purchase or maintain. This project aims to develop a multi-tenant SaaS application using a general purpose forecasting engine which can be applicable to any stock-keeping business, called StockSense. It features a free point of sale (POS) solution, location- and variant-level demand forecasting, a markdown/discount optimizer and stock-redistribution suggestions are available as paid subscription options. However, StockSense differs from current tools in that it recommends inter-branch redistribution as its top priority, and alerts the officer with an alert early enough that they can act on it before it becomes a "dead stock", not after it has already died. Existing businesses using a POS or spreadsheets can sign up with uploading their sales reports.

Executive Summary

Retailers face two opposing problems each day that cost them revenue. Dead stock is stored for months, requiring capital and space, while stock outs result in loss of sales as the item the customer is interested in is not available, and less popular items are increasing in quantity. This affects any retailer that manages physical inventory across variants, sizes, or store locations, from clothing and footwear shops to many other kinds of stores. The issue is most severely felt in smaller businesses because restocking decisions are typically made based on instinct. No systematic understanding of what items at what locations meet demand; a manager reorders what he or she thinks sold well.

Large retailers solve this problem by using enterprise demand-forecasting platforms like SAP and Blue Yonder, but these are also expensive and designed for those with an established data team and ERP system. Larger retailers may be able to add another forecasting service to the top of their ERP. Small retailers using basic POS systems or spreadsheets lack an equivalent solution. Even the lighter of inventory software solutions for small businesses usually provide aggregate forecasts (demand predicted per product, not per size or variant), reorder points (alerts only when stock runs low), and slow-mover reports (lists items that already sold poorly), and work with each store's inventory as a separate entity.

To fill this gap, StockSense is a multi-tenant SaaS platform. It is based on a general purpose forecasting engine which can be used for any business where stocks are kept, not just a particular product category. The platform enables small retailers to get a free POS that allows them to record sales and keep an eye on stock and brings their data to the platform, meeting a day-to-day requirement. In addition to that data, a paid subscription opens up the location and variant level demand forecasts and provides an automated markdown/discount optimizer and stock-redistribution recommendations. Companies that already use another point-of-sale system or spreadsheets can join without having to change anything, simply by uploading their previous sales data in CSV or Excel format. In both, owners and managers access their past sales data, current prediction and suggestions for restocking via a simple web dashboard accessible to non-technical users.

StockSense differs from existing solutions in three main ways. First, it prioritizes redistribution. Existing tools generate recommendations for each store independently and mention stock transfers only as one tactic among several. If a business has 2, 3 or more branches, it is almost always cheaper and faster to move unsold stock from a branch where it is not selling to a branch

where it is sold, than to sell the same stock at a discount or place a new order – StockSense identifies this as a first class recommendation, ahead of markdowns or reordering.

Second, its intervention is preventive rather than reactive. Existing technologies see stock as dead when it is too old to be a viable option, and then offer write-off or discount. The stock problem now becomes a timing problem because StockSense markdown optimizer detects a stock as it is coming to a slowdown and alerts the manager in time to take action. Thirdly, all recommendations are explained. Every suggestion is accompanied by a simple explanation of why it is warranted and an estimate of the cost, so that a business owner can better understand and be confident in the suggestion rather than blindly following it as a black box.

Preliminary work has included a review of the demand-forecasting and small-business inventory literature, surveying the available enterprise and small-business platforms, and identifying available public retail sales datasets for initial model development and validation. We could not find any platform that offers a free POS system with a possibly paid dead-stock-reduction layer to help small retailers, making StockSense dead-stock-reduction layer freemium seem to be the only one of its kind. The accuracy of the forecast will be judged using standard error metric and potential reduction of dead stock will be simulated with historical data.

To summarize, StockSense was designed to help a small business leverage the sales data that they already gather, but never analyze, to make actionable and inexpensive decisions to decrease dead stock and recover lost sales. It makes demand forecasting and inventory optimization, something traditionally only large retailers have access to, available to small businesses.

Table of Contents

List of Figures xiii

List of Tables xiv

1 Introduction 1
1.1 Purpose of this Document . . . 1
1.2 Intended Audience . . . 1
1.3 Definitions, Acronyms, and Abbreviations . . . 1
1.4 Conclusion . . . 2

2 Project Vision 3
2.1 Problem Domain Overview . . . 3
2.2 Problem Statement . . . 3
2.3 Problem Elaboration . . . 3
2.4 Goals and Objectives . . . 4
2.5 Project Scope . . . 4
2.6 Sustainable Development Goal (SDG) . . . 5
2.6.1 SDG-8 (Decent Work and Economic Growth) . . . 5
2.6.2 SDG-9 (Industry, Innovation, and Infrastructure) . . . 5
2.7 Constraints . . . 5
2.8 Business Opportunity . . . 5
2.9 Stakeholders Description / User Characteristics . . . 6
2.9.1 Key High-Level Goals and Problems of Stakeholders . . . 6
2.10 Conclusion . . . 6

3 Literature Review / Related Work 7
3.1 Definitions and Acronyms . . . 7
3.1.1 Definitions . . . 7
3.1.2 Acronyms . . . 7

3.2 Detailed Literature Review . . . 7
3.2.1 Exploring the Use of Deep Neural Networks for Sales Forecasting in Fashion Retail . . . 7
3.2.2 Clearance Pricing Optimization for a Fast-Fashion Retailer . . . 8
3.2.3 Inventory Models with Lateral Transshipments: A Review . . . 9
3.2.4 Forecasting Across Time Series Databases Using Recurrent Neural Networks on Groups of Similar Series: A Clustering Approach . . . 9
3.2.5 M5 Accuracy Competition: Results, Findings, and Conclusions . . . 9
3.2.6 Demand Forecasting for Improved Inventory Management in Small and Medium-Sized Businesses . . . 10
3.2.7 Retail Forecasting: Research and Practice . . . 10
3.2.8 Principles and Algorithms for Forecasting Groups of Time Series: Locality and Globality . . . 11
3.2.9 Reducing the Cost of Demand Uncertainty Through Accurate Response to Early Sales . . . 11
3.2.10 Sales Forecasts in Clothing Industry: The Key Success Factor of the Supply Chain Management . . . 12
3.2.11 Predictive Big Data Analytics for Supply Chain Demand Forecasting: Methods, Applications, and Research Opportunities . . . 12
3.2.12 Forecasting: Theory and Practice . . . 13
3.2.13 From Predictive to Prescriptive Analytics . . . 13
3.2.14 Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead . . . 13
3.2.15 Desperately Seeking Shelf Availability: An Examination of the Extent, the Causes, and the Efforts to Address Retail Out-of-Stocks . . . 14
3.3 Literature Review Summary Table . . . 14
3.4 Related Work . . . 17
3.4.1 Blue Yonder . . . 18
3.4.2 Nextail . . . 18
3.4.3 Shopify POS (with Stocky) . . . 18
3.4.4 Lightspeed Retail . . . 18
3.4.5 Odoo . . . 19
3.5 Related Work Summary Table . . . 19
3.6 Conclusion . . . 19

3.7 Selected Model 20

4 Software Requirement Specifications 21

4.1 List of Features 21

4.2 Functional Requirements 21

4.2.1 Functional Requirements of the Business Owner 21

4.2.2 Functional Requirements of the Store Manager 22

4.2.3 Functional Requirements of the System 22

4.3 Quality Attributes 23

4.3.1 Reliability 23

4.3.2 Maintainability 23

4.3.3 Usability 23

4.3.4 Correctness 23

4.3.5 Efficiency 23

4.3.6 Flexibility 23

4.4 Non-Functional Requirements 23

4.4.1 Performance 23

4.4.2 Reliability 24

4.4.3 Security 24

4.4.4 Compatibility 24

4.5 Assumptions 24

4.6 Use Cases 24

4.7 Hardware and Software Requirements 30

4.7.1 Hardware Requirements 30

4.7.2 Software Requirements 30

4.8 Graphical User Interface 30

4.8.1 Login Page 30

4.8.2 Signup Page 30

4.8.3 Dashboard 30

4.8.4 Point of Sale 30

4.8.5 Upload Sales Report 35

4.8.6 Inventory 35

4.8.7 Recommendations 35

4.9 Database Design 38

4.9.1 ER Diagram 38

4.9.2 Data Dictionary . . . 39
4.10 Risk Analysis . . . 39
4.11 Conclusion . . . 40

5 Proposed Approach and Methodology 41
5.1 Overall Pipeline . . . 41
5.2 Data Ingestion . . . 41
5.3 Data Validation, Cleaning, and Normalization . . . 42
5.4 Demand Series Construction . . . 42
5.5 Feature Engineering . . . 43
5.6 Demand Forecasting Model . . . 43
5.6.1 Global Gradient-Boosted Model . . . 43
5.6.2 Loss Function and Forecast Horizon . . . 44
5.6.3 Training and Tuning . . . 44
5.6.4 Statistical Baselines . . . 44
5.6.5 Initial Training Period . . . 44
5.6.6 Forecast Uncertainty . . . 45
5.7 Recommendation Engine and Stock Redistribution . . . 45
5.8 Markdown Optimization . . . 46
5.9 Explainable Recommendations . . . 47
5.10 Forecast and Recommendation Evaluation . . . 47
5.10.1 Data . . . 47
5.10.2 Protocol . . . 47
5.10.3 Metrics . . . 48
5.10.4 Experiments . . . 48
5.10.5 Recommendation Backtest . . . 48
5.11 Iterative Development . . . 49
5.12 Conclusion . . . 49

6 High-Level and Low-Level Design 50
6.1 System Overview . . . 50
6.1.1 Tenant and User Management . . . 50
6.1.2 POS and Inventory Management . . . 50
6.1.3 Data Ingestion and Cleaning . . . 50
6.1.4 Demand Forecasting . . . 50

6.1.5 Stock Redistribution and Reorder Recommendations 50
6.1.6 Markdown Optimization 51
6.1.7 Explainable Recommendations 51
6.1.8 Dashboard and Reporting 51
6.2 Design Considerations 51
6.2.1 Assumptions and Dependencies 51
6.2.2 General Constraints 52
6.2.3 Goals and Guidelines 53
6.2.4 Development Methods 53
6.3 System Architecture 54
6.3.1 Subsystem Architecture 54
6.4 Architectural Strategies 55
6.4.1 Multi-Tenant Architecture 55
6.4.2 Modular Architecture 55
6.4.3 Programming Languages and Frameworks 56
6.4.4 Database 56
6.4.5 Background Processing 56
6.4.6 Error Detection and Recovery 56
6.5 Domain Model/Class Diagram 56
6.6 Sequence Diagrams 57
6.6.1 UC-01: Register Business 59
6.6.2 UC-02: Login 59
6.6.3 UC-03: Manage Stores 59
6.6.4 UC-04: Manage Store Managers 59
6.6.5 UC-05: Manage Products and Variants 63
6.6.6 UC-06: Adjust Inventory 63
6.6.7 UC-07: Record Sale (POS) 63
6.6.8 UC-08: Upload Historical Sales Data 63
6.6.9 UC-09: View Dashboard and Alerts 67
6.6.10 UC-10: View Demand Forecast 67
6.6.11 UC-11: View and Act on Redistribution Recommendation 67
6.6.12 UC-12: View Markdown Recommendation 67
6.6.13 UC-13: View Reorder Recommendation 71
6.6.14 UC-14: Manage Subscription 71

6.6.15 UC-15: Manage Tenants . . . 71
6.7 Policies and Tactics . . . 71
6.7.1 Tools to be Used . . . 71
6.7.2 Coding Guidelines and Conventions . . . 74
6.7.3 User Interface . . . 74
6.7.4 Extensibility . . . 75
6.7.5 Tenant Data Isolation and Security . . . 75
6.7.6 Data Validation . . . 75
6.7.7 Explainable Recommendations . . . 75
6.7.8 System Testing . . . 76
6.7.9 Software Maintenance . . . 76
6.7.10 Algorithms . . . 76
6.8 Conclusion . . . 77

7 Implementation and Test Cases 78
7.1 Implementation . . . 78
7.1.1 Development Environment . . . 80
7.1.2 Tenant and User Management . . . 80
7.1.3 Store, Product and Inventory Management . . . 80
7.1.4 Point of Sale . . . 80
7.1.5 Data Ingestion and Cleaning . . . 80
7.1.6 Demand Forecasting . . . 81
7.1.7 Recommendation Engine . . . 81
7.1.8 Markdown Optimizer . . . 81
7.1.9 Explainable Recommendations . . . 81
7.1.10 Dashboard, Alerts and Subscription . . . 82
7.2 Test Case Design and Description . . . 82
7.2.1 Non-Functional Test Cases . . . 91
7.3 Test Metrics . . . 92
7.4 Conclusion . . . 93

9 Experimental Results and Discussion 94
9.1 Objectives of the Experiments . . . 94
9.2 Experimental Setup . . . 94
9.2.1 Datasets . . . 94

9.2.2 Data Preparation 94
9.2.3 Environment and Tools 95
9.2.4 Evaluation Protocol 95
9.3 Exploratory Analysis of the Demand Data 95
9.4 Experiment 1: LightGBM Against Statistical Baselines 95
9.5 Experiment 2: Global Model Against Separate Models 96
9.6 Experiment 3: Amount of Training History 96
9.7 Experiment 4: Loss Function Comparison 97
9.8 Experiment 5: Recommendation Backtest 97
9.9 Discussion 98
9.9.1 Threats to Validity 98
9.10 Conclusion 98

10 Conclusion and Future Work 100
10.1 Conclusion 100
10.2 Future Work 101

List of Figures

4.1 Login Page 31
4.2 Signup Page 32
4.3 Dashboard 33
4.4 Point of Sale 34
4.5 Upload Sales Report 35
4.6 Inventory 36
4.7 Recommendations 37
4.8 ER Diagram 38

6.1 System Architecture Diagram 54
6.2 Domain Model / Class Diagram 58
6.3 Sequence Diagram: Register Business 59
6.4 Sequence Diagram: Login 60
6.5 Sequence Diagram: Manage Stores 61
6.6 Sequence Diagram: Manage Store Managers 62
6.7 Sequence Diagram: Manage Products and Variants 63
6.8 Sequence Diagram: Adjust Inventory 64
6.9 Sequence Diagram: Record Sale (POS) 65
6.10 Sequence Diagram: Upload Historical Sales Data 66
6.11 Sequence Diagram: View Dashboard and Alerts 67
6.12 Sequence Diagram: View Demand Forecast 68
6.13 Sequence Diagram: View and Act on Redistribution Recommendation 69
6.14 Sequence Diagram: View Markdown Recommendation 70
6.15 Sequence Diagram: View Reorder Recommendation 71
6.16 Sequence Diagram: Manage Subscription 72
6.17 Sequence Diagram: Manage Tenants 73

List of Tables

2.1 Stakeholders of StockSense 6

3.1 Summary of Literature Review 14
3.2 Summary of Related Work 19

4.1 UC-01: Register Business 24
4.2 UC-02: Login 25
4.3 UC-03: Manage Stores 25
4.4 UC-04: Manage Store Managers 25
4.5 UC-05: Manage Products and Variants 26
4.6 UC-06: Adjust Inventory 26
4.7 UC-07: Record Sale (POS) 26
4.8 UC-08: Upload Historical Sales Data 27
4.9 UC-09: View Dashboard and Alerts 27
4.10 UC-10: View Demand Forecast 27
4.11 UC-11: View and Act on Redistribution Recommendation 28
4.12 UC-12: View Markdown Recommendation 28
4.13 UC-13: View Reorder Recommendation 28
4.14 UC-14: Manage Subscription 29
4.15 UC-15: Manage Tenants 29
4.16 Data Dictionary for Tenant Table 39
4.17 Data Dictionary for Store Table 39
4.18 Data Dictionary for Product Variant Table 39
4.19 Data Dictionary for Sale Table 40
4.20 Data Dictionary for Recommendation Table 40

7.1 Implementation Status of StockSense Modules 79
7.2 Test Case 1: Register Business 82
7.3 Test Case 2: Login 83
7.4 Test Case 3: Manage Stores 83

7.5 Test Case 4: Manage Store Managers 84
7.6 Test Case 5: Manage Products and Variants 84
7.7 Test Case 6: Adjust Inventory 85
7.8 Test Case 7: Record Sale (POS) 85
7.9 Test Case 8: Upload Historical Sales Data 86
7.10 Test Case 9: Upload of an Unsupported or Broken File 86
7.11 Test Case 10: View Dashboard and Alerts 87
7.12 Test Case 11: View Demand Forecast 87
7.13 Test Case 12: Redistribution Recommendation 88
7.14 Test Case 13: Markdown Recommendation 89
7.15 Test Case 14: Reorder Recommendation 89
7.16 Test Case 15: Manage Subscription 90
7.17 Test Case 16: Manage Tenants 90
7.18 Test Case 17: Tenant Data Isolation 91
7.19 Test Case 18: POS Speed 91
7.20 Test Case 19: Large File Upload 92
7.21 Test Case 20: Password and Connection Security 92
7.22 Functional Test Case Metrics 93
7.23 Non-Functional Test Case Metrics 93

9.1 Distribution of Demand Classes 95
9.2 Forecast Accuracy of LightGBM and Baselines (Weekly) 96
9.3 Global Model Against Separate Models 96
9.4 Forecast Accuracy Against Training History 97
9.5 Comparison of LightGBM Objective Functions 97
9.6 Recommendation Backtest Results 98

Chapter 1 Introduction

Retailers need the right products in the right quantities and locations, but small and medium-sized businesses often rely on spreadsheets, basic POS systems, or manual records without systematically analyzing sales data. This can cause stock outs and dead stock, particularly when demand for different products, sizes and locations is different in different branches. Large retailers have enterprise forecasting and inventory optimization tools, but these are expensive and difficult to use. StockSense is proposed as a multi-tenant SaaS solution that makes data-driven inventory management more accessible through a free POS system, demand forecasting, inter-branch stock redistribution, markdown optimization, and explainable recommendations. StockSense leverages existing sales data to enable retailers to detect inventory risks early and take action before products become unsold stock or cause stock outs.

1.1 Purpose of this Document

This document provides a detailed description of the StockSense Final Year Project (FYP), which aims to develop a multi-tenant SaaS platform for retail inventory management and demand forecasting. It provides small and medium-sized retailers with an easy-to-use solution to transform POS and historical sales data into actionable inventory decisions. The document covers the project's problem domain, objectives, scope, requirements, methodology, system design, and implementation, including demand forecasting, inventory redistribution, markdown recommendations, and explainable decision support.

1.2 Intended Audience

The main target audience for this report are the Final Year Project evaluation panel, the project supervisor, software developers, researchers, and stakeholders who are interested in retail inventory management and demand forecasting. It has also been written with the reader in mind who would like to learn about the use of data-driven technologies for small and medium sized retailers in order to better plan their inventories and minimize risk from excess inventory and stock out situations.

1.3 Definitions, Acronyms, and Abbreviations

StockSense: The proposed multi-tenant SaaS platform for retail inventory management, demand forecasting, and inventory recommendations.

FYP: Final Year Project.

POS: Point of Sale.

SaaS: Software as a Service.

AI: Artificial Intelligence.

ML: Machine Learning.

ERP: Enterprise Resource Planning.

CSV: Comma-Separated Values.

UI: User Interface.

UX: User Experience.

API: Application Programming Interface.

CRUD: Create, Read, Update, Delete.

MVP: Minimum Viable Product.

SMR: Small and Medium-sized Retailers.

1.4 Conclusion

In conclusion, StockSense is recommended to support small and medium business-store owners in better decision making on inventory management, by combining sales management and actionable sales recommendations and forecasting. The platform is centered around the challenge of stock out and dead stock - stock out at the product level, dead stock at the variant and location level. It also features prioritizing the movement of current stock between branches when necessary and also identifies slow moving stock quickly enough to take preventative measures.

This project merges a free POS layer with subscription-based forecasting and inventory optimization capabilities for businesses to leverage their existing sales data without the need of a complex enterprise system. StockSense aims to bring inventory analytics to smaller retailers by forecasting, redistribution recommendations, markdown optimization, and easy-to-understand explanations of recommendations. The vision and requirements of the project is presented in detail in the following chapters, along with related work, methodology and system design.

Chapter 2 Project Vision

StockSense aims to give the small and medium sized retailer a low cost solution for analyzing their sales pattern and making data-driven decisions on stock. It targets any retailer that stocks physical products with variants, sizes, or multiple stores, such as clothing, footwear, and similar goods, where demand fluctuates greatly by product, size and store location.

2.1 Problem Domain Overview

Retail inventory management is a process that's used to decide what products should be offered, how much, and where. For small businesses these are normally made based on experience and intuition more than formal forecasting. However, StockSense solves this issue by gathering historical sales data and then applying it to forecast demand across product size and location, and then making a recommendation – for instance, reordering stock, moving stock from one branch to another, or even discounting stock that is on the verge of expiring. The system is supposed to be multi tenant software-as-a-service (SaaS) with each business being a standalone tenant that maintains its own products, stores, inventory, sales, forecasts and recommendations. Historical sales reports can be uploaded as a csv or excel file for businesses with an external POS or spreadsheet, or sales can be recorded by businesses with the built-in POS functionality.

2.2 Problem Statement

It can be difficult for retailers to turn past sales information into product, size and location-based inventory decisions. This may result in overstocking, dead stock, unnecessary investment and stock out losses. Hence an integrated solution is required to assist with demand forecasting and inventory optimization, while providing data-driven stock management.

2.3 Problem Elaboration

The main problems addressed by StockSense are:

- Retailers lack the tools to analyze there previous sales data.
- Existing enterprise demand-forecasting solutions can be expensive and technically complex for small businesses.
- Demand can differ between stores, making aggregate product-level forecasts insufficient.

- Demand can also differ significantly between product sizes or variants.
- A store may have excess inventory while another branch needs the same product.
- Slow-moving inventory may only be identified after it has already become dead stock.
- Business owners could find machine-learning-based recommendations difficult to trust if the reasoning behind them is not explained.

2.4 Goals and Objectives

The main objectives of StockSense are:

- Design a multi-tenant platform that allows multiple small businesses to securely manage their sales and inventory data.
- Develop a data ingestion and cleaning pipeline for CSV and Excel sales reports, normalized into a schema of product, size, store/location, date, and quantity sold.
- Develop a demand-forecasting model for product-size-location combinations.
- Develop a recommendation engine that determines what to reorder, how much to reorder, and where existing inventory should be redistributed.
- Develop a markdown/discount optimizer for slow-moving stock.
- Provide explanations and estimated financial impact for every recommendation.
- Evaluate forecast performance using standard forecasting error metrics and estimate potential dead-stock reduction through historical-data simulation.
- Provide the platform under a freemium model consisting of a free POS layer and paid forecasting and recommendation features.

2.5 Project Scope

StockSense target companies have products like clothing, footwear, or any other physical product with variants, sizes, or stores that are out of stock or dead stock. It will allow several companies to be tenants on the platform with their stores, products, inventories and sales history being managed separately. The system will enable multi-tenant business management, store and product/variant management, sales and inventory recording over POS, CSV/Excel historical data upload, data cleaning/normalization, product-size-location demand forecast, inventory redistribution/reordering recommendations, markdown and discount recommendations,

explainable recommendations with estimated impact on finances, visualization in dashboard.

First, the forecasting period will be targeted on weekly and monthly demand, instead of a long-term prediction of the trend. For full automatic POS integrations to external systems, it's probably a stretch, but manual/batch upload (CSV/Excel) will be supported with the core data-ingestion mechanism.

2.6 Sustainable Development Goal (SDG)

StockSense aligns with the following United Nations Sustainable Development Goals.

2.6.1 SDG-8 (Decent Work and Economic Growth)

The technology that StockSense provides and enables is something that previously was a privilege of the large retailer and StockSense is unlocking and enabling for small retailers, helping to grow and improve the productivity of small enterprises, which are a primary driver of jobs.

2.6.2 SDG-9 (Industry, Innovation, and Infrastructure)

By providing data-driven innovation, StockSense helps retailers more effectively utilize existing capital, stores space and stock, and minimize excess accumulation of unsold inventory, helping to use resources responsibly.

2.7 Constraints

The main challenges of StockSense include limited availability and quality of historical sales data, the need for new businesses to upload sufficient sales history before reliable forecasts can be generated, and the dependence of forecast accuracy on data quality and quantity. Public datasets may not represent small retailers accurately. The system initially focuses on physical products, such as clothing and footwear, and short- to medium-term demand forecasting. Additionally, redistribution may not always be cost-effective due to transportation and handling costs, transfer delays, and the risk of stockouts at the source branch.

2.8 Business Opportunity

StockSense is a freemium SaaS startup. The basic POS features are included for free, enabling small businesses to keep track of their inventory and sales without paying any subscription fees. A paid subscription provides advanced functionality, such as demand forecasting, inventory

redistribution, markdown optimization, benchmarking, and explainable financial recommendations. This model enables companies to start leveraging the platform with a minimum upfront investment, yet offers the potential for a seamless evolution for stores aiming advanced analytics.

2.9 Stakeholders Description / User Characteristics

The key users of StockSense are business owners, who view business level performance, inventory, forecasts, and recommendations; branch managers, who view inventory level performance, sales, forecasts, and recommendations for their branch; and the system administrator, who manages overall platform level operations, tenants, subscriptions, and system configuration.

Table 2.1: Stakeholders of StockSense

Stakeholder	Responsibilities
Business Owner	Monitor business performance, inventory, forecasts, and recommendations.
Store Manager	Monitor branch-level sales, inventory, forecasts, and stock movement.
System Administrator	Manage tenants, platform configuration, and system-level operations.

2.9.1 Key High-Level Goals and Problems of Stakeholders

Business owners need an easy solution that makes existing sales data into meaningful inventory decisions. Managers need to see what they're selling and the demand for it at the store level. The system administrator needs ways to keep tenants separate and platform running. The common problem is that there's no simple, affordable system which can integrate sales recording, forecasting and inventory suggestions into a single platform.

2.10 Conclusion

This chapter explained the problem domain, goals, scope, and the stakeholders of StockSense. This platform focuses on providing a solution that helps small and medium-sized businesses leverage existing sales data to inform actionable business decisions for inventory management—without the expense or complexity of enterprise solutions. The next chapter examines the literature and current commercial products related to this project.

Chapter 3 Literature Review / Related Work

This chapter summarizes the academic research on problem formulation, demand forecasting, inventory redistribution and markdown optimization, as well as reviews commercial platforms that are relevant to similar problems. This chapter ends with an identification of the gap StockSense is intended to fill and the modeling approach that was chosen for this project.

3.1 Definitions and Acronyms

3.1.1 Definitions

Demand forecasting: The process of estimating future customer demand for a product using historical sales data.
Lateral transshipment: The movement of inventory between locations at the same level of a supply chain, such as between two retail branches, rather than from a warehouse.
Markdown optimization: The process of determining the size and timing of a price discount to sell slow-moving inventory before it becomes unsellable.
SKU: Stock Keeping Unit, a unique identifier for a specific product variant (e.g., a particular size and colour).

3.1.2 Acronyms

MAPE: Mean Absolute Percentage Error.
RMSE: Root Mean Squared Error.
ERP: Enterprise Resource Planning.
POS: Point of Sale.
ML: Machine Learning.

3.2 Detailed Literature Review

Research on fashion retail also considers forecasting demand for new items, where limited sales history makes early prediction especially difficult [1].

3.2.1 Exploring the Use of Deep Neural Networks for Sales Forecasting in Fashion Retail

Summary: Loureiro, Miguéis, and da Silva [2] compare deep neural networks against shallow methods such as support vector regression for forecasting the sales of new fashion products,

using a real dataset from a fashion retailer. They combine data-mining features describing a product's physical and distribution characteristics with expert-defined predictive variables.

Online retail studies related to StockSense [3] include price optimization that complements its forecasting goals, making it relevant to the research area StockSense falls into.

Critical Analysis: This paper demonstrates not only how deep learning is better than shallow machine learning models in terms of the accuracy of the forecasting of sales of new products but that it applies a real company's database rather than an artificially created one. What is more important, it does make predictions on a new product basis, and its data belong to a single large company, which renders the findings of the model difficult to generalize to a smaller business.

Relationship to StockSense: The choice of learning based Demand forecasting over manual estimation of demands, along with the use of product features as predictive variables has a direct connection with the fact that StockSense would be required to do demand forecasting for a particular product in a specific store.

3.2.2 Clearance Pricing Optimization for a Fast-Fashion Retailer

Summary: Caro and Gallien [4] outline a forecast-and-optimize approach that superseded the manual markdown decision making used by Zara. A field experiment across all Belgian and Irish stores during a fall-winter season showed the new process increased clearance revenue by approximately six percent, and the process was subsequently adopted company-wide.

Critical Analysis: It's an uncommon, documented and now applied – at scale, with a revenue impact – markdown-optimization model. The major drawback of it for StockSense is that it is intended for a very large vertically-integrated retailer, with extensive historical pricing data for thousands of styles, that will not exist in many of the small retailers at which StockSense operates.

Relationship to StockSense: The paper confirms the fundamental concept behind StockSense' markdown optimizer: A formal forecasting/pricing process can be better than a manual and instinctual markdown decision. StockSense takes this concept and applies it to a smaller data set, but with a mindset of "markdown" not as a clearance-event process for large retailers, but as a "preventive, early-warning" measure.

3.2.3 Inventory Models with Lateral Transshipments: A Review

Summary: Paterson, Kiesmüller, Teunter, and Glazebrook [5] review the academic literature on lateral transshipments — stock movements between locations of the same echelon — categorizing research by whether transshipments are proactive (planned in advance) or reactive (triggered by a stockout).

Critical Analysis: As a review paper, it provides broad coverage of transshipment models and clearly identifies gaps in the literature, but it does not propose a specific algorithm suited to a small retailer with only two or three branches, nor does it address how transshipment decisions should be combined with markdown or reorder decisions.

Relationship to StockSense: This review establishes the academic basis for treating inter-branch stock transfer as a distinct decision from reordering or discounting. StockSense builds on the proactive-transshipment framing described here, since its redistribution recommendations are intended to prevent dead stock and stockouts before they occur, rather than reacting to an empty shelf.

3.2.4 Forecasting Across Time Series Databases Using Recurrent Neural Networks on Groups of Similar Series: A Clustering Approach

Summary: Bandara, Bergmeir, and Smyl [6] propose clustering related time series and training a single global recurrent neural network per cluster, rather than one model per series, to improve forecast accuracy when individual series have limited history.

Critical Analysis: The clustering-based global model approach is well suited to situations with many short or sparse series, which is a strong match for a small retailer's sales history. The disadvantage of the method is that the model was tested mostly using benchmark time series instead of actual sales data from retail at the product-size-store level.

Relationship to StockSense: The work presented in this paper confirms the StockSense strategy of having a universal model for forecasting that will be trained using multiple product-size-store data sets instead of building separate models for each product-size-store pair because of the small number of observations for an individual retailer.

3.2.5 M5 Accuracy Competition: Results, Findings, and Conclusions

Summary: In Makridakis, Spiliotis, and Assimakopoulos' [7] work, the authors describe the outcome of the M5 forecasting competition, where thousands of participants were predicting

42,840 retail sales time series hierarchies for Walmart. It was concluded in the M5 competition that machine-learning algorithms, especially gradient-boosted trees such as LightGBM, have better forecasting accuracy than statistical approaches.

Critical Analysis: The M5 competition can be considered as one of the largest empirical tests of forecasting approaches in the retail sector. Its conclusions are often used as a basis for evaluation of how good different model families are on real-world retail sales data. However, the size of the competition indicates a very big company with long-term sales history; forecasting for small retailers with short sales history is not considered.

Relationship to StockSense: The conclusion of M5 about superiority of gradient-boosted trees over statistical models in the context of hierarchical retail sales time series directly impacts the choice of a family of forecasting models for StockSense.

3.2.6 Demand Forecasting for Improved Inventory Management in Small and Medium-Sized Businesses

Summary: Purnamasari, Permadi, Saepudin, and Agusdin [8] compare machine-learning approaches, including an ensemble gradient-boosting model, against classic statistical projection methods for demand forecasting in small leathercraft businesses, using K-means clustering to group products by similarity before forecasting.

Critical Analysis: This is one of the few studies that directly targets small-business demand forecasting rather than large retail chains, and its finding that machine-learning methods outperform classic statistical approaches is directly relevant. The study is limited to a single small-business dataset in one product category, and does not address size- or location-level demand.

Relationship to StockSense: This paper is the most similar to StockSense in terms of its target audience – a small business with no data-science team. Its finding that gradient-boosting-based methods outperform classic statistical approaches on small-business sales data supports StockSense's choice of a gradient-boosted model with statistical baselines for comparison.

3.2.7 Retail Forecasting: Research and Practice

Summary: Fildes, Ma, and Kolassa [9] conduct an extensive literature review on academic research on retail forecasting considering the forecasting needs from strategic to tactical and operational levels as well as the gap between academic forecasting research and retail store practice.

Critical Analysis: It is a survey of the field, and thus can serve as a map of the retail-forecasting research landscape, and clearly shows that new-product forecasting is not studied as much as product forecasting. It is a review and does not itself propose or test a model.

Relationship to StockSense: The paper's finding that most published research and tools focused on retail forecasting still rely on a data-science team and an IT infrastructure is consistent with the premise of this project: that existing research and tools are not well suited to small retailers, and StockSense aims to address this gap.

3.2.8 Principles and Algorithms for Forecasting Groups of Time Series: Locality and Globality

Summary: Montero-Manso and Hyndman [10] describe conditions under which it is preferable to predict a set of related time series using a shared (global) model rather than a separate (local) model for each series, and demonstrate that global models can outperform local models even when there is no obvious similarity between the series.

Critical Analysis: The paper offers a sound theoretical and empirical foundation for global forecasting models, directly applicable to justify StockSense's model-selection approach. It is a general time-series paper, not a retail-specific one, so its recommendations need to be adapted to the specific structure of product-size-store sales data.

Relationship to StockSense: This theoretical result supports StockSense's use of one shared forecasting model across all of a tenant's products and stores, rather than training a separate model for each product-size-store series, since the latter would require more historical data than a small retailer is likely to have.

3.2.9 Reducing the Cost of Demand Uncertainty Through Accurate Response to Early Sales

Summary: Fisher and Raman [11] show that fashion retailers who shorten lead times and react to early-season sales, rather than committing to production and ordering decisions months in advance, can substantially cut costs. Applied with a major ski-wear firm, their method increased profits by roughly 60% relative to the firm's existing informal response process.

Critical Analysis: This is a classic example of research that shows that responding to demand signals early is superior to traditional, instinctive methods of stocking. The drawback of using this article in relation to StockSense is that it talks about the decision of manufacturing and ordering for just one season for one particular retail chain.

Relationship to StockSense: This article comes early in the literature and serves as a foundation for the concept that StockSense relies on - i.e., responding quickly to demand signals reduces both stock-outs and overstocking. It is evidence for the need to treat the sales velocity as an important signal for recommendation engine and markdown optimizer.

3.2.10 Sales Forecasts in Clothing Industry: The Key Success Factor of the Supply Chain Management

Summary: Thomassey [12] provides a literature review on the forecasting methods used in the clothing industry and develops forecasting methods tailored for handling the shorter life cycle, high seasonality, and lack of historical data of the clothing sales.

Critical Analysis: The paper explicitly tackles the issue of data scarcity and seasonality, which makes forecasting of the demand of apparel products difficult compared to the forecasting of staple products, making the paper highly applicable to the problem being addressed by StockSense.

Relationship to StockSense: The paper supports the project's understanding that the demand of size- and variant-specific apparel products is highly seasonal compared to the product-level demand, which is why StockSense decided to develop the forecasting methods that take into account the product-size-location combinations.

3.2.11 Predictive Big Data Analytics for Supply Chain Demand Forecasting: Methods, Applications, and Research Opportunities

Summary: The review by Seyedan and Mafakheri [13] surveys the methodologies and applications of big-data predictive demand forecasting for supply chain management systems, categorizing methodologies and pointing out the areas of open research questions.

Critical Analysis: As a general survey, this provides a good overview of the current methodologies for data driven forecasting and shows that the majority of applications in practice rely on large centrally controlled supply chains with analytical capacity. This does not present or test any particular model.

Relationship to StockSense: The gap found by Seyedan and Mafakheri [13] between advanced forecast models and companies without analytical infrastructure makes a good basis for the current research idea.

3.2.12 Forecasting: Theory and Practice

Summary: Petropoulos et al. [14] have written a comprehensive review on the topic of forecasting theory, methods, and practice.

Critical Analysis: The scope of this review is helpful to refer to the forecast evaluation practices and model selection guidelines; however, as it is an encyclopedic review, it lacks the retail and small business focus.

Relationship to StockSense: The review provides useful insights into the forecast evaluation and model selection that guide the comparison of the gradient-boosted model and basic statistical benchmarks, as discussed in the forecast evaluation methodology of StockSense.

3.2.13 From Predictive to Prescriptive Analytics

Summary: A framework for transforming prediction models into prescriptions by Bertsimas and Kallus [15] is provided using the example of real-life inventory management for a large media company.

Critical Analysis: The article represents a good theoretical foundation for going beyond simple prediction and reaching a decision stage, which is important in the context of StockSense's design. The use case presented is one for a very large distribution network, whereas StockSense's case is that of a multi-branch retailer.

Relationship to StockSense: This article helps to support the design decision of StockSense that goes beyond simply predicting but provides a transformation of prediction into an actionable prescription (redistribution, reordering, markdown).

3.2.14 Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead

Summary: In high-stakes decision-making processes, according to Rudin [16], there is a need for inherently interpretable models rather than black-box models and subsequent explanations because explanations generated by black-box models may not be reliable.

Critical Analysis: The argument is well-supported and widely cited, though it is framed around high-stakes domains such as criminal justice and healthcare rather than retail inventory decisions, where the stakes of an individual recommendation are comparatively low.

Relationship to StockSense: Although the recommendations made by StockSense are not high-risk as those discussed by Rudin, it raises the same issue where an entrepreneur would definitely

be skeptical about the recommendation which he doesn't understand. Hence, it justifies the approach adopted by StockSense of always providing an explanation for every recommendation.

3.2.15 Desperately Seeking Shelf Availability: An Examination of the Extent, the Causes, and the Efforts to Address Retail Out-of-Stocks

Summary: Corsten and Gruen [17] conducted a global study of out-of-stock in retail and showed that there was no improvement in the number of stockouts over the last decade, and that a substantial proportion of consumers leave the store rather than substitute the item when facing out-of-stocks.

Critical Analysis: This is a major, well-cited paper which shows out-of-stocks as a consistent and expensive issue related to consumer experience, thus providing the motivation for this project from practical grounds, not just internally. It concerns FMCGs, not apparel/footwear.

Relationship to StockSense: This paper strengthens the business case for prioritizing redistribution and timely reordering: since customers facing a stockout may leave the store entirely rather than buy a different item, preventing a stockout has a larger business impact than simple lost-sale accounting would suggest.

3.3 Literature Review Summary Table

Table 3.1: Summary of Literature Review

No.	Paper Title	Year	Summary	Relevance to StockSense
1	Exploring the Use of Deep Neural Networks for Sales Forecasting in Fashion Retail	2018	Deep learning outperforms shallow models for new fashion-product sales forecasting	Supports learning-based forecasting of fashion product sales

Continued on next page

No.	Paper Title	Year	Summary	Relevance to StockSense
2	Clearance Pricing Optimization for a Fast-Fashion Retailer	2012	Formal markdown model increased clearance revenue at Zara by 6%	Validates preventive markdown optimization as a business lever
3	Inventory Models with Lateral Transshipments: A Review	2011	Reviews proactive and reactive stock-transfer models between locations	Basis for treating branch-to-branch redistribution as a first-class action
4	Forecasting Across Time Series Databases Using RNNs on Groups of Similar Series	2020	Clustering similar series and forecasting them with one shared model improves accuracy	Basis for shared/clustered forecasting models
5	M5 Accuracy Competition: Results, Findings, and Conclusions	2022	Gradient-boosted tree models outperformed statistical baselines on large-scale retail data	Informs choice of forecasting model family
6	Demand Forecasting for Improved Inventory Management in SMBs	2023	ML outperforms classic methods for small-business demand forecasting	Directly matches StockSense's target user segment

Continued on next page

No.	Paper Title	Year	Summary	Relevance to StockSense
7	Retail Forecasting: Research and Practice	2022	Survey of retail-forecasting research and the gap with practitioner adoption	Confirms the small-retailer research and tooling gap
8	Principles and Algorithms for Forecasting Groups of Time Series	2021	Formalizes when a shared global model outperforms per-series local models	Theoretical basis for shared forecasting models
9	Reducing the Cost of Demand Uncertainty Through Accurate Response to Early Sales	1996	Quick Response to early sales data increased a fashion retailer's profits by ∼60%	Validates acting on recent sales velocity for stocking decisions
10	Sales Forecasts in Clothing Industry: The Key Success Factor of Supply Chain Management	2010	Reviews forecasting models suited to apparel's short life cycles and seasonality	Supports forecasting at the product-size-location level
11	Predictive Big Data Analytics for Supply Chain Demand Forecasting	2020	Survey of data-driven forecasting methods and research gaps across supply chains	Confirms the gap for organizations without analytics infrastructure

Continued on next page

No.	Paper Title	Year	Summary	Relevance to StockSense
12	Forecasting: Theory and Practice	2022	Encyclopedic review of forecasting theory, methods, and evaluation practice	Informs forecast evaluation and model-selection approach
13	From Predictive to Prescriptive Analytics	2020	Framework for converting predictive models into actionable decisions	Basis for converting forecasts into recommendations, not just numbers
14	Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead	2019	Argues interpretable models are more trustworthy than explained black boxes	Supports explainable, plain-language recommendations
15	Desperately Seeking Shelf Availability	2003	Worldwide study finds stockouts persistent; many customers switch stores entirely	Strengthens business case for prioritizing stockout prevention

3.4 Related Work

SAP IBP is an enterprise demand-sensing and planning platform designed to take in real-time signals from the point of sale to fine-tune forecasts and prevent stockouts and overstocking. It's designed to work for large businesses with a planning staff already in place and enterprise licensing costs, and cannot be used by a small retailer running two or three stores, and who has no budget for enterprise licensing.

3.4.1 Blue Yonder

Home to large retailers like Sainsbury's, Blue Yonder, a part of Panasonic, offers machine learning-based demand planning and retail-planning software. It has causal forecasting (not only based on historical averages) and also offers the ability to explain its predictions in a "glassbox" manner. It's on an enterprise level like SAP, so it takes a lot of effort to implement and technical infrastructure, which a small business does not have.

3.4.2 Nextail

Nextail is an AI-powered merchandising platform specifically designed for fashion, footwear and apparel retailers that enables in-season allocation, replenishment and inventory rebalancing between stores that helps to ensure full-price sell-through and minimise end-of-season markdowns. It's the most similar thing to StockSense's redistribution focus, but it applies to mid-size and large fashion chains, costs a lot more, and gives its recommendations as a part of a larger merchandising tool as opposed to a free one for a very small retailer.

3.4.3 Shopify POS (with Stocky)

In addition to multi-location inventory management, stock transfer between locations, low-stock alerts, shopify POS has inventory-rebalancing recommendations between stores and demand-forecasting reorder suggestions through a module called Stocky and Sidekick AI assistant. It's the easiest of the tools mentioned for a small retailer who is selling on Shopify. The core level subscription fee, however, does not include the advanced inventory features, which are available only with paid Shopify plans, while its forecasting and rebalancing capabilities are at product level—not size or variant level.

3.4.4 Lightspeed Retail

Offering stock transfer, custom reorder points per location and reports that highlight slow moving items, Lightspeed Retail is a POS system designed for multi-location retailers with a large number of SKUs. It also lacks variant-level or size-level demand prediction, explainable recommendations or an independent markdown-timing tool as Shopify has, and doesn't come with a free tier—meaning that retailers will have to pay money for the POS, even if they don't need advanced inventory analytics.

3.4.5 Odoo

Odoo is an open-source business platform whose free Community edition includes a Point of Sale app and an Inventory app with built-in reordering rules and multi-warehouse stock transfers. It is the closest existing tool to StockSense's free-POS model, since the core POS and inventory tracking cost nothing. However, Odoo's free edition has no built-in demand forecasting, size/variant-level analysis, or explainable recommendations; such capabilities exist only as separately priced third-party marketplace apps or within its paid Enterprise plans, and even those add-ons generally forecast at the product level rather than prioritizing branch-to-branch redistribution the way StockSense does.

3.5 Related Work Summary Table

Table 3.2: Summary of Related Work

No.	Platform	Target Users	Gap Relative to StockSense
1	SAP Integrated Business Planning	Large enterprises with ERP	Requires ERP infrastructure and enterprise budget; no free tier
2	Blue Yonder	Large retail chains	Enterprise-only; requires dedicated planning/IT teams
3	Nextail	Mid-size to large fashion chains	Closest in focus to redistribution, but priced and scaled for many-store chains, not a free small-retailer entry point
4	Shopify POS (Stocky/Sidekick)	Shopify merchants of any size	Product-level, not size/variant-level, forecasting; advanced features gated behind paid plans
5	Lightspeed Retail	Multi-location, high-SKU retailers	No demand forecasting or explainable recommendations; no free tier
6	Odoo (Community)	Small businesses wanting free, self-hosted POS/Inventory	Free tier has no demand forecasting or explainable recommendations; forecasting only via paid add-ons, product-level not size/variant-level

3.6 Conclusion

The literature demonstrates that machine-learning predictions and formal markdown and redistribution methods can enhance inventory choices and income. There are solutions out there, though, that are geared toward larger retailers, who have infrastructure in place and data teams. Multiple solutions were reviewed, but few are free to use and include explainable, size-

and location-level forecasting with redistribution as a priority for small, multi-branch retailers. StockSense is designed to fill this void.

3.7 Selected Model

According to this review, StockSense's forecasting algorithm will employ the gradient-boosted tree model family as the main one, as this family is the one that was found to be performing well in the M5 competition on the hierarchical retail sales data, while simple statistical methods are kept as a reference to compare the performance of the other models. The recommendations for redistribution will be based on the proactive-transshipment framing from Paterson et al., with the emphasis put on moving stocks between branches rather than ordering and/or discounting. The markdown optimizer will do what Caro and Gallien did - it will mark items down even before they are considered unsellable.

Chapter 4 Software Requirement Specifications

This chapter describes the functional and non-functional requirements of StockSense, derived from the project goals and scope described in Chapter 2.

4.1 List of Features

- Multi-tenant business registration and secure login.
- Free built-in point-of-sale (POS) for recording sales and tracking inventory.
- CSV/Excel upload of historical sales reports for businesses using an external POS.
- Data validation, cleaning, and normalization of uploaded sales data.
- Product, size/variant, and store/location management.
- Product-size-location demand forecasting.
- Recommendations for inventory re-distribution between branches.
- Reordering recommendations.
- Slow moving stock mark down/discount optimizer.
- Recommendations that are easy to explain, and have an estimated financial impact.
- Web dashboard to access sales, inventory, forecasts and recommendations.
- Freemium subscription management (free POS tier and paid analytics tier).
- Cross-tenant benchmarking (if available).

4.2 Functional Requirements

4.2.1 Functional Requirements of the Business Owner

- FR-1: The owner of the business will be able to sign up business on behalf of the tenant and to open a business account.
- FR-2: The business owner should be able to manage stores within the business to add, edit and remove them.
- FR-3: The owner of the business will have access to all stores' sales, inventory, and forecast data consolidated.
- FR-4: The business owner should have access to and can execute redistribution, reorder and markdown suggestions.
- FR-5: Business owner will be able to increase or decrease subscription level.
- FR-6: The business owner shall be able to invite and manage store-manager accounts.

4.2.2 Functional Requirements of the Store Manager

- FR-7: The store manager shall be able to record sales through the POS interface.
- FR-8: The store manager shall be able to view current inventory for their assigned store.
- FR-9: The store manager shall be able to view demand forecasts and recommendations relevant to their store.
- FR-10: The store manager shall be able to confirm or reject a redistribution recommendation involving their store.

4.2.3 Functional Requirements of the System

- FR-11: The system shall separate tenant's data from other tenants.
- FR-12: The system should be able to accept and validate sales-report uploads in CSV and Excel.
- FR-13: The system shall normalize uploaded data into a common schema (product, size, store, date, quantity).
- FR-14: After each sale recorded, the system shall automatically update the inventory.
- FR-15: The system shall create product-size-location level demand forecasts.
- FR-16: The system shall determine if there are shortages or surpluses of the inventory as compared to the forecasted demand.
- FR-17: When two or more branches have overstocked items and another branch has a projected shortage of the same type of item and size, FR-17 shall give a recommendation to redistribute.
- FR-18: The system shall make a reorder suggestion if there is no other source of adequate surplus.
- FR-19: The system shall identify inventory at risk of becoming dead stock and generate a markdown recommendation before it becomes unsellable.
- FR-20: Recommendations shall be accompanied by a clear statement of its consequences and a reasonable cost estimate.
- FR-21: The system shall restrict access to paid features based on the tenant's subscription tier.
- FR-22: The system shall evaluate forecast accuracy using standard error metrics (e.g., MAPE, RMSE).

4.3 Quality Attributes

4.3.1 Reliability

The system shall provide consistent forecasts and recommendations from the same input data, and shall not crash or corrupt tenant data on the server on receiving malformed uploads.

4.3.2 Maintainability

The forecasting, recommendation, markdown and pos logic shall be built as separate modules enabling one to be changed, retrained or replaced without affecting other modules.

4.3.3 Usability

The dashboard should share forecasts and suggestions in a business owner-friendly format, accessible without any data-science or machine-learning expertise.

4.3.4 Correctness

The quantity of the inventory items shown to the user will always be the latest amount sold and transferred based on recommendations.

4.3.5 Efficiency

The system shall be able to provide a forecast and recommendations for the entire product catalog of a tenant within a reasonable time, depending on the size of the uploaded data set.

4.3.6 Flexibility

The system shall be able to add new forecasting models or recommendation rules without redesigning the data ingestion and/or dashboard layers.

4.4 Non-Functional Requirements

4.4.1 Performance

It should handle a typical small retailer's monthly sales upload (a few thousand rows) and be able to provide forecasts in a couple of minutes.

4.4.2 Reliability

The system should be highly available for both the POS and the dashboard, as the store manager might rely on the POS during the business hours.

4.4.3 Security

Tenant information will be isolated at the database level. Secure hashing algorithm shall be used to store passwords. Communication between client and server shall be encrypted (HTTPS/TLS).

4.4.4 Compatibility

Access to the dashboard should be via modern web browser, with no need to install a native application.

4.5 Assumptions

- Businesses have, or are willing to generate, sales data at the product-size-store level.
- Users have basic computer literacy and a stable internet connection.
- Uploaded CSV/Excel files follow, or can be mapped to, a reasonably consistent column structure.
- A single store belongs to exactly one tenant at a time.
- Forecasting accuracy will improve as more historical data accumulates for a given tenant.

4.6 Use Cases

Table 4.1: UC-01: Register Business

Name	Register Business
Actors	Business Owner
Summary	A new business signs up as a tenant on StockSense.
Pre-Conditions	The business is not already registered.
Post-Conditions	A new tenant account is created on the free POS tier.
Basic Flow	1. The owner opens the registration page. 2. The owner enters business and account details. 3. The system creates the tenant and sends a verification email.
Alternative Flow	If the email is already registered, the system displays an error and asks for a different email.

Table 4.2: UC-02: Login

Name	Login
Actors	Business Owner, Store Manager, System Administrator
Summary	A registered user logs in using an email and password.
Pre-Conditions	The user has an account and is not already logged in.
Post-Conditions	A session is established and the user is redirected to the dashboard.
Basic Flow	1. The user opens the login page. 2. The user enters the email and password. 3. The system verifies the credentials and opens the dashboard.
Alternative Flow	If the credentials are invalid, the system displays "Incorrect email or password" and stays on the login page.

Table 4.3: UC-03: Manage Stores

Name	Manage Stores
Actors	Business Owner
Summary	The owner adds, edits, or removes the stores (branches) of the business.
Pre-Conditions	The owner is logged in.
Post-Conditions	The list of stores of the tenant is updated.
Basic Flow	1. The owner opens the Stores page. 2. The owner chooses to add, edit, or remove a store and enters the details. 3. The system validates and saves the change.
Alternative Flow	If a store being removed still has inventory or sales history, the system archives it instead of deleting it.

Table 4.4: UC-04: Manage Store Managers

Name	Manage Store Managers
Actors	Business Owner
Summary	The owner invites store managers, assigns them to a store, or removes their access.
Pre-Conditions	The owner is logged in and at least one store exists.
Post-Conditions	The store manager accounts of the tenant are updated.
Basic Flow	1. The owner opens the Users page. 2. The owner invites a manager by email and selects the store to assign. 3. The system creates the account and sends an invitation.
Alternative Flow	If the email already belongs to another account, the system rejects the invitation and shows an error.

Table 4.5: UC-05: Manage Products and Variants

Name	Manage Products and Variants
Actors	Business Owner
Summary	The owner adds, edits, or removes products and their sizes/variants.
Pre-Conditions	The owner is logged in.
Post-Conditions	The product catalog of the tenant is updated.
Basic Flow	1. The owner opens the Products page. 2. The owner creates or edits a product and its sizes/variants. 3. The system validates and saves the product catalog.
Alternative Flow	If a variant with the same SKU already exists, the system rejects the entry and asks for a different SKU.

Table 4.6: UC-06: Adjust Inventory

Name	Adjust Inventory
Actors	Business Owner, Store Manager
Summary	A user corrects the stock quantity of a variant at a store, for example after a physical count.
Pre-Conditions	The user is logged in and the variant exists.
Post-Conditions	The stock quantity is updated and the adjustment is logged.
Basic Flow	1. The user selects a store and a variant. 2. The user enters the new quantity and a reason. 3. The system updates the inventory and records the adjustment.
Alternative Flow	If the entered quantity is negative, the system rejects the adjustment.

Table 4.7: UC-07: Record Sale (POS)

Name	Record Sale (POS)
Actors	Store Manager
Summary	A store manager records a sale through the built-in POS.
Pre-Conditions	The manager is logged in; the product and size exist in inventory.
Post-Conditions	A sales record is created and inventory is decremented.
Basic Flow	1. The manager selects the product and size sold. 2. The manager confirms the quantity and completes the sale. 3. The system logs the sale and updates inventory.
Alternative Flow	If inventory is insufficient, the system warns the manager before confirming.

Table 4.8: UC-08: Upload Historical Sales Data

Name	Upload Historical Sales Data
Actors	Business Owner
Summary	The owner uploads a CSV or Excel file of historical sales from an external POS or spreadsheet.
Pre-Conditions	The owner is logged in and belongs to a registered tenant.
Post-Conditions	Valid rows are cleaned, normalized, and stored; invalid rows are listed in an upload report.
Basic Flow	1. The owner selects a CSV/Excel file. 2. The system shows the detected columns and the owner maps them to the required fields. 3. The system validates and normalizes the data and shows an upload report. 4. The owner approves the report and the system imports the rows.
Alternative Flow	If the file format is unsupported or unreadable, the system rejects the upload and explains why. Rejected rows are listed in the report with reasons.

Table 4.9: UC-09: View Dashboard and Alerts

Name	View Dashboard and Alerts
Actors	Business Owner, Store Manager
Summary	The user views a summary of sales, inventory, forecasts, recommendations, and alerts.
Pre-Conditions	The user is logged in.
Post-Conditions	The dashboard displays current, tenant-scoped data.
Basic Flow	1. The user logs in and is directed to the dashboard. 2. The system displays sales trends, current inventory, open recommendations, and alerts.
Alternative Flow	If the tenant has no sales data yet, the dashboard asks the user to record or upload sales.

Table 4.10: UC-10: View Demand Forecast

Name	View Demand Forecast
Actors	Business Owner, Store Manager
Summary	The user views the forecasted weekly or monthly demand of a product-size-store combination.
Pre-Conditions	The user is logged in and the tenant has a paid subscription.
Post-Conditions	The forecast is displayed next to the past sales.
Basic Flow	1. The user opens the Forecasts page. 2. The user selects a product, size, and store. 3. The system displays the stored forecast with the past sales.
Alternative Flow	If the tenant has not yet provided enough sales history to train a model, no forecast is shown and the user is asked to record or upload more data. On the free tier, the system asks the user to upgrade.

Table 4.11: UC-11: View and Act on Redistribution Recommendation

Name	View and Act on Redistribution Recommendation
Actors	Business Owner, Store Manager
Summary	The user reviews a suggestion to move stock from one branch to another and approves or dismisses it.
Pre-Conditions	A forecasted shortage at one store coincides with a forecasted surplus of the same item at another.
Post-Conditions	An approved recommendation creates a stock transfer; a dismissed one is closed.
Basic Flow	1. The user opens the Recommendations page and selects a redistribution recommendation. 2. The system displays source and destination stores, quantity, explanation, and estimated financial impact. 3. The user approves it and the system creates the stock transfer and notifies both stores.
Alternative Flow	The user dismisses the recommendation and the system marks it as dismissed.

Table 4.12: UC-12: View Markdown Recommendation

Name	View Markdown Recommendation
Actors	Business Owner, Store Manager
Summary	The user views a suggestion to discount a slow-moving item before it becomes dead stock.
Pre-Conditions	The markdown optimizer has flagged an item as at risk of becoming dead stock.
Post-Conditions	The user sees the recommended discount, explanation, and estimated impact.
Basic Flow	1. The user opens the Recommendations page. 2. The user selects a markdown recommendation. 3. The system displays the item, suggested discount, explanation, and estimated financial impact.
Alternative Flow	The user can dismiss the recommendation if a markdown is not desired.

Table 4.13: UC-13: View Reorder Recommendation

Name	View Reorder Recommendation
Actors	Business Owner
Summary	The user views a suggestion to reorder an item when no surplus exists at other branches.
Pre-Conditions	A forecasted shortage remains after redistribution has been considered.
Post-Conditions	The user sees the suggested reorder quantity with its explanation.
Basic Flow	1. The user opens the Recommendations page. 2. The user selects a reorder recommendation. 3. The system displays the item, store, suggested quantity, and explanation.
Alternative Flow	The user marks the recommendation as actioned or dismisses it.

Table 4.14: UC-14: Manage Subscription

Name	Manage Subscription
Actors	Business Owner
Summary	The owner views the current plan and upgrades or downgrades between the free and paid tiers.
Pre-Conditions	The owner is logged in.
Post-Conditions	The subscription tier is updated and paid features are enabled or disabled accordingly.
Basic Flow	1. The owner opens the Subscription page. 2. The owner selects a plan and confirms the change. 3. The system updates the tier and the available features.
Alternative Flow	If the payment fails, the system keeps the current tier and informs the owner.

Table 4.15: UC-15: Manage Tenants

Name	Manage Tenants
Actors	System Administrator
Summary	The administrator views tenants and manages their status and plans.
Pre-Conditions	The administrator is logged in.
Post-Conditions	The status or plan of the selected tenant is updated.
Basic Flow	1. The administrator opens the Tenants page. 2. The administrator selects a tenant and chooses an action (suspend, reactivate, or change plan). 3. The system applies the change and records it.
Alternative Flow	Users of a suspended tenant cannot log in until the tenant is reactivated.

4.7 Hardware and Software Requirements

4.7.1 Hardware Requirements

- Server: Minimum 8 GB RAM, quad-core processor, cloud-hosted or on-premise.
- Client: Any modern computer or smartphone with a web browser and internet access.

4.7.2 Software Requirements

- Backend: Python (FastAPI) for the application and forecasting services.
- Frontend: React, for the web dashboard.
- Database: PostgreSQL.
- Forecasting/ML: scikit-learn and LightGBM (gradient-boosted trees), following the M5-informed model choice from Chapter 3.
- Operating System: Linux server environment.

4.8 Graphical User Interface

GUI mockups of each screen, with a description of the users and functionality, are given below.

4.8.1 Login Page

Any registered user, whether a business owner or a store manager, logs in from this page using their email and password.

4.8.2 Signup Page

A new business owner registers their business as a tenant from this page, creating the initial account for StockSense.

4.8.3 Dashboard

On this page, the user can view an overview of monthly sales, items at risk of stockout or dead stock, recent sales activity, and inventory health across branches.

4.8.4 Point of Sale

A store manager records a sale from this page by selecting the branch, product, and variant sold. Inventory is updated automatically once the sale is confirmed.

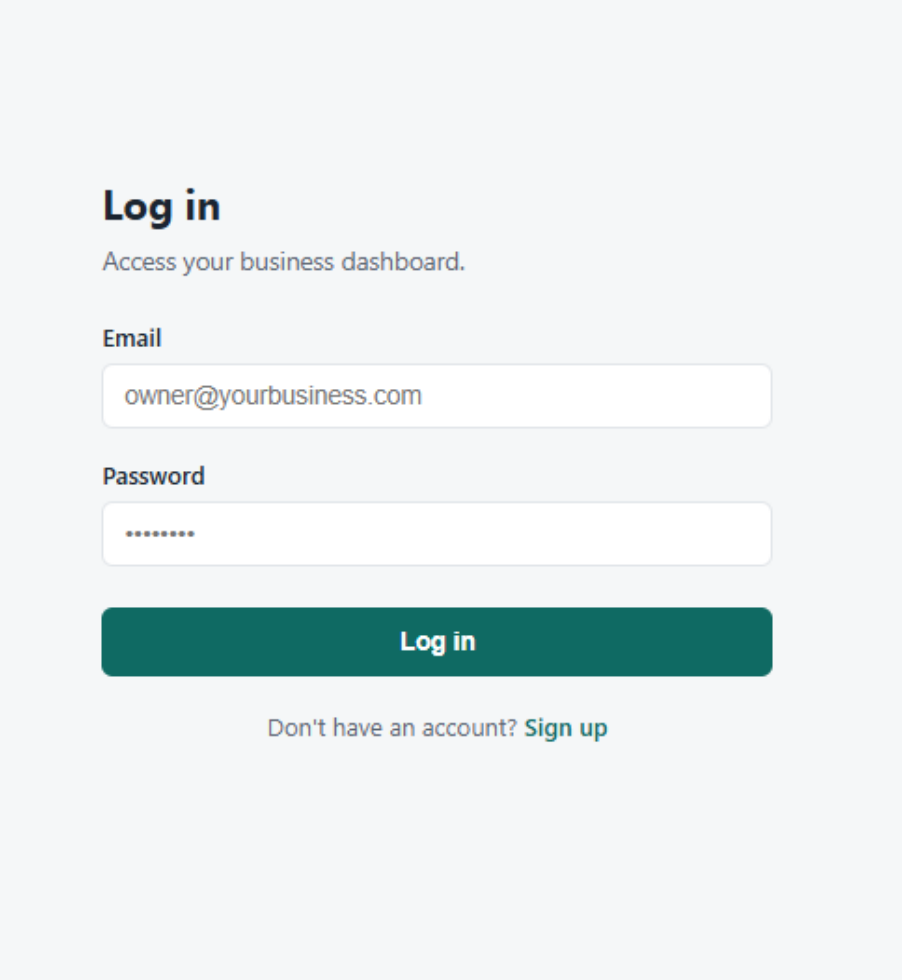


Figure 4.1: Login Page

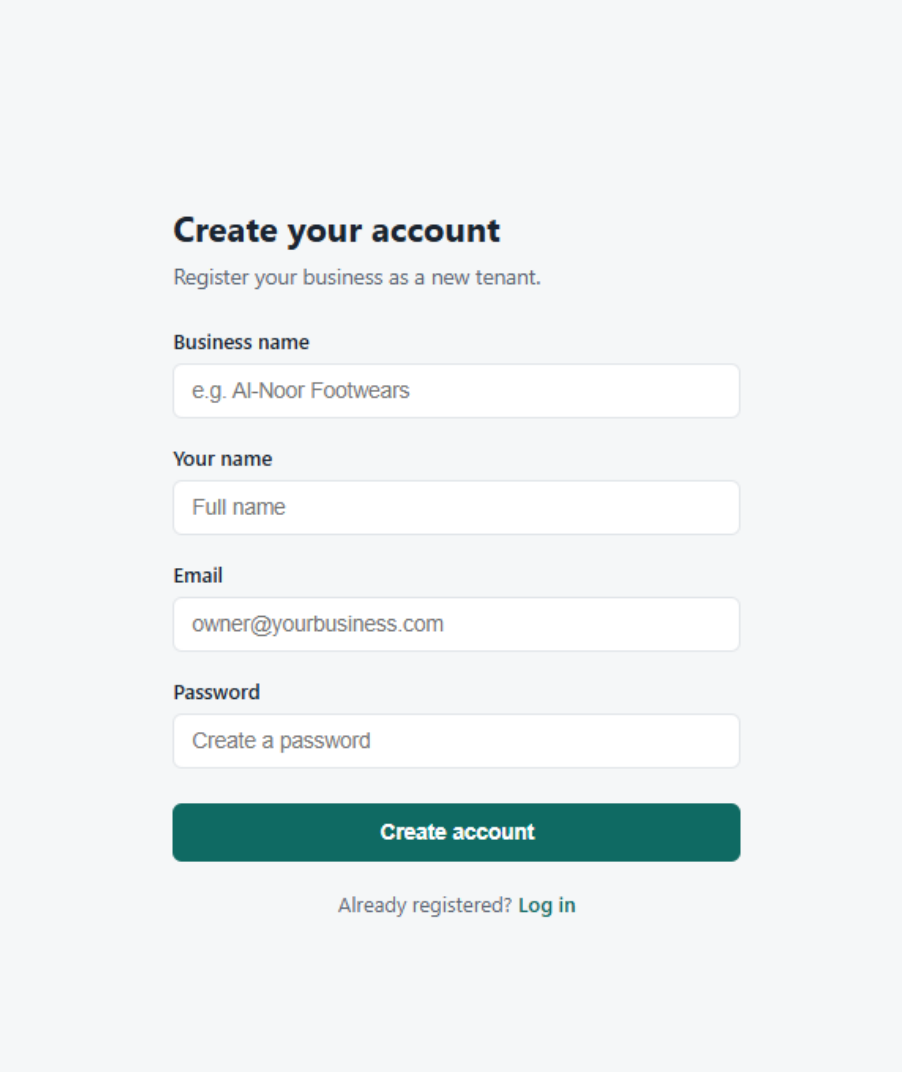


Figure 4.2: Signup Page

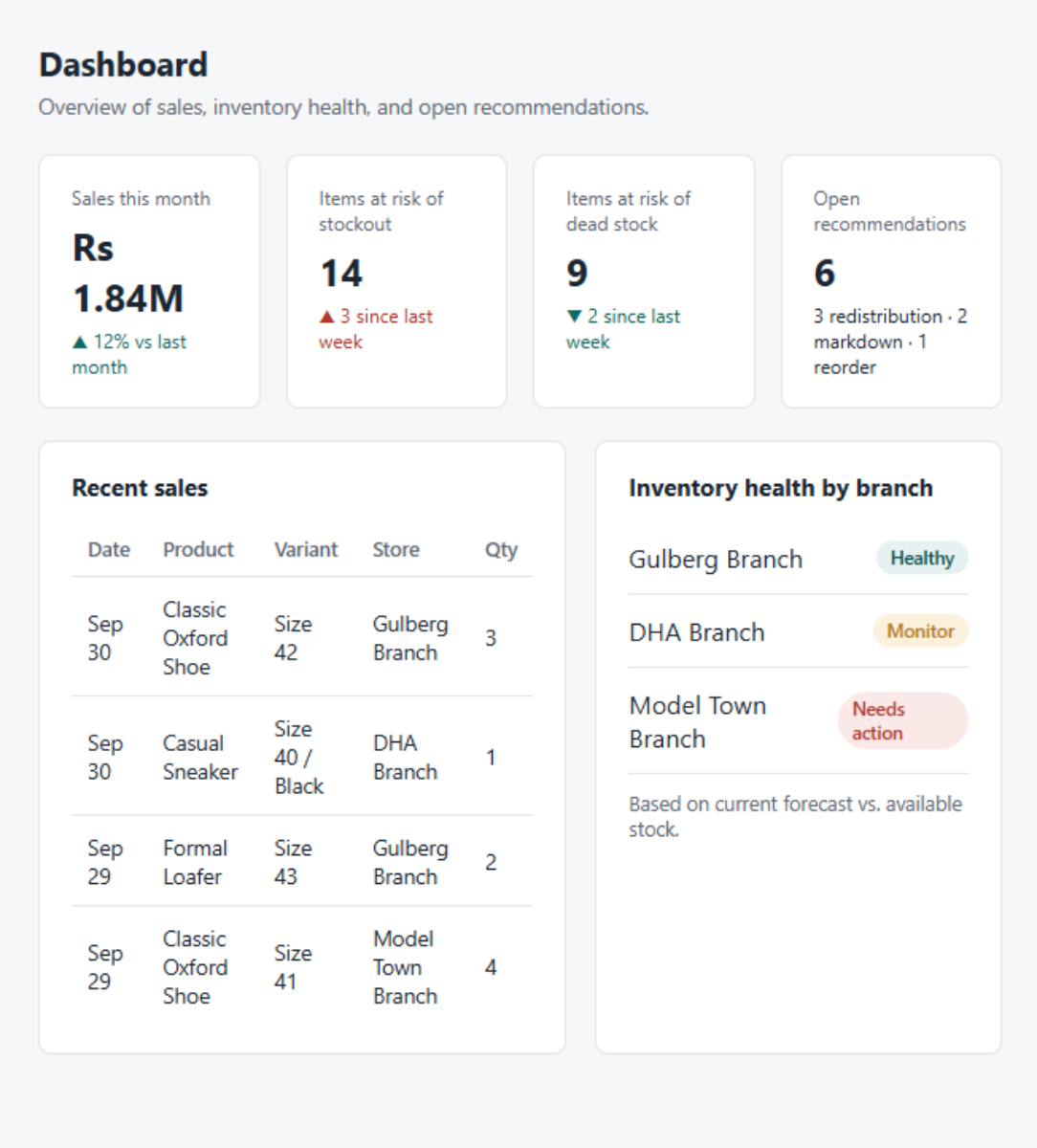


Figure 4.3: Dashboard

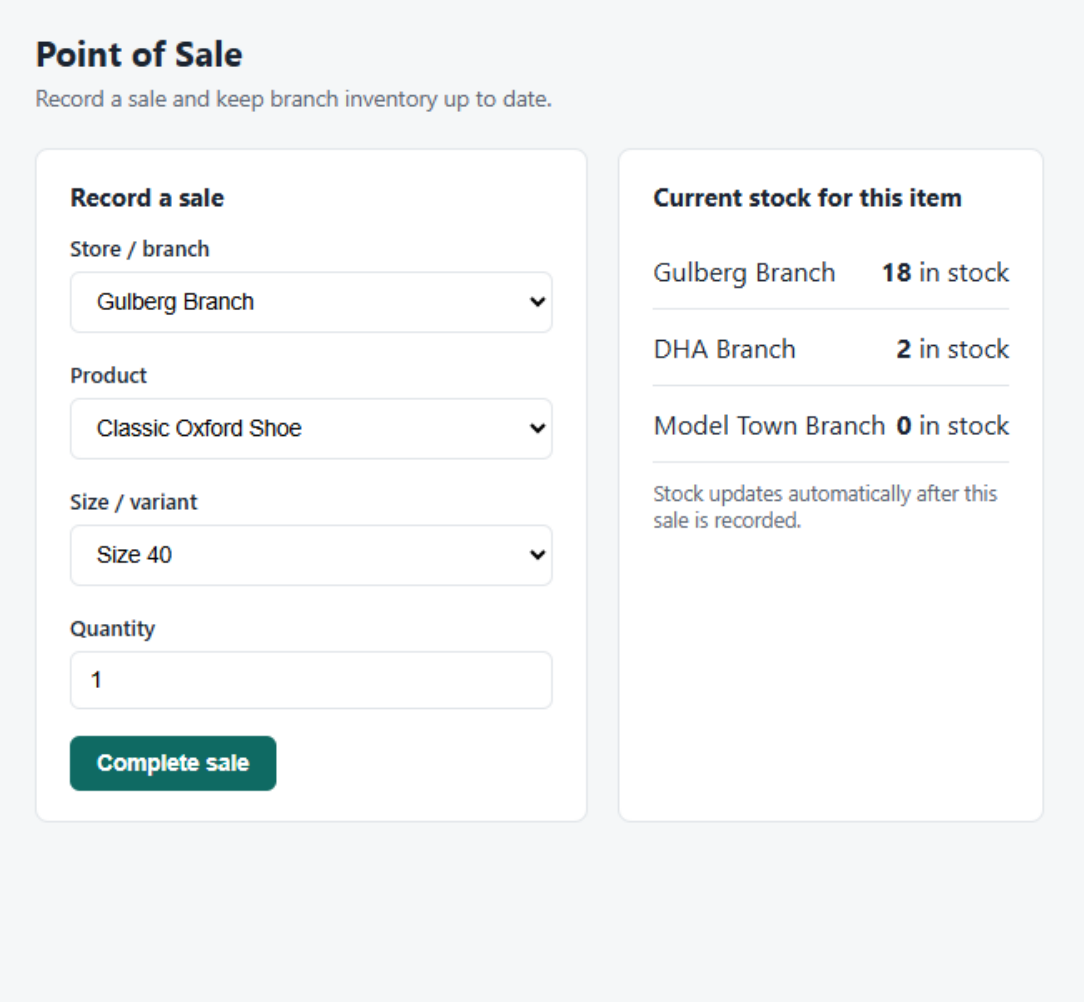


Figure 4.4: Point of Sale

4.8.5 Upload Sales Report

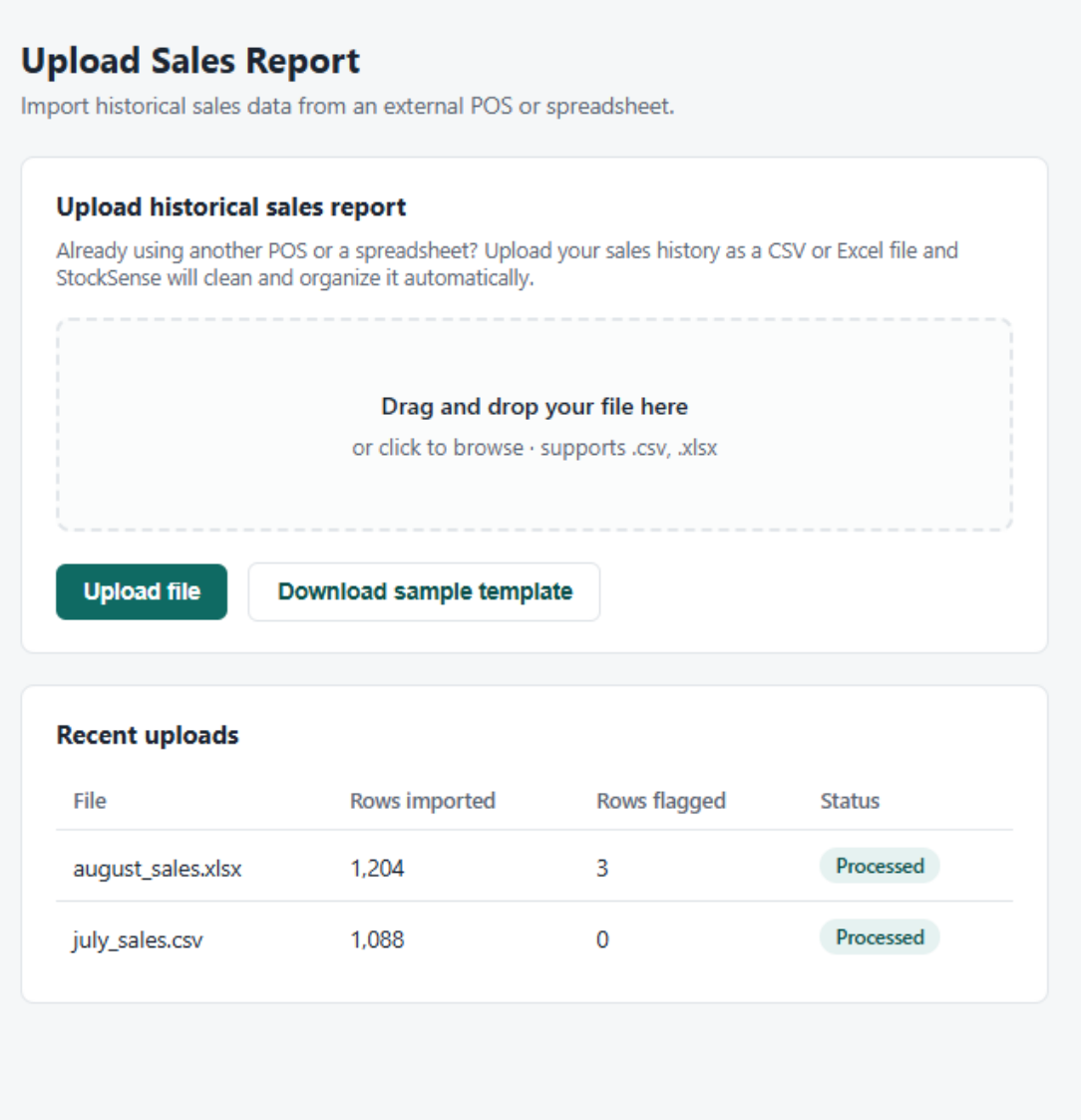


Figure 4.5: Upload Sales Report

Businesses already using an external POS or spreadsheet can upload their historical sales data as a CSV or Excel file from this page, which is then validated and cleaned by the system.

4.8.6 Inventory

This page displays current stock levels for every product variant across all branches, filterable by store, along with a status indicator for each item.

4.8.7 Recommendations

This page lists the system's redistribution, markdown, and reorder recommendations, each with a plain-language explanation and an estimated financial impact, which the user can approve or

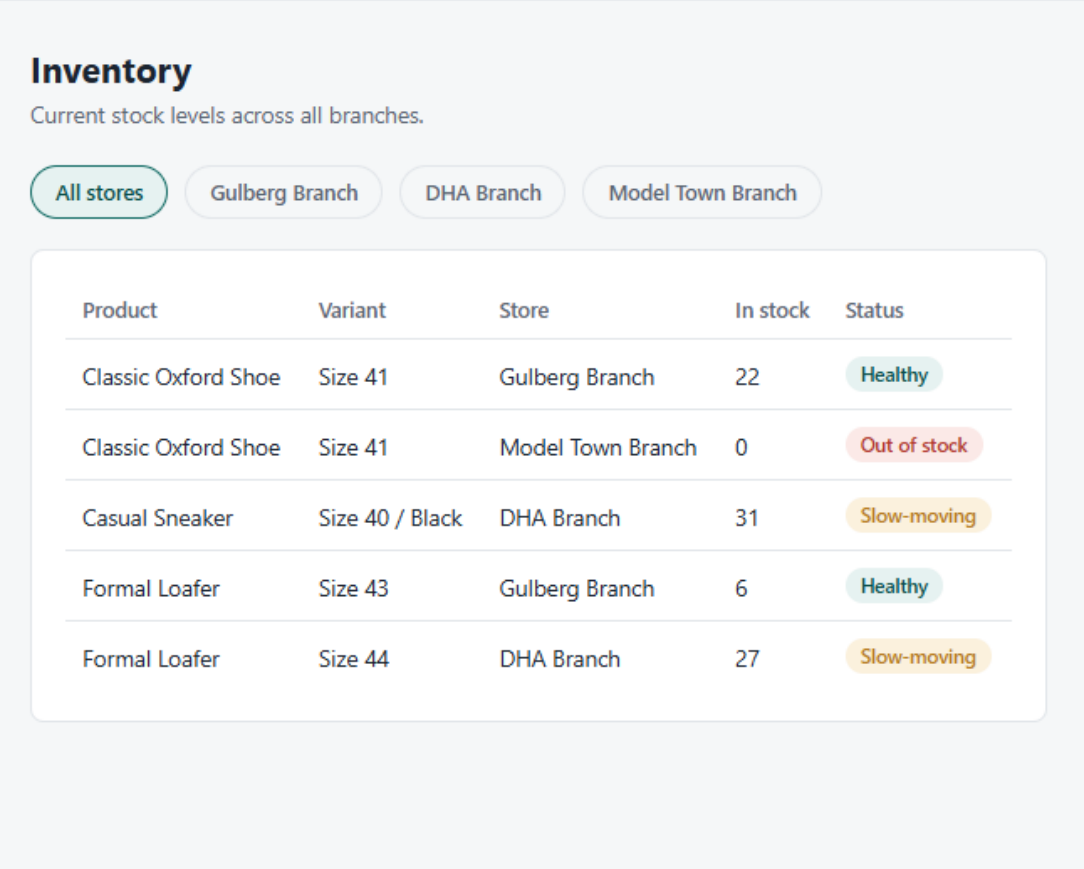


Figure 4.6: Inventory

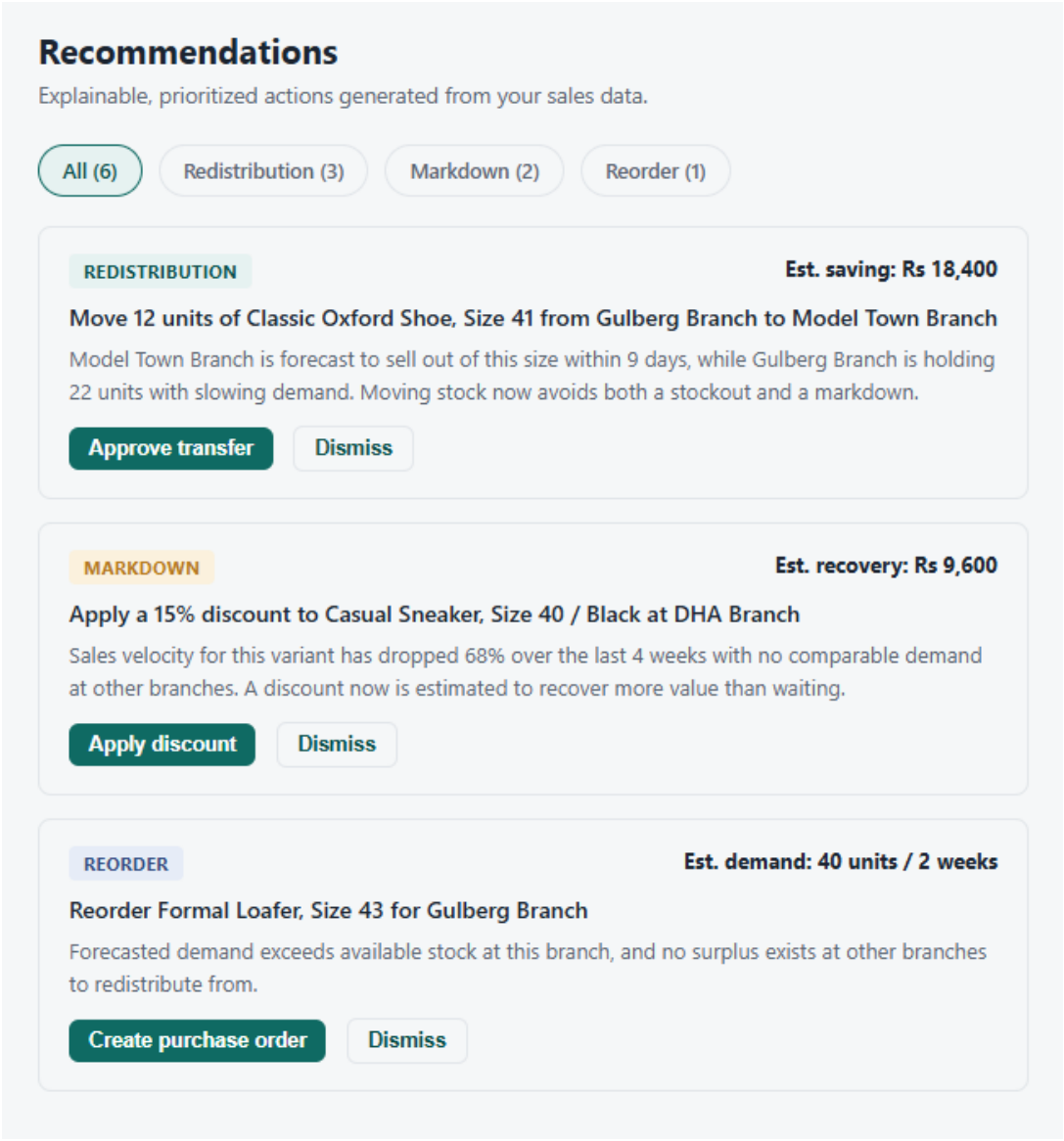


Figure 4.7: Recommendations

dismiss.

4.9 Database Design

4.9.1 ER Diagram

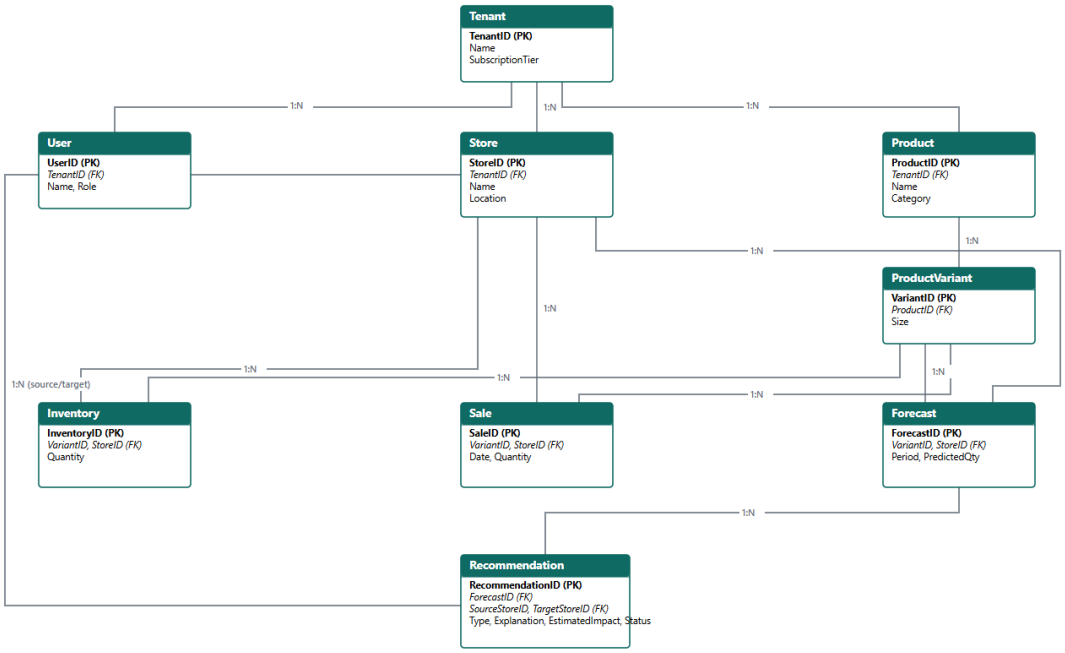


Figure 4.8: ER Diagram

This is the design of the StockSense database. Each Tenant maintains separate Users, Stores, and Products, so that all business data is kept separate. A Product is associated with a number of Variants (e.g. sizes) and Inventory keeps track of how many of each Variant a Store has. Sales records and Forecasts are created at the Variant level per Store; Recommendations (redistribution, markdown, and reorder) are made based upon Forecasts and refer to the Store(s) involved.

The major entities and their relationships are:

- A Tenant has many Users, Stores, and Products.
- A Product has many Product Variants (size/colour combinations).
- A Store maintains Inventory records per Product Variant.
- A Sale references a Product Variant, a Store, and a date/quantity.
- Historical Sales feed the Forecast entity, generated per Product Variant per Store.
- A Forecast is consumed by the Recommendation entity (redistribution, reorder, or markdown type).

4.9.2 Data Dictionary

Table 4.16: Data Dictionary for Tenant Table

Field	Type	Description
TenantID	Integer	Unique identifier for each business
Name	String	Business name
SubscriptionTier	String	Free or Paid
CreatedDate	Date	Date the tenant registered

Table 4.17: Data Dictionary for Store Table

Field	Type	Description
StoreID	Integer	Unique identifier for each store
TenantID	Integer	Owning tenant
Name	String	Store/branch name
Location	String	Store address or region

Table 4.18: Data Dictionary for Product Variant Table

Field	Type	Description
VariantID	Integer	Unique identifier for a product-size combination
ProductID	Integer	Parent product
Size	String	Size or variant label
Category	String	Product category

4.10 Risk Analysis

- Technical Risk: Forecast accuracy may be poor for tenants with very limited historical sales data.
- Data-Quality Risk: Inconsistent or incomplete uploaded sales reports may require significant cleaning effort.
- Adoption Risk: Small-business owners may be reluctant to trust automated recommendations without a visible track record.
- Scalability Risk: As more tenants and historical data accumulate, forecasting jobs must remain performant.
- Business Risk: Changing project requirements or limited availability of real business data for testing.

Table 4.19: Data Dictionary for Sale Table

Field	Type	Description
SaleID	Integer	Unique identifier for each sale
VariantID	Integer	Product variant sold
StoreID	Integer	Store where the sale occurred
Date	Date	Date of sale
Quantity	Integer	Units sold

Table 4.20: Data Dictionary for Recommendation Table

Field	Type	Description
RecommendationID	Integer	Unique identifier for each recommendation
Type	String	Redistribution, Reorder, or Markdown
VariantID	Integer	Product variant involved
SourceStoreID	Integer	Store with surplus (redistribution only)
TargetStoreID	Integer	Store with forecasted need
Explanation	String	Plain-language reasoning for the recommendation
EstimatedImpact	Float	Estimated financial impact
Status	String	Pending, Actioned, or Dismissed

4.11 Conclusion

In this chapter, you learned the functionality and non-functional requirements, use cases, and database design of StockSense. Requirements focus on three pillars identified in the Project Vision and Literature Review: affordable multi-tenant data collection using a free POS, product-size-location demand forecasting based on the literature for model selection, and explainable recommendations that focus on redistribution. The next chapter describes the proposed approach and methodology used to implement these requirements.

Chapter 5 Proposed Approach and Methodology

The proposed approach and methodology is presented in this chapter. The chapter explains how StockSense transforms raw sales data into inventory recommendations. It traces the data from ingestion to validation, construction of the demand series, forecasting, recommendation, optimization of markdowns, explanation, and evaluation. The literature reviewed in Chapter 3 is the basis for each design decision.

5.1 Overall Pipeline

StockSense is a sequential pipeline in which each stage reads data from and writes data to the PostgreSQL database through the modules described in Chapter 6. Sales are entered through the built-in POS or by file upload and are stored in one canonical schema. A demand series is created for each product-size-store combination, features are created from the history of the demand series, and the model provides weekly and monthly demand forecasts. These forecasts are then compared with the current inventory to generate redistribution, markdown, and reorder recommendations, all of which include an explanation and an estimated financial impact.

The forecast and recommendation operations are carried out as background processes and the results are put into the forecast and recommendations tables. `ModelName` is associated with each forecast, enabling different models to be compared or swapped without changing any other part of the system. In order to honor tenant isolation, the models are trained per tenant. The model will be used for all products, sizes, and stores of a business, but not between businesses.

5.2 Data Ingestion

Sales are created in two ways: through records of type `Sale` and `SaleItem` which are available from the standard POS (Source = POS) and through CSV/Excel files coming from an external source (Source = Upload). Both types have the same schema and therefore subsequent steps are independent of the source.

The CSV files are processed using chunks via pandas library, while the Excel files are processed in batches using pandas and `openpyxl` libraries in order to keep memory usage low for files containing up to 100,000 rows. The system shows the detected columns and the user maps them to canonical columns (product, size, store, date, quantity). Validation and import is done as a background task. Validated rows are buffered until the report is approved by the owner and then imported in a single database transaction. In case of failure, the transaction is rolled back.

5.3 Data Validation, Cleaning, and Normalization

The following validations are run on each record:

- Structural: all mandatory attributes are checked to be available with the correct datatype.
- Value: the date is correct, the quantity attribute is numeric, and the price is greater than or equal to zero. Dates in the form of 03/04/2025 are ambiguous; hence the owner has an option to select the date format while mapping columns.
- Referential: the name of the product, size, and store should be normalized (truncated, case insensitive, and size should be standardized like "Medium" to "M"), and should match a variant and a store under the same tenant.
- Duplicated records: Duplicate records are found through the use of a composite key composed of the store, variant, date, quantity, and price. They are identified but will not be deleted since identical sales transactions may occur in reality.
- Returns and outliers: Negative values of quantity are considered returns, while extreme values of quantity are identified and capped only on the training copy.

The accepted records are saved in the standard format. The rejected and flagged records are included in the upload report with corresponding justifications.

5.4 Demand Series Construction

Sales data are cleaned and aggregated to product/size/store weekly and monthly levels. Weekly sales forecasts are made using weekly series; monthly sales forecasts are made using monthly series, so that the forecast horizon and evaluation period have the same time resolution.

$$y(v,s,t) = \sum_{i \in \mathscr{S}(v,s,t)} q_i, \tag{5.1}$$

where $\mathscr{S}(v,s,t)$ is the set of sale items of variant v in store s during period t, and q_i is the quantity of sale item i.

A period with no sales is assumed to represent zero demand only starting from the date when the variant appeared in this store, since there is no demand prior to that date, not necessarily zero.

Series at size level are often sparse and intermittent. Therefore, each series is classified as either smooth, erratic, intermittent, or lumpy according to the Syntetos–Boylan classification by the average inter-demand interval (ADI) and squared coefficient of variation of non-zero demands (CV^2) thresholds, e.g. $\mathrm{ADI} = 1.32$ and $\mathrm{CV}^2 = 0.49$. This aids in choosing baseline methods and

allows reporting forecasting accuracy for different demand types.
If a stock-out occurs, observed sales underrepresent demand. If stock history is known, weeks in which the variant was unavailable most of the time are considered as censored and ignored in the training loss.

5.5 Feature Engineering

Each data sample contains information about one particular variant in one shop at some forecast starting point t, where the demand at point $t+h$ is the target variable. There are several groups of features:

- Lag features: demand at lags 1 to 8 weeks, and at 52 weeks if at least one year of history is available (level, seasonality per year).
- Rolling statistics: average, standard deviation, and fraction of zero sales weeks during the last 4, 8, and 12 weeks (level, volatility, intermittency).
- Calendar features: week number, month, and holidays and festivals flags (seasonality, events).
- Price features: selling price, discount depth, and price relative to the category average (price effect).
- Product and store attributes: category, size, colour, store, and city, used as categorical features (sharing patterns across similar series).
- Lifecycle features: weeks since first sale and weeks since last sale (product age).

To prevent data leakage, features at origin t use only data available up to t. Only features that are known in advance, such as calendar events, may refer to the future.

5.6 Demand Forecasting Model

5.6.1 Global Gradient-Boosted Model

The individual size-level series are too short and sparse to support one model per series. Following the global models of Montero-Manso and Hyndman and the forecasting of groups of similar series by Bandara et al., StockSense builds one global model per tenant on all of its series, with product, size, and store as categorical features. The chosen model is LightGBM (gradient-boosted decision trees). Gradient-boosted trees performed strongly against several statistical methods in the M5 competition (Makridakis et al.), and LightGBM handles missing values and categorical variables natively and trains quickly on tabular data.

5.6.2 Loss Function and Forecast Horizon

Demand is non-negative and contains many zeros, so squared error is not a suitable loss. The model uses the Tweedie objective (power between 1 and 2), which models a point mass at zero together with a positive continuous part. The Poisson objective and squared error are retained as alternatives and compared experimentally.

Multi-step forecasts use a direct approach: a separate model is trained for each step $h = 1, \ldots, H$ of the forecast horizon, so errors do not accumulate through recursive predictions:

$$\hat{y}(v, s, t+h) = f_h\big(\mathbf{x}_{v,s,t}\big), \tag{5.2}$$

where $\mathbf{x}_{v,s,t}$ denotes the features of variant v in store s at time t.

5.6.3 Training and Tuning

The hyperparameters (number of leaves, learning rate, minimum data per leaf, feature fraction, bagging fraction, and boosting rounds) are tuned with rolling-origin validation, using an expanding training window and early stopping on each fold. Random shuffling is never used, because it would expose future data to the model. Sample weights exclude censored periods from the loss.

5.6.4 Statistical Baselines

To evaluate whether the more complex model is necessary, the following baselines are computed on the same splits: the naive forecast (last value), the seasonal naive forecast (with at least one year of history), the moving average, Holt–Winters exponential smoothing (`statsmodels`), and Croston's method for intermittent series.

5.6.5 Initial Training Period

StockSense does not ship with a pre-trained model, and the data of one tenant is never shared with another. A new tenant therefore starts without a forecasting model. The owner first records sales through the POS or uploads historical sales reports. Until a minimum amount of history is available (for example, several weeks of sales for the tenant's products), no model-based forecast is displayed and the user is told that more data is needed (UC-10). During this period, simple baseline forecasts may be shown if enough observations exist. Once the minimum is reached, the first global model is trained on the tenant's own data, and model-based forecasting and recommendations start working. The model is then retrained on a schedule as new sales arrive;

accuracy is measured rather than assumed to improve as history grows. The minimum history is determined experimentally (Section 5.10).

5.6.6 Forecast Uncertainty

In addition to the point forecast, an upper-quantile forecast (for example, the 0.9 quantile using the quantile objective of LightGBM) is trained. The recommendation engine uses it to establish safety stock, so that decisions account for uncertainty and not only the mean.

5.7 Recommendation Engine and Stock Redistribution

For every variant v and store s, the engine compares the forecast with the stock position over a planning horizon of H weeks. Define:

- I = on-hand stock plus pending incoming transfers (pending transfers are already accepted, so they are not recommended again),
- D = forecast demand over the next H weeks,
- $Q_{0.9}$ = the upper-quantile forecast over the horizon, and $SS = \max(0, Q_{0.9} - D)$ = safety stock above the expected demand.

The net position is therefore

$$N = I - (D + SS). \tag{5.3}$$

If $N > 0$, the store has a surplus of N units; if $N < 0$, it has a shortage of $-N$ units.

Redistribution. Following the proactive-transshipment approach of Paterson et al., transfers are planned before a stock-out or dead stock occurs. For each variant, the transfer quantities x_{ij} from surplus store i to shortage store j are obtained by solving a transportation problem:

$$\max_{x} \quad \sum_{i}\sum_{j} \left(m - c_{ij}\right) x_{ij} \tag{5.4}$$

$$\text{s.t.} \quad \sum_{j} x_{ij} \leq \text{surplus}_i \quad \forall i, \tag{5.5}$$

$$\sum_{i} x_{ij} \leq \text{shortage}_j \quad \forall j, \tag{5.6}$$

$$x_{ij} \geq 0. \tag{5.7}$$

Here m_{ij} is the relevant unit margin at destination store j and c_{ij} is the unit transfer cost between stores i and j. A transfer is recommended only when $m_{ij} - c_{ij} > 0$. Transfer quantities are constrained to non-negative integer units; because the transportation matrix and the supply and demand values are integral, the linear-program solution has an integral optimum. Since

retailers have few branches, each problem is small and cheap to solve.The model is currently missing the transfer lead times and capacity constraints, which are captured as assumptions during the backtest.

Reorder. If there is still a shortage left after the redistribution phase, this is treated as a reorder suggestion with a recommended order quantity.

Priority. The recommendations are handled according to the priority redistribution, markdown, reorder since redistributing the existing inventory is easier and cheaper than discounting or reordering.

5.8 Markdown Optimization

Any excess stock not removed through redistribution is given to the markdown optimization model. This too works in a forecast-then-optimize framework, like Caro and Gallien's method, but is intended to serve as an early warning system rather than season-end clearance method. The condition that causes a variant-store to be declared in danger is that its weeks of cover is higher than the selling time left. In case of positive forecasted demand, weeks of cover is computed as:

$$\text{WOC} = \frac{I}{\bar{d}} > T, \tag{5.8}$$

where $\bar{d}$ is the average weekly demand and T is the number of weeks that the product needs to be sold. If $\bar{d} = 0$ and $I > 0$, the item is treated as at risk and handled separately because its weeks of cover is undefined. The candidate discounts are evaluated against expected margin and sell-through; the smallest discount meeting the target is selected only when it does not produce a lower expected margin than the no-discount case.

For each candidate discount d from a grid (for example, 5% to 50% in steps of 5%), demand is adjusted using a constant price-elasticity model:

$$D(d) = D \cdot (1-d)^{-\varepsilon}, \qquad \text{Sold}(d) = \min\big(I,\, D(d)\big), \tag{5.9}$$

where $D(d)$ is the total adjusted demand over the horizon.

The system recommends the smallest discount for which the projected sell-through reaches a target, subject to a margin floor $p\,(1-d) \geq c$, where p is the selling price and c is the cost price. The impact measure is the difference in margins between the two states of using the discount:

$$\text{Impact} = \text{Sold}(d)\,\big(p(1-d) - c\big) - \text{Sold}(0)\,\big(p - c\big). \tag{5.10}$$

The elasticity ε is measured through a log-log regression on the historical prices of the tenant's product if there is enough variability in prices; otherwise, a cautious category-level estimate is used.

5.9 Explainable Recommendations

Each recommendation is saved along with the two parts of an explanation:

- A list of numbers relating to the situation that is relevant: forecast demand, stock level, net position, weeks of supply, source and destination stores, margins, costs of transferring products, impact of the recommendation.
- Forecast drivers, or the contributions from LightGBM (`pred_contrib` in SHAP-style, plain English phrases like "recent weekly demand" or "seasonality".

All of these two pieces are placed in textual templates, such as the following: "The Size M variant of this item will be sold 12 pieces at Store B during the next 4 weeks, but there are 3 pieces in stock; there are 15 pieces at Store A that will be sold 4 pieces over the next 4 weeks. This explanation is retained on creation and is always correct and matches the situation that led to the recommendation.

5.10 Forecast and Recommendation Evaluation

5.10.1 Data

Public retail data sets are employed for preliminary testing and validation. A dataset such as M5 contains store-item sales and prices, which supports evaluation at the store level and the hierarchy level, but it does not contain size-level sales. Size-level behaviour is therefore evaluated on real retailer sales data if access is secured, and the findings from public data are interpreted with this limitation in mind, as stated in Chapter 2.

5.10.2 Protocol

All splits are chronological. Hyperparameters are tuned with rolling-origin validation, and the final comparison is made on the most recent period, which the models never see during training or tuning.

5.10.3 Metrics

The primary metrics are

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{k=1}^{n}\left(a_k - f_k\right)^2}, \tag{5.11}$$

$$\text{MAPE} = \frac{100}{|P|}\sum_{k\in P}\frac{|a_k - f_k|}{a_k}, \qquad P = \{k : a_k > 0\}, \tag{5.12}$$

where a_k is the actual demand and f_k is the forecast.

Size-level series contain many zeros and MAPE is undefined for zero actuals, so non-zero-demand MAPE is computed only on periods with positive actual demand, together with the proportion of periods excluded. WAPE (total absolute error divided by total actual sales) and MAE are emphasized as supplementary measures, since they remain valid for sparse series. Results are presented separately for each demand class, for fast and slow movers.

5.10.4 Experiments

- LightGBM against the statistical baselines (the main claim of Chapter 3).
- The global model against separate models for each series.
- Forecast accuracy against the amount of training history available (for example, 4, 8, and 12 weeks), to estimate how much data a new tenant must provide before forecasts become reliable.
- Tweedie and Poisson objectives against squared error.

5.10.5 Recommendation Backtest

The value of the recommendations is estimated by replaying history. At each cut-off date, the engine produces recommendations using only the information available up to that date. The recommended transfers and discounts are applied to a simulated stock record, and the following weeks are replayed with actual sales as demand. The simulation specifies transfer lead time, assumes that stock is unavailable at the source while it is in transit, and applies an accepted markdown from its effective date. The outcomes are compared with a “no action” run and include dead stock (units whose cover is greater than T weeks), units of lost sales caused by stock-outs, and the sell-through rate.This is based on the fact that in the simulation process the transfer does not affect the demand, while the demand changes using the elasticity method if there is a markdown.

5.11 Iterative Development

StockSense development occurs incrementally, and at the end of each increment comes testing activities as detailed in Chapter 6, along with a review by the supervisor:

1. Multi-tenant infrastructure: tenancy, users, products, inventory, POS, and the ingestion and validation pipeline.
2. Experiments on forecasting in Jupyter Notebook: feature engineering, baselines, and LightGBM – all done offline.
3. Integration: forecasting done in batch, recommendation, and redistribution of items.
4. Optimization of markdown, explanations, notifications, and dashboards.

Development of the forecasting models offline first gives an opportunity to evaluate the model chosen using data.

5.12 Conclusion

A decision tree begins with each transaction. The dataset is then validated and standardized, and a global LightGBM model is trained to predict the product-size-store level demand, which is then compared with some statistical models. For the new tenant, the sales data is given first to train the model on the sales history of the new tenant and once trained, forecasting begins. The next stage of the pipeline is to translate these forecasts into actions, in order: redistribution via a small transportation problem; early markdown via the sell-through model; and reordering for the remaining shortage. Explanation and estimated impact of each action. Evaluation was done using a chronological split of the data and replay of past data to assess the forecast accuracy and dead stock reduction potential.

Chapter 6 High-Level and Low-Level Design

6.1 System Overview

StockSense is a multi-tenant SaaS solution for small and medium-sized retailers to minimise dead stock and stock out in their stores, leveraging their own sales data. Every business is an independent tenant and has its own users, stores, products, inventory and sales. The sales are made using the built-in POS system or via CSV/Excel files which can be uploaded. The system predicts demand at product-size-store level and suggests actions with an accompanying clear explanation on a single web dashboard. These will be the modules that will be incorporated into our system:

6.1.1 Tenant and User Management

This module processes the registration of a business, user logon and roles. Ensures that an owner, store manager and/or administrator can only access information for which they have the authority and tenant.

6.1.2 POS and Inventory Management

This module is the free part of StockSense. It records sales and keeps the current stock of every product, size, and store up to date.

6.1.3 Data Ingestion and Cleaning

This module imports historical sales data from CSV or Excel files, validates the data, identifies the invalid records and formats the data into a consistent standard format. It allows businesses that already have a point of sale or spreadsheet to participate without having to change their system.

6.1.4 Demand Forecasting

This module forecasts the weekly and monthly demand of each product, size and store. It trains a model on each tenant's own sales history, so a new tenant must first record or upload enough sales data before forecasts are shown.

6.1.5 Stock Redistribution and Reorder Recommendations

This module is a comparison of the forecast and the current stock of each branch. It recommends moving stock from a branch with more stock to one with less stock first and suggests a reorder

only if the amount of stock is insufficient being moved.

6.1.6 Markdown Optimization

The module identifies slow moving stock and suggests selling the stock at an early stage prior to it being dead.

6.1.7 Explainable Recommendations

Each recommendation is accompanied by a brief description of why it is recommended, and an approximate financial impact so the owner can comprehend and commit to the recommendation.

6.1.8 Dashboard and Reporting

This module displays the sales, inventory, forecasts, and recommendations to the Owner and the Store Managers in an easy to use web interface.

6.2 Design Considerations

This section lists the dependencies and limitations that affect the design of StockSense, so that they can be handled before the implementation starts.

6.2.1 Assumptions and Dependencies

6.2.1.1 Operating System and Browser

StockSense is a web application. Users must be running a relatively new version of Chrome, Edge, Firefox or Safari on their laptop, desktop or tablet.

6.2.1.2 End User Characteristics

- Users have a stable internet connection.
- Owners/managers have not mastered computers but know some of the basic uses.
- The retailer can access historical sales data or can record new sales in the built-in, point-of-sale system.

6.2.1.3 Possible Changes in Functionality

The quality of forecasts depends on the quality and amount of sales data. During development, the functionality may change as a result of testing of the datasets and feedback from the supervisor. That is the intent of the modular design to facilitate such change.

6.2.2 General Constraints

6.2.2.1 Hardware and Software Environment

- More computing power is required for forecasting and data processing (than for normal page requests).
- The interface needs to be compatible with various devices and browsers.
- There may be constraints on what we can do when implementing with external libraries or service.

6.2.2.2 End-User Environment

- Slow or unstable internet in some shops.
- Many managers have no training in data analysis and the interface should not be technologic.

6.2.2.3 Data Format Differences

- Uploaded files vary in column names, date formatting and product naming.
- All of them are to be converted to a single standard internal format.

6.2.2.4 Data Repository and Tenant Isolation

- Never any one business should have access to another business data.
- All records should be associated with tenant.
- The database needs to expand to include additional tenants and additional sales history.

6.2.2.5 Security Requirements

- It is user's responsibility to log in with their own username and password, and to keep their password protected.
- Access should be based on user role.
- Data needs to be safeguarded whilst in transit and whilst stored.

6.2.2.6 Performance Requirements

- Forecasting and large uploads have to be performed in the background.
- The results with frequent use, like the latest forecasts, are stored and do not need to be calculated time and time again.
- Clearly error messages should be produced for invalid input.

6.2.2.7 Interoperability Requirements

- The system needs to be able to import CSV and Excel files from external point-of-sales (POS) systems and spreadsheets.
- The internal modules should communicate using unambiguous interfaces.

6.2.2.8 Verification and Validation Requirements

- It is important to test the upload pipeline with bad and garbled files.
- Tenant separation is necessary to be tested for data leakage.
- Predictions need to be judged against past data, as described in Chapter 5.

6.2.3 Goals and Guidelines

6.2.3.1 The KISS Principle

The design must remain simple enough in order to allow for easy maintenance and usage even by non-technical users.

6.2.3.2 Separation of Tenant Data

Each company needs its own secure data room.

6.2.3.3 Modularity and Extensibility

Forecasting, recommendations, POS, ingestion and markdown should be segregated to allow for the addition of different forecasting algorithms without altering the system design.

6.2.3.4 Explainability

Every recommendation should have the rationale behind it.

6.2.4 Development Methods

The StockSense application will be created iteratively in phases. The initial phase would contain the modules for tenant, product and inventory management and data ingestion. The second phase will include forecasting, recommendations, markdown optimization and dashboard creation. An evaluation would be done for every phase before moving to the next phase.

6.3 System Architecture

StockSense has a multi-layered web structure. The users interact with the web dashboard or the POS interface, which will communicate with the API in the back end. The API verifies the identity of the user, their tenant and passes the request to the appropriate module. The cleaned sales data feeds into a forecasting engine, while the forecasts are fed into a recommendation engine and the valid stock data from the POS and inventory module feeds into a recommendation engine. The slow moving stock is passed to the markdown optimizer. All the results are recorded in the database and displayed back to the user at the dashboard. The full picture is displayed in Figure 6.1.

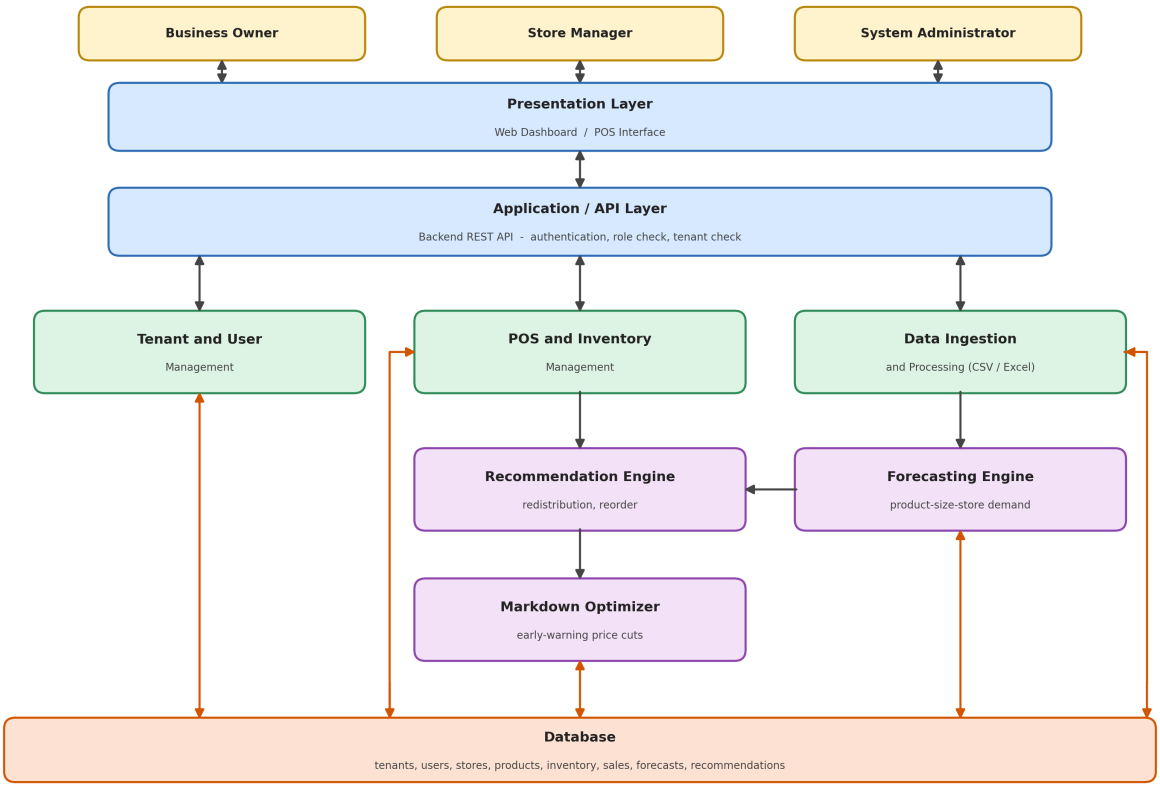


Figure 6.1: System Architecture Diagram
This figure shows the System Architecture Diagram of our Application

6.3.1 Subsystem Architecture

6.3.1.1 User and Tenant Management

This subsystem contains business and business users. It supports roles, login and registration, and ensures that every user only has access to data from his/her tenant.

6.3.1.2 POS and Inventory Subsystem

This subsystem maintains the current item count for each item, size and store, and documents the total sales for each item, size, store.

6.3.1.3 Data Ingestion Subsystem

This subsystem is able to accept CSV and Excel files, validate them, flag invalid records and convert files to the standard format used internally.

6.3.1.4 Forecasting Subsystem

This subsystem analyzes the sales history of a tenant and forecasts the weekly and monthly demand at product-size-store level. It trains a model on the tenant's own data, and no forecast is shown until enough history has been recorded or uploaded.

6.3.1.5 Recommendation Subsystem

This subsystem is a comparison of forecasts with the level of each branch. It advises to move stock from one branch to another first, and then recommends a reorder when this is not sufficient. These recommendations along with the reason and estimated financial impact are stored with each recommendation.

6.3.1.6 Markdown Optimization Subsystem

This subsystem is looking for stock that is not selling fast enough and advises a reduction in price early enough that it can be sold.

6.4 Architectural Strategies

6.4.1 Multi-Tenant Architecture

Many companies will share the same platform but the data will remain logically separate. All records that are important will have a tenant identifier on them, and all requests will be compared to it before reading or updating any data.

6.4.2 Modular Architecture

The modules will be forecasting, recommendations, POS, data ingestion, markdown optimization. If one of them changes (e.g. a new forecasting model), then the others should not need to

change significantly.

6.4.3 Programming Languages and Frameworks

The web interface will be created using React. The backend REST API will be developed in python (FastAPI or Django), the main model for forecasting will be lightgbm, the statistical baseline will be statsmodels and pandas and openpyxl will be used for processing of csv and excel files.

6.4.4 Database

A PostgreSQL relational database will store all business data. Each table in a business has a business tenant identifier, which is helpful for tenant isolation covered in Section 6.4.1. The detailed design will be provided in the database design part of chapter 4.

6.4.5 Background Processing

Forecasting and cleaning of large files may be time-consuming. These jobs will continue to run in the background and the user will be told when the results are available so the dashboard remains speedy.

6.4.6 Error Detection and Recovery

The system will record the following information regarding failed uploads, rejected records and failed forecasting runs to help identify the cause: In case a forecasting run is unsuccessful, previous valid forecasts will remain and not be eliminated from the forecasting dashboard.

6.5 Domain Model/Class Diagram

The domain model of StockSense is shown in Figure 6.2. It follows the data dictionary of Chapter 4: every business-owned class belongs to exactly one Tenant, and the foreign keys of the database are expressed as associations between classes.
The major classes of the system are:

- Tenant and Subscription: a business on the platform and its free/paid plan history.
- User: Business Owner, Store Manager, or System Administrator (distinguished by the `role` attribute).
- Store: a branch of a business.
- Product and ProductVariant: a product and its sizes/variants, each with its own SKU.

- Inventory: the quantity of one variant held at one store.
- Sale and SaleItem: a customer purchase (either POS-based or uploaded) and its variations.
- UploadJob: an uploaded CSV/Excel file with its accepted and rejected row counts.
- Forecast: the predicted demand of one variant at one store for a weekly or monthly period.
- Recommendation (abstract) with the subclasses RedistributionRecommendation, ReorderRecommendation, and MarkdownRecommendation.
- StockTransfer: the movement of stock between two stores, created when a redistribution recommendation is accepted.
- Notification: an alert sent to a user when a recommendation is raised.

The basic relationships between these classes are as follows:

- A Tenant has one or more Subscriptions over time, and owns many Stores, Products, and UploadJobs.
- A Tenant has many Users; a System Administrator belongs to no Tenant.
- A Store Manager is assigned to a Store; a Store can have many Users assigned to it.
- A Product has one or more ProductVariants (sizes).
- Inventory links one ProductVariant to one Store with a quantity.
- A Sale occurs at one Store and contains one or more SaleItems; each SaleItem refers to one ProductVariant. A Sale may come from an UploadJob (null for POS sales).
- A Forecast is generated for one ProductVariant at one Store for a period.
- A Recommendation concerns one ProductVariant, refers to a source Store, and may be based on a Forecast. It is accepted or dismissed by a User.
- A RedistributionRecommendation also has a destination Store and, when accepted, triggers one StockTransfer between two Stores.
- A Notification is received by one User and may refer to a Recommendation.

6.6 Sequence Diagrams

This section presents one sequence diagram for each use case described in Chapter 4. Each diagram shows the order of messages exchanged between the actor, the user interface, the application layer (API / controller or service), and the database for the basic flow of the use case. Alternative flows are drawn as `alt` fragments, optional behavior as `opt` fragments, repeated steps as `loop` fragments, and operations that must succeed or fail together as a single database transaction. Every request is restricted to the data of the user's tenant, as described in the multi-tenant architecture.

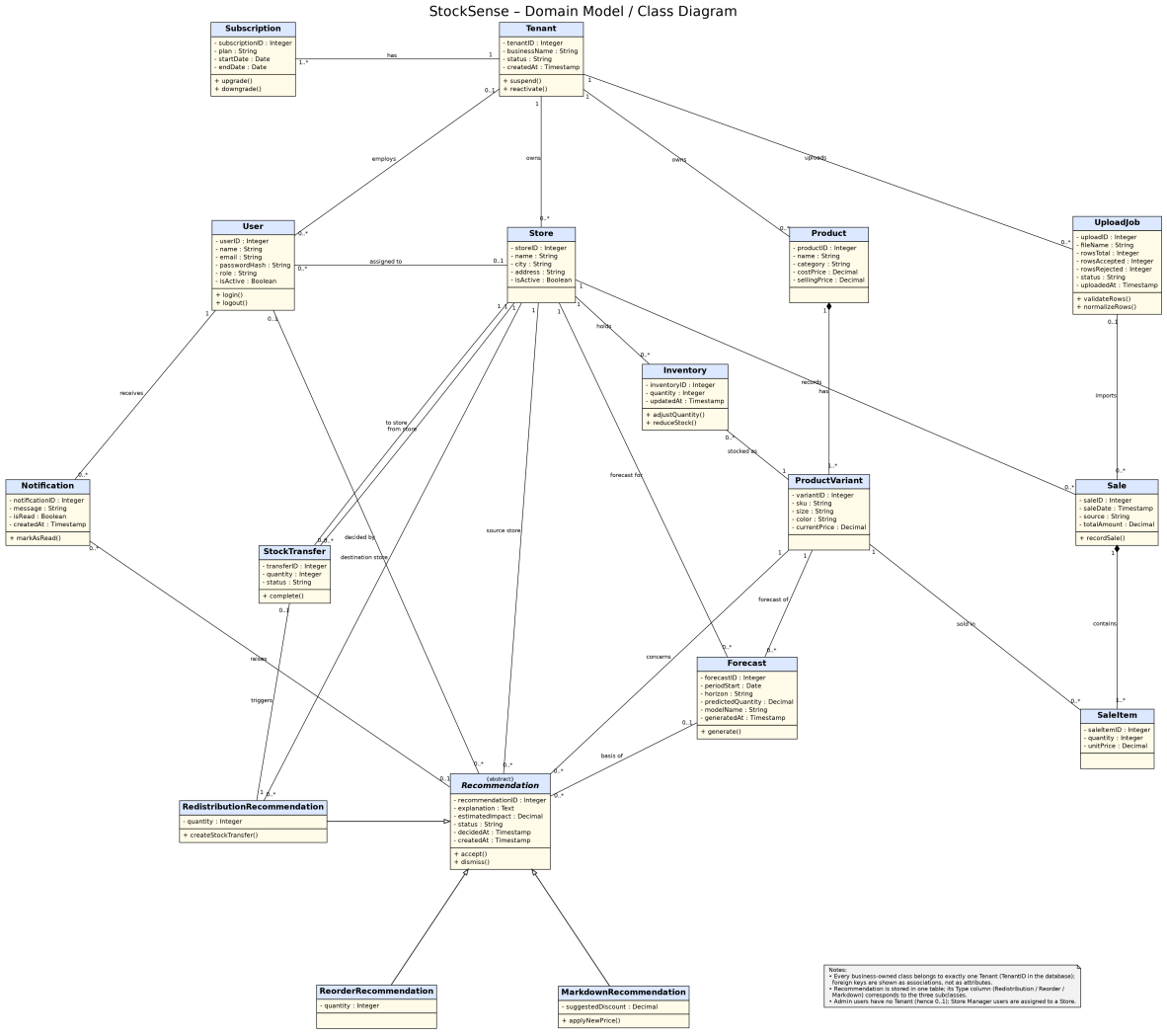


Figure 6.2: Domain Model / Class Diagram
This figure shows the Domain Model / Class Diagram of our Application

6.6.1 UC-01: Register Business

This diagram shows the interaction for the use case Register Business (UC-01).

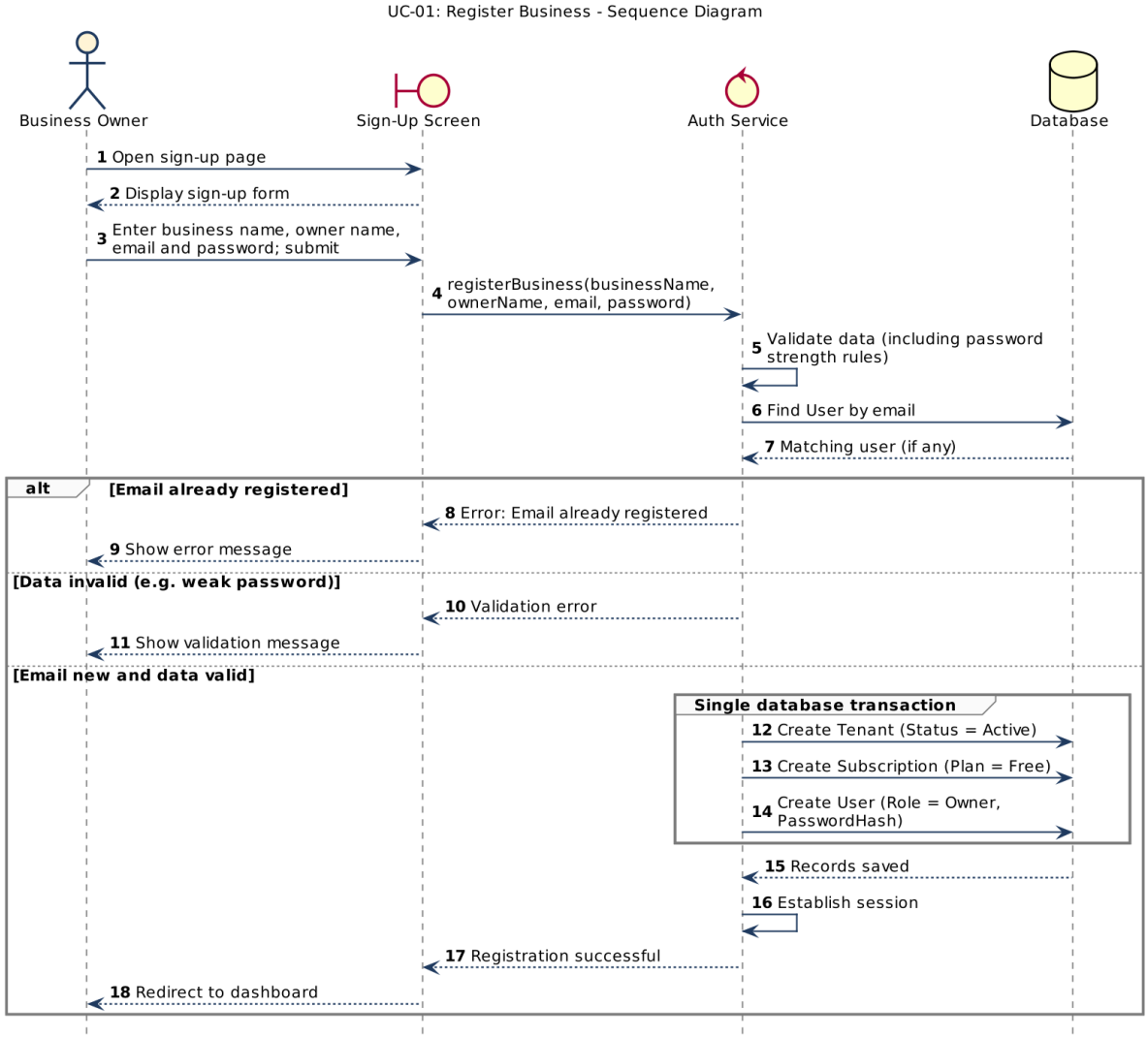


Figure 6.3: Sequence Diagram: Register Business
This figure shows the sequence diagram of UC-01 (Register Business)

6.6.2 UC-02: Login

This diagram shows the interaction for the use case Login (UC-02).

6.6.3 UC-03: Manage Stores

This diagram shows the interaction for the use case Manage Stores (UC-03).

6.6.4 UC-04: Manage Store Managers

This diagram shows the interaction for the use case Manage Store Managers (UC-04).

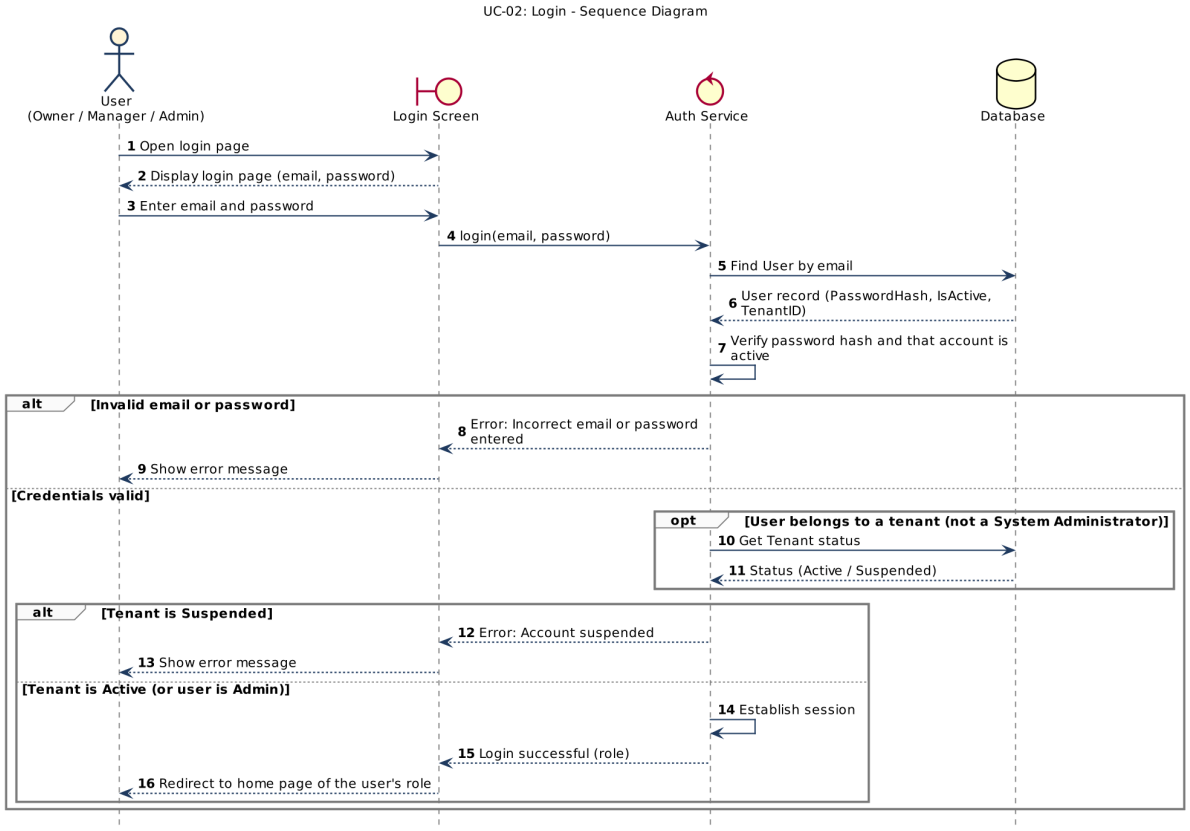


Figure 6.4: Sequence Diagram: Login
This figure shows the sequence diagram of UC-02 (Login)

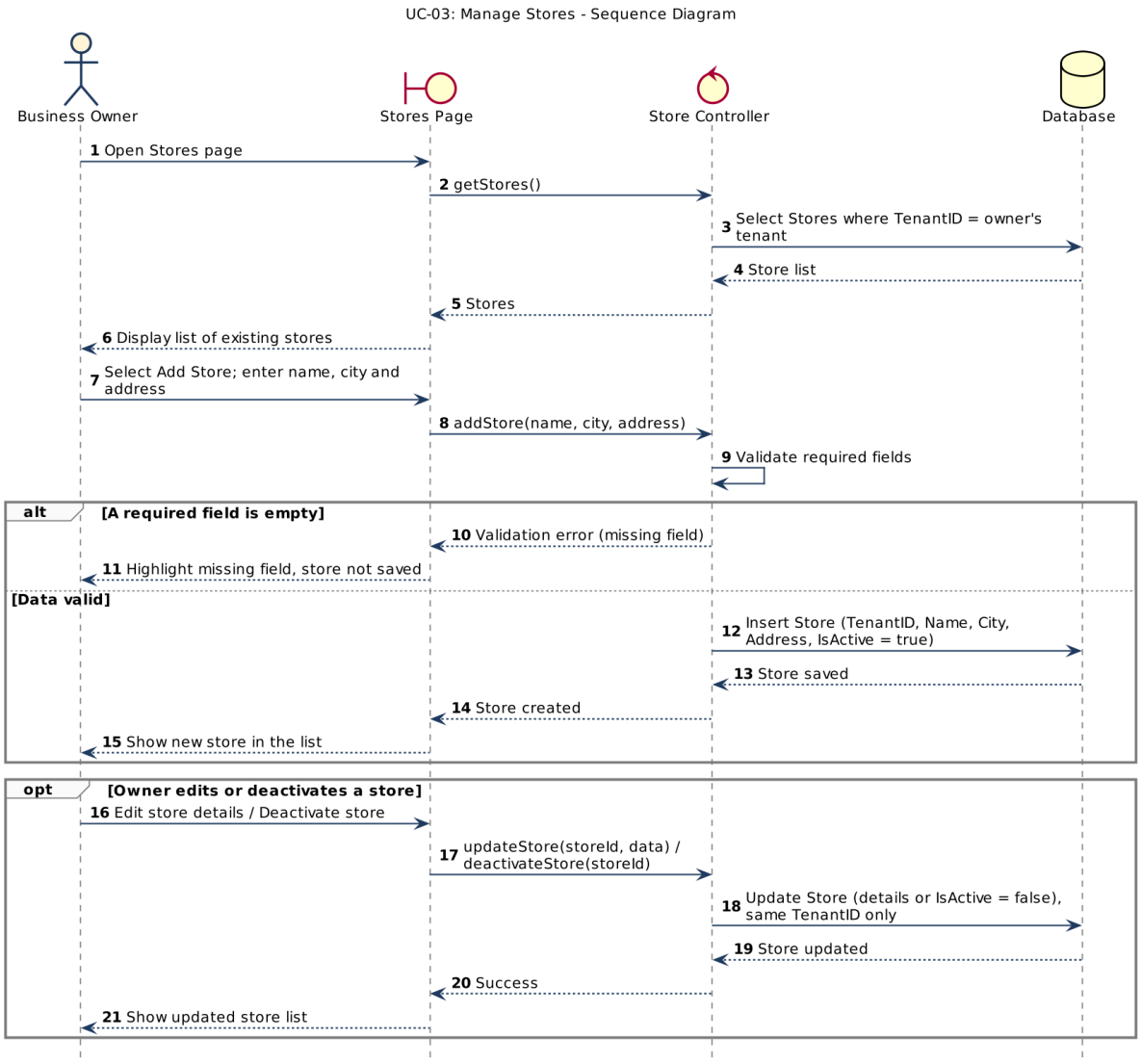


Figure 6.5: Sequence Diagram: Manage Stores
This figure shows the sequence diagram of UC-03 (Manage Stores)

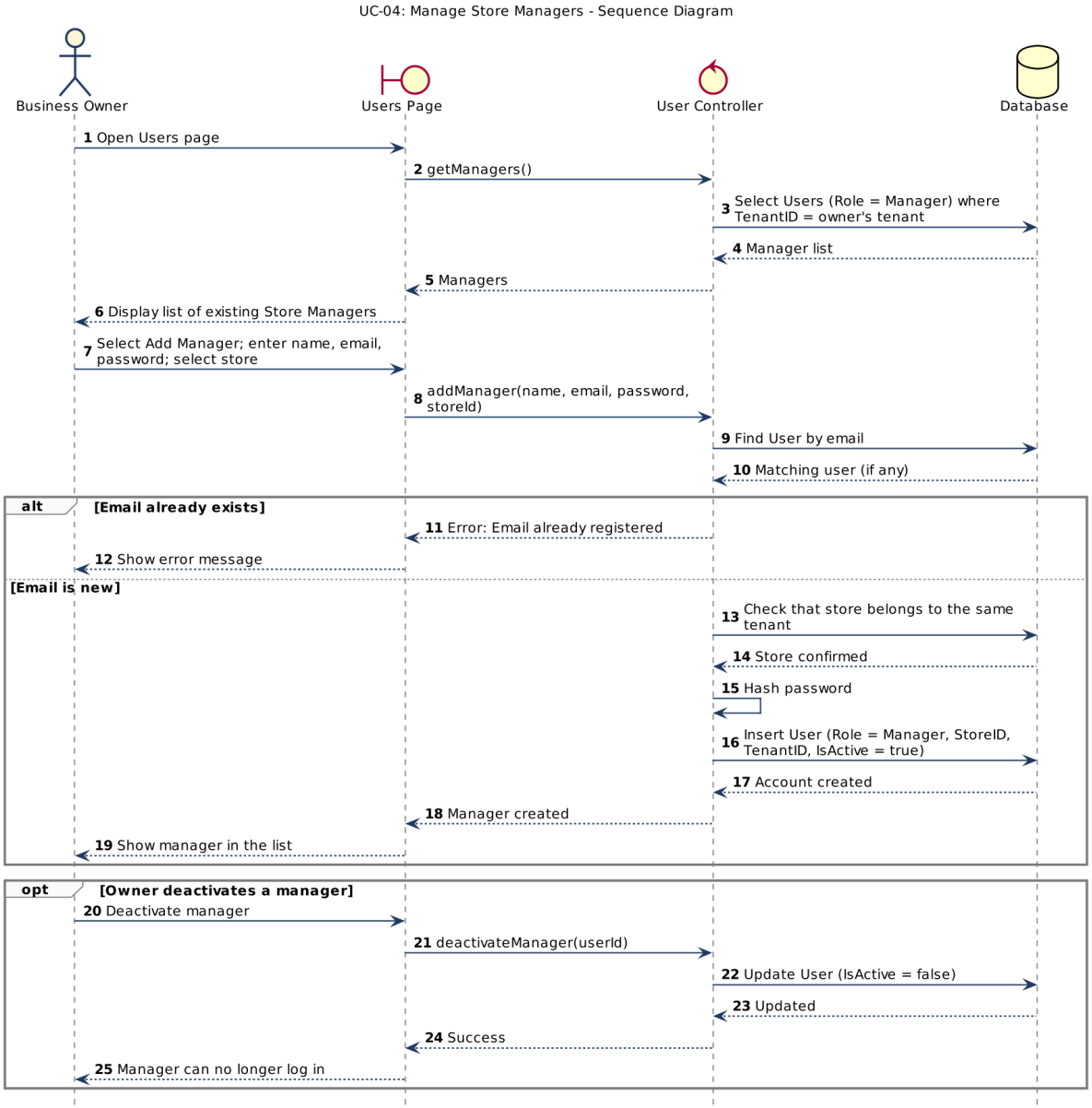


Figure 6.6: Sequence Diagram: Manage Store Managers
This figure shows the sequence diagram of UC-04 (Manage Store Managers)

6.6.5 UC-05: Manage Products and Variants

This diagram shows the interaction for the use case Manage Products and Variants (UC-05).

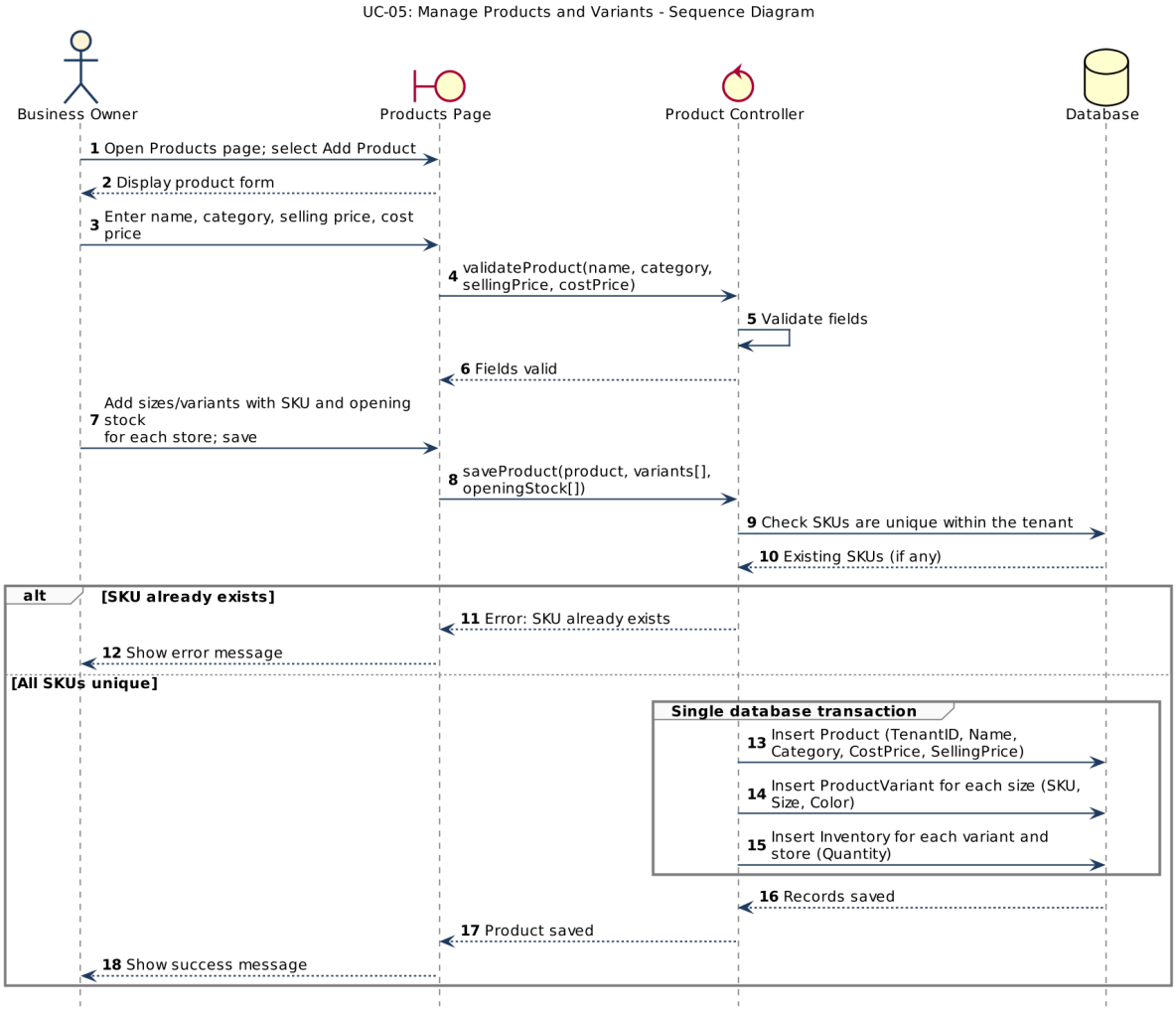


Figure 6.7: Sequence Diagram: Manage Products and Variants
This figure shows the sequence diagram of UC-05 (Manage Products and Variants)

6.6.6 UC-06: Adjust Inventory

This diagram shows the interaction for the use case Adjust Inventory (UC-06).

6.6.7 UC-07: Record Sale (POS)

This diagram shows the interaction for the use case Record Sale (POS) (UC-07).

6.6.8 UC-08: Upload Historical Sales Data

This diagram shows the interaction for the use case Upload Historical Sales Data (UC-08).

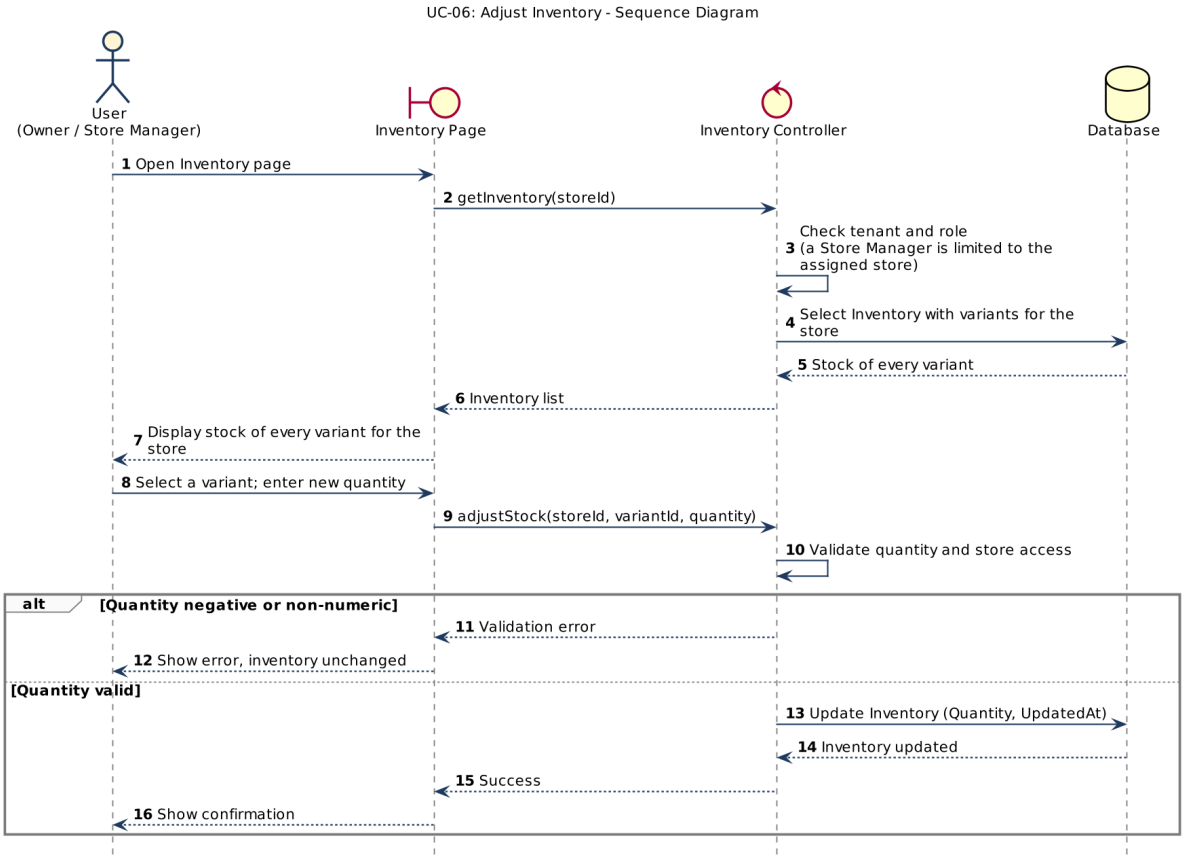


Figure 6.8: Sequence Diagram: Adjust Inventory
This figure shows the sequence diagram of UC-06 (Adjust Inventory)

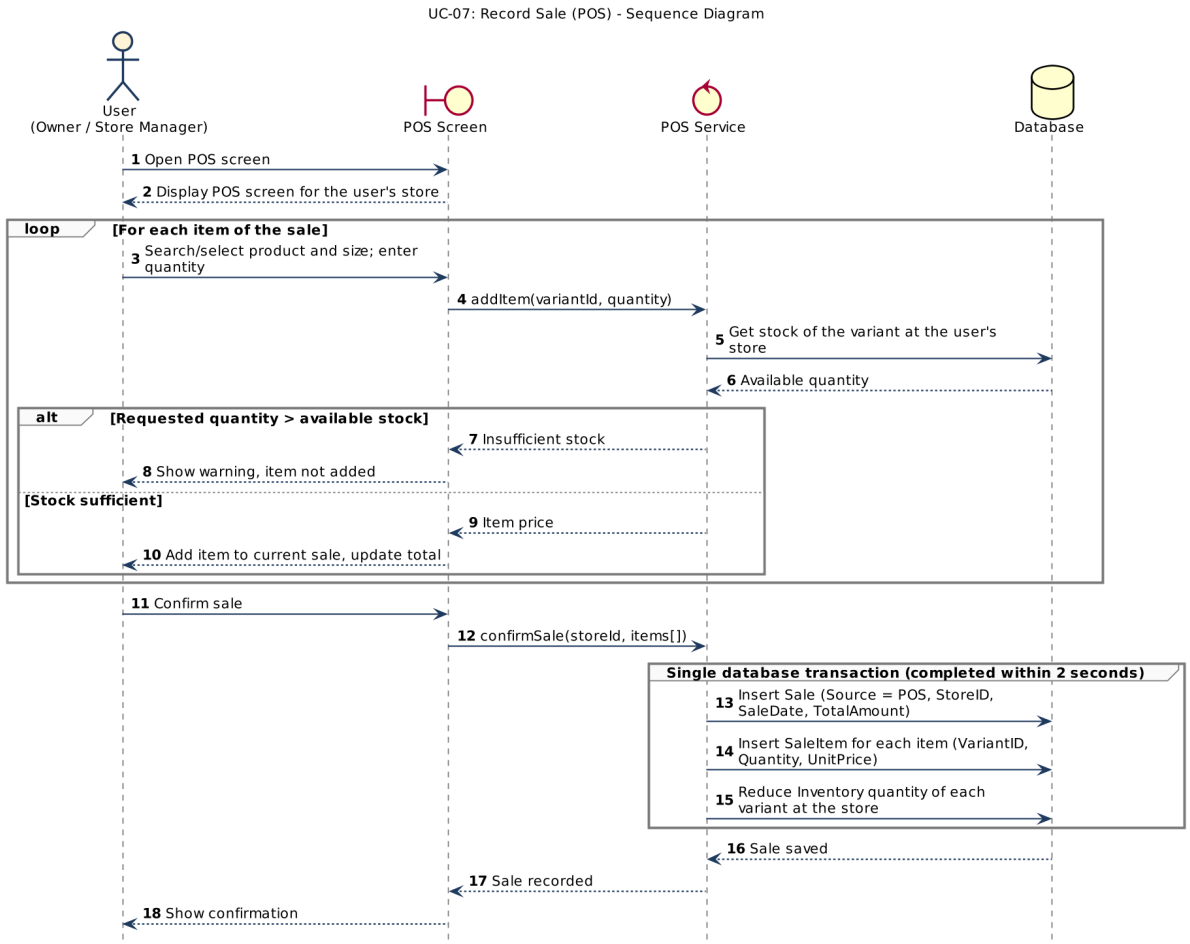


Figure 6.9: Sequence Diagram: Record Sale (POS)
This figure shows the sequence diagram of UC-07 (Record Sale (POS))

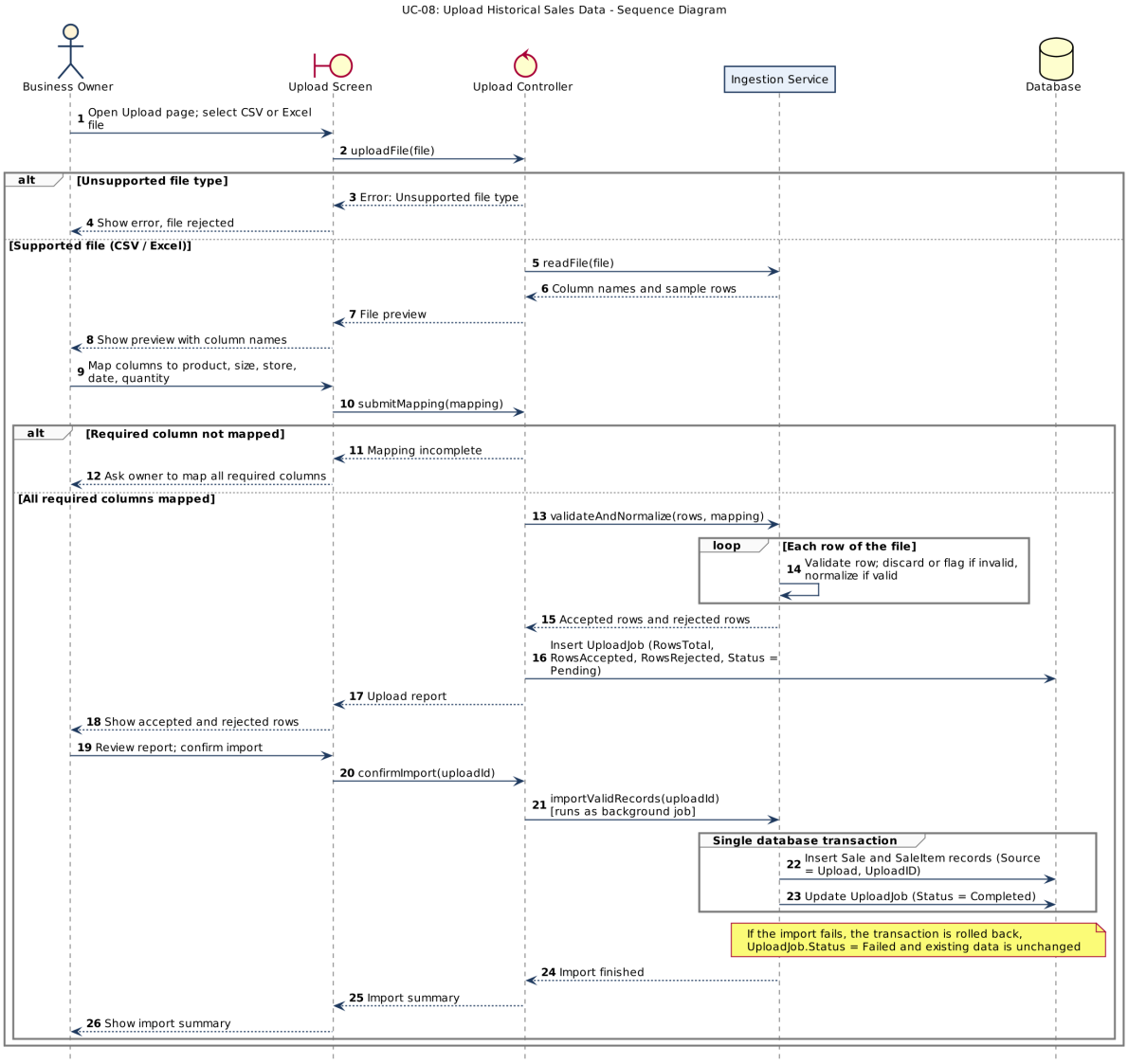


Figure 6.10: Sequence Diagram: Upload Historical Sales Data
This figure shows the sequence diagram of UC-08 (Upload Historical Sales Data)

6.6.9 UC-09: View Dashboard and Alerts

This diagram shows the interaction for the use case View Dashboard and Alerts (UC-09).

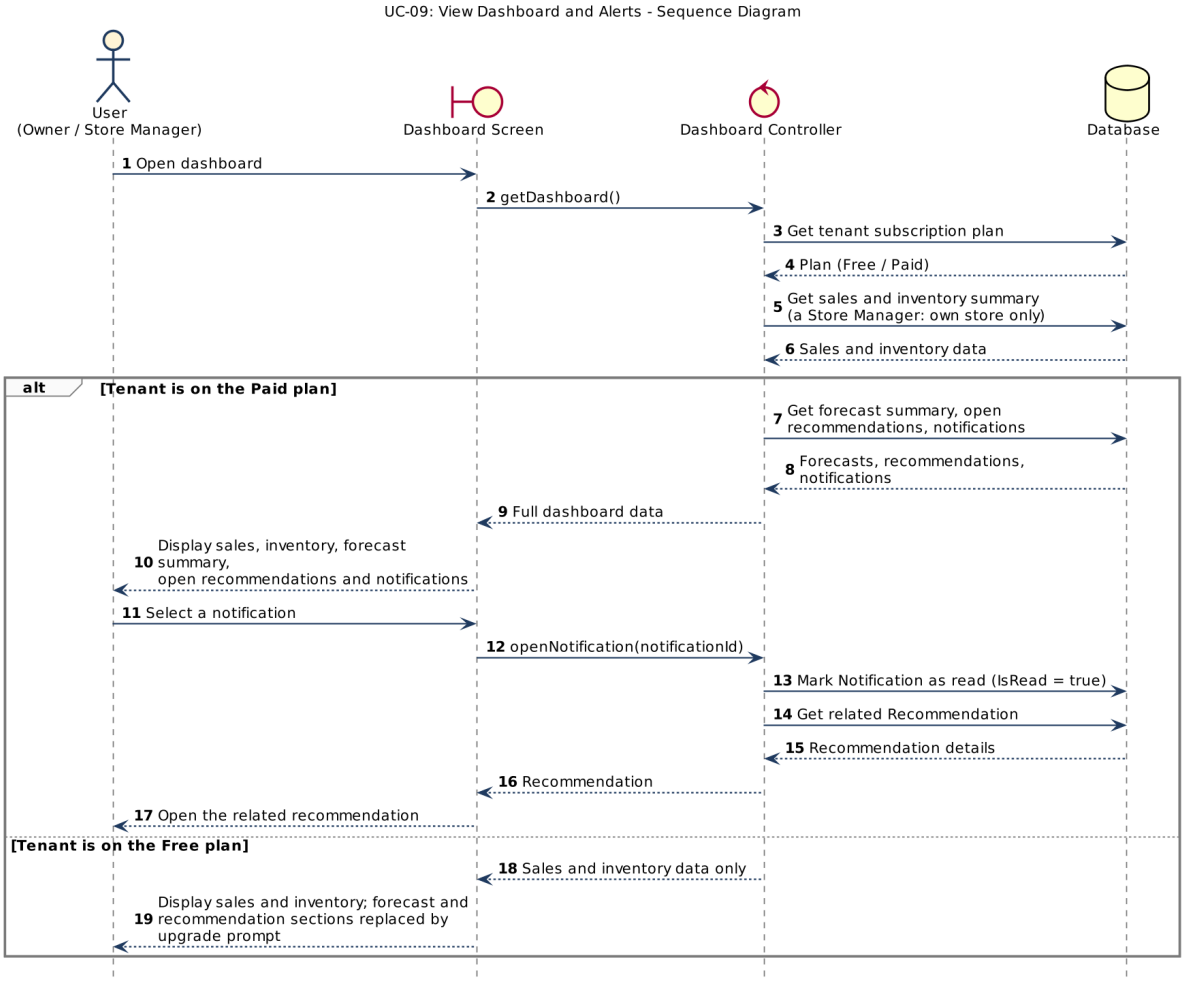


Figure 6.11: Sequence Diagram: View Dashboard and Alerts
This figure shows the sequence diagram of UC-09 (View Dashboard and Alerts)

6.6.10 UC-10: View Demand Forecast

This diagram shows the interaction for the use case View Demand Forecast (UC-10).

6.6.11 UC-11: View and Act on Redistribution Recommendation

This diagram shows the interaction for the use case View and Act on Redistribution Recommendation (UC-11).

6.6.12 UC-12: View Markdown Recommendation

This diagram shows the interaction for the use case View Markdown Recommendation (UC-12).

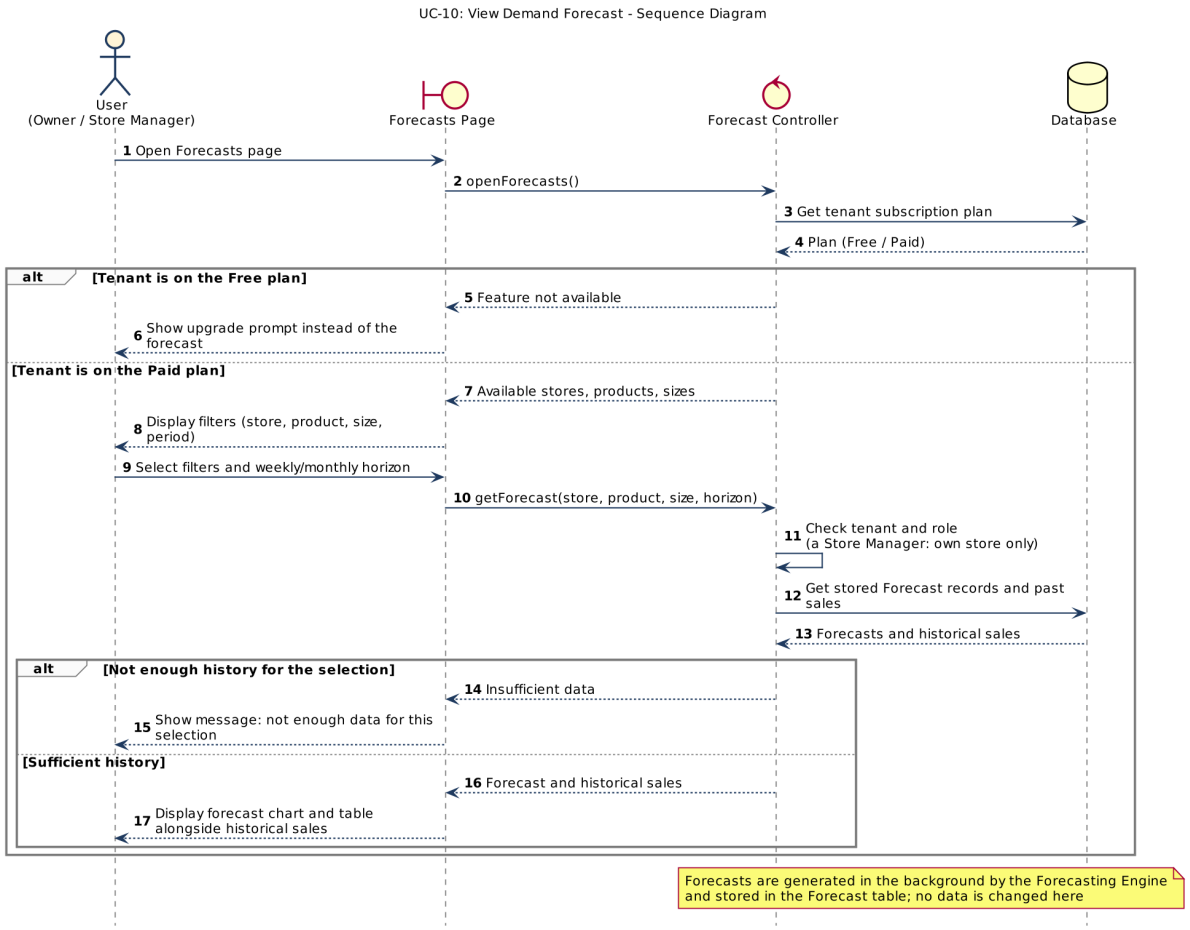


Figure 6.12: Sequence Diagram: View Demand Forecast
This figure shows the sequence diagram of UC-10 (View Demand Forecast)

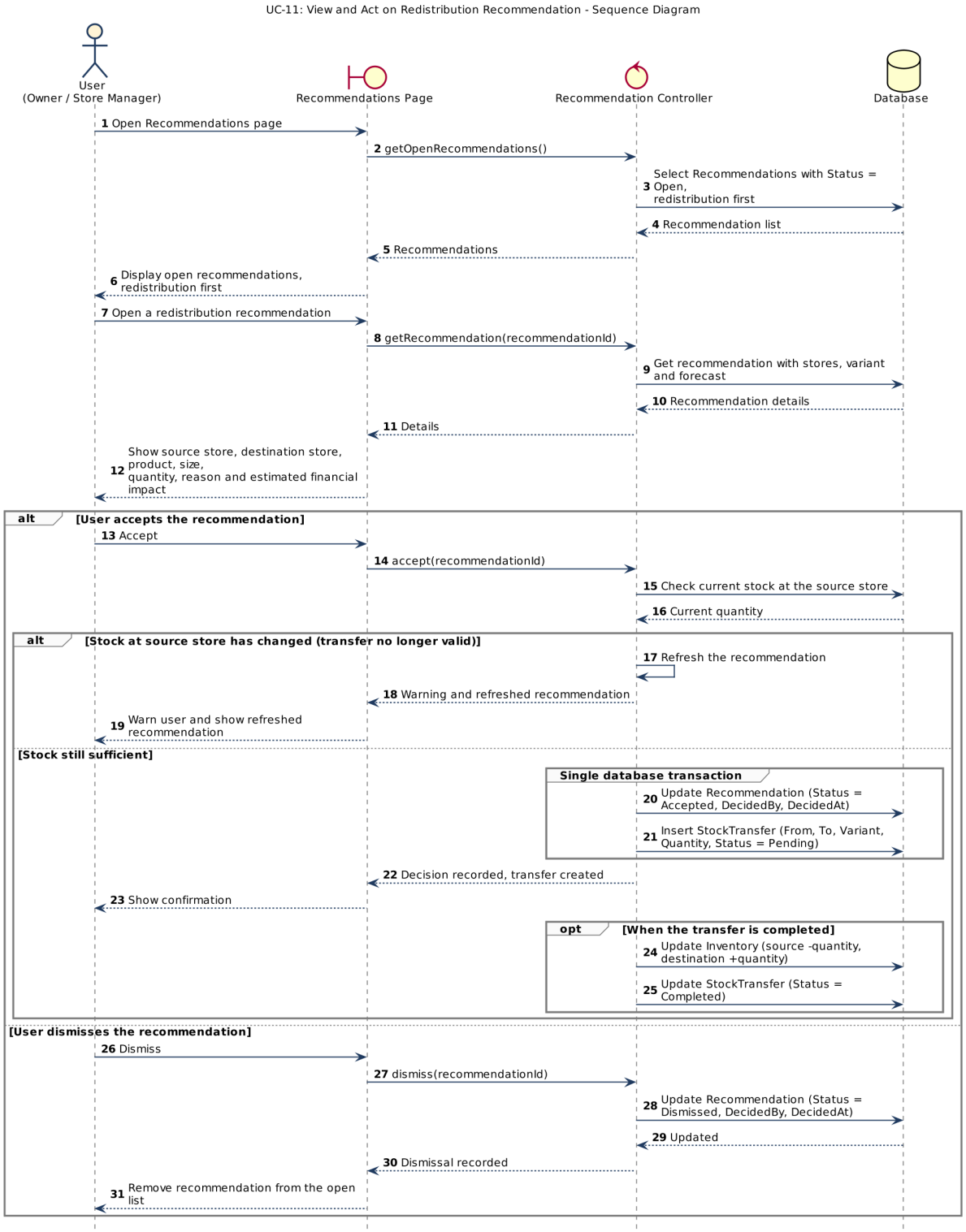


Figure 6.13: Sequence Diagram: View and Act on Redistribution Recommendation
This figure shows the sequence diagram of UC-11 (View and Act on Redistribution Recommendation)

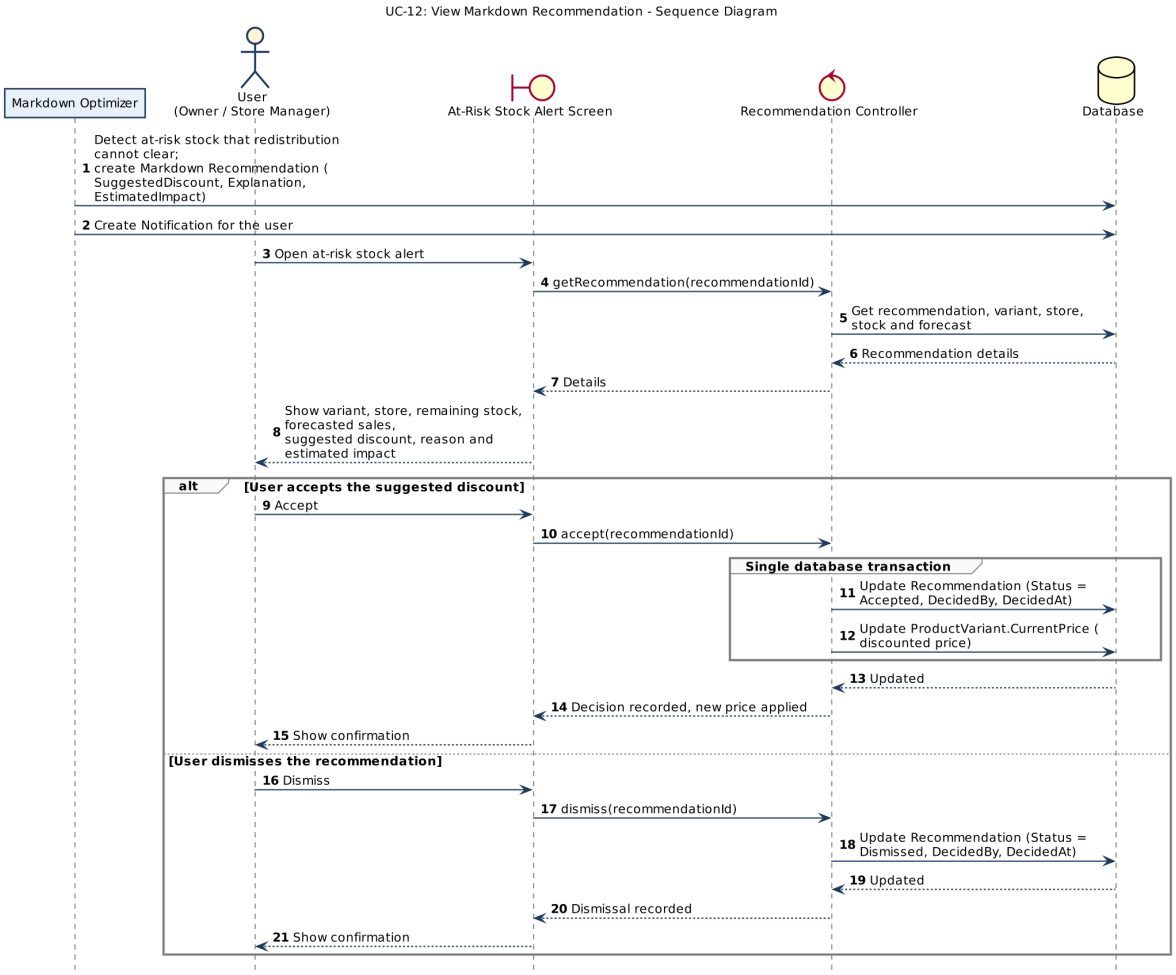


Figure 6.14: Sequence Diagram: View Markdown Recommendation
This figure shows the sequence diagram of UC-12 (View Markdown Recommendation)

6.6.13 UC-13: View Reorder Recommendation

This diagram shows the interaction for the use case View Reorder Recommendation (UC-13).

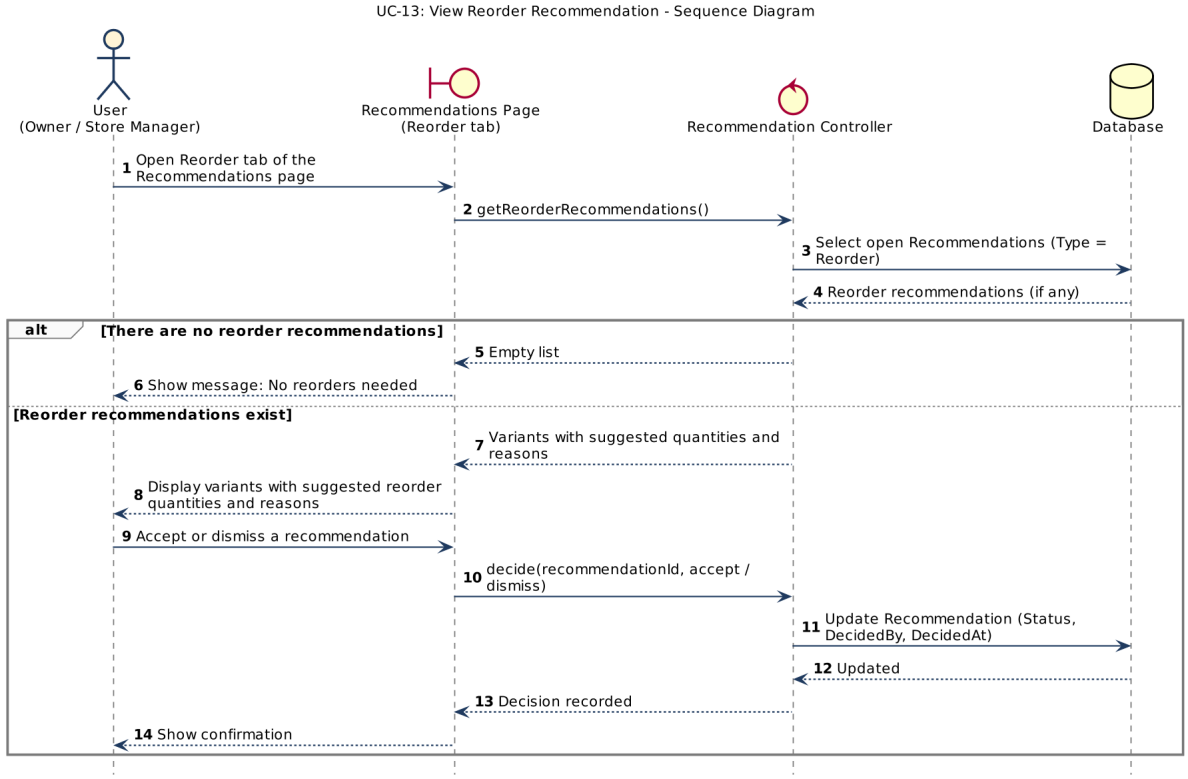


Figure 6.15: Sequence Diagram: View Reorder Recommendation
This figure shows the sequence diagram of UC-13 (View Reorder Recommendation)

6.6.14 UC-14: Manage Subscription

This diagram shows the interaction for the use case Manage Subscription (UC-14).

6.6.15 UC-15: Manage Tenants

This diagram shows the interaction for the use case Manage Tenants (UC-15).

6.7 Policies and Tactics

While creating StockSense, the following policies and tactics will be applied:

6.7.1 Tools to be Used

A number of tools and libraries will be employed to develop StockSense. These include the following:

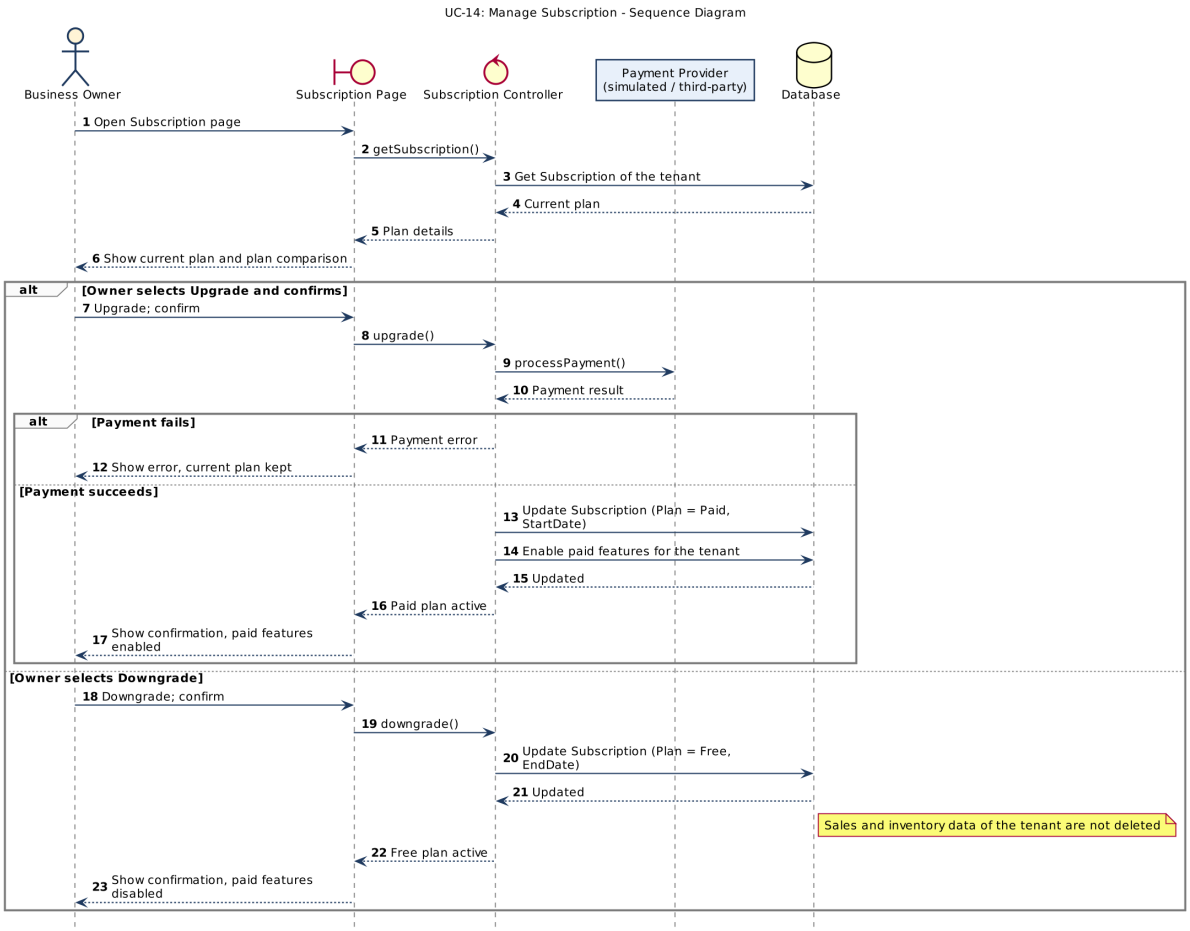


Figure 6.16: Sequence Diagram: Manage Subscription
This figure shows the sequence diagram of UC-14 (Manage Subscription)

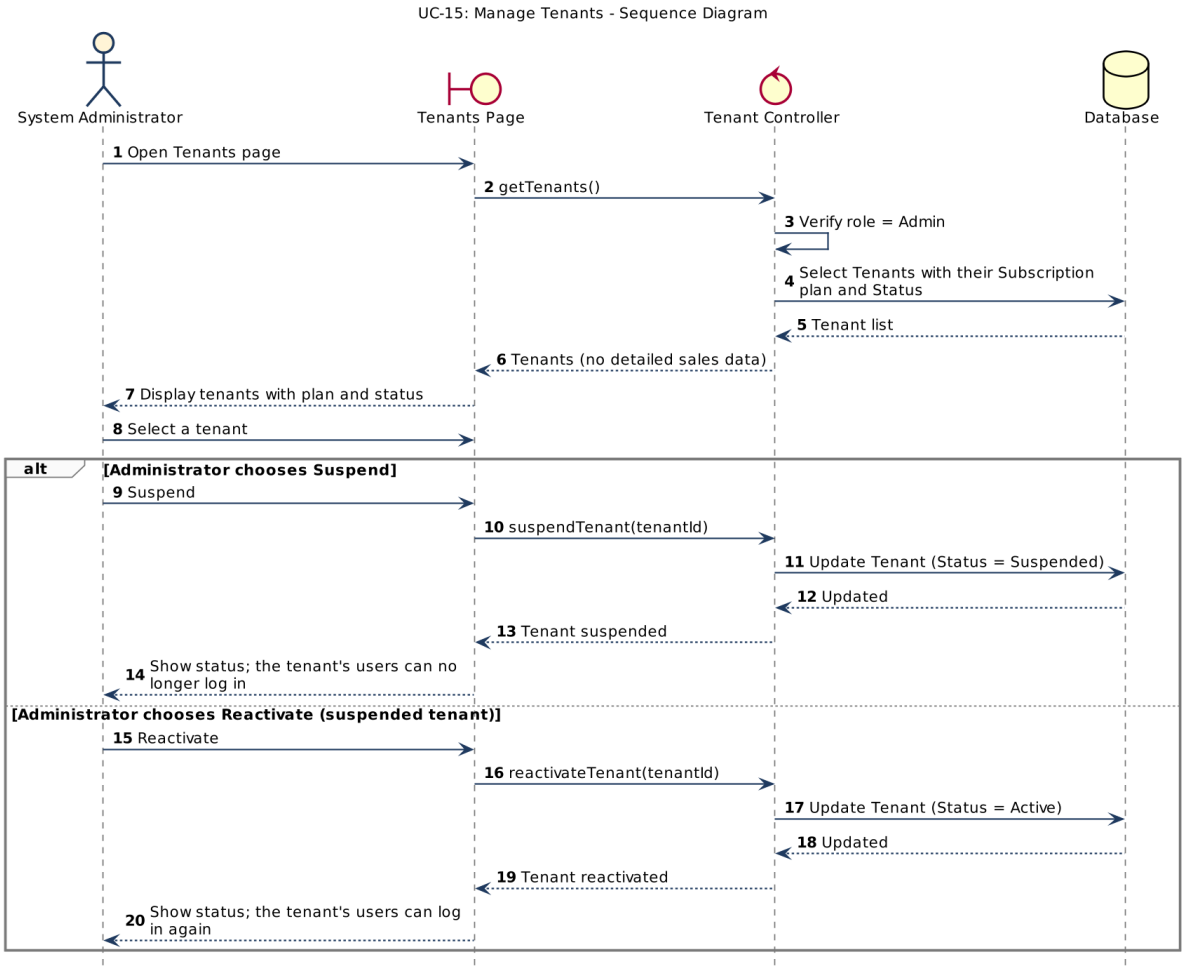


Figure 6.17: Sequence Diagram: Manage Tenants
This figure shows the sequence diagram of UC-15 (Manage Tenants)

- Web dashboard and POS screens will be developed using React.
- Backend REST API will be implemented using Python and FastAPI or Django which will be responsible for authentication, checking of tenants, and communication between the modules.
- All business data will be saved in PostgreSQL with the help of a tenant identifier in every table that contains the data of a particular business.
- CSV and Excel files that have been uploaded will be parsed, cleaned, and normalized using pandas and openpyxl.
- Data preparation, grouping of similar time series, and calculating error metrics will be enabled using scikit-learn and NumPy where the main forecasting algorithm will be LightGBM while the basic statistical models will be calculated using statsmodels.
- Version control will be implemented using Git and GitHub.
- Development will take place using Visual Studio Code.
- API testing will be carried out using Postman.
- Experiments with the model will be conducted using Jupyter Notebook.

6.7.2 Coding Guidelines and Conventions

We will use Python to build the backend and forecast algorithm. However, the front end will be created in JavaScript and React. We will ensure that every code action follows well defined naming convention rules. The implementations of techniques will be short and simple to understand, and comments will be provided for complex parts. Each code module will be properly organized in their own package. Every change will be reviewed and approved by another team member. Being good programmers, the following practices will be implemented: password hashing, parameterized queries, and avoiding secret keys in programming code.

6.7.3 User Interface

StockSense is designed to serve non-professional stock traders and administrators, so it should be, in principle, easy to understand and navigate with minimal education. The system should be intuitive enough for first-time users to carry out their first trading operation without consultation of the user manual. The format of the predictions should be such that they are presented in charts along with stocks sold, and similarly, every recommendation should go along with detailed explanations that are easy to comprehend. It should be ensured that all messages about errors are written in simple and straightforward terms so that every error would be comprehensible; for example, in case there is something wrong with the uploaded file, the error message

should make it clear what went wrong and what the problem is in a simple way.

6.7.4 Extensibility

The system will be created in a modular way; thus, each module will have its function. The modules will include POS Inventory, data ingestion, forecasting, recommendation, and markdown optimization. It will be possible to change one of the modules without changing the others due to the clear boundaries between the modules. The architecture of the forecasting module will be modular, allowing anything to test multiple models without changing the whole application. The forecasting engine will be of general purpose providing any business with the possibility to stock its product with different variations, sizes, or different places of storage, and the well as increasing amount of recommendation rules.

6.7.5 Tenant Data Isolation and Security

Each record in the system will be assigned to a tenant, and the backend will verify tenant ownership before each request reads or updates any data. Features will be accessible based on the role of the user, such that a store manager will only be able to view and update data of the store they have been assigned to. All communication will be via HTTPS. The system administrator can set up tenants and plans but will not be able to view the granular sales data unless required as part of support.

6.7.6 Data Validation

The uploaded sales data will be validated before being sent to the forecasting pipeline. Data that are invalid or incomplete will either be flagged or rejected, while an upload report will notify the owner about the accepted data and rejected data. Should an upload fail midway, no damage will be done to the data that have already been saved.

6.7.7 Explainable Recommendations

In case there is a recommendation, the main reasons for making that particular recommendation (forecasts, existing stock, surplus/shortage, source branch) will be stored together with it, along with the estimated financial impact. In this way, the owner of the company will be able to understand why this particular suggestion was made.

6.7.8 System Testing

Unit testing, integration testing, and end-to-end testing will all be part of the testing policy of StockSense. Individual tests of the system will be conducted on the individual elements of the system (e.g. validation rules and inventory update after a purchase transaction). Integration testing will be performed on API, Database and Modules for proper integration of them and REST Endpoints will be tested using postman. End-to-end tests will be done using complete user journeys, like registering a business, logging a sale, uploading a file, and performing an action after a recommendation is generated. Project specific tests will include tenant separation for no leak, role-based access for each user type and upload pipeline for less than complete and ill formatted files. Forecasting models will be compared to the statistical baseline by the use of the test set which was not present to the forecasting model or the statistical baseline when they were trained. Before launch, tests will also be performed for performance requirements from the requirements chapter, for example: to save a POS sale in less than 2s and to process a file with 100,000 rows in less than 5 minutes.

6.7.9 Software Maintenance

The code will be developed in the version controlled GitHub repository with bugs and tasks monitored, with no loss of any issues. The failures on uploads, the records that are rejected and the failures in forecasting will be recorded and this will make it easier to identify the problem. As the sales trend changes, the models will have to be updated often with the latest data as the forecasts will get inaccurate. It is necessary to regularly back up the database so that, should the database fail, the business information can be retrieved.

6.7.10 Algorithms

Algorithms that will be used in the implementation of the StockSense application will be chosen depending on time and space efficiencies and checked using Big-O notation. LightGBM algorithm is an ensemble of gradient-boosted trees which has proven itself to be very effective in a large retail dataset and was used to predict demand in the M5 competition. The same model will be trained for each tenant with regard to all of its products and sizes. To see a forecast, one needs to record or upload a sufficient sales history, which is required for a new tenant. Simple statistical models, using statsmodels, will be kept as baseline to determine whether the more complicated model is really needed. Forecasted demand will be compared to the current stock level of all branches for each variation by the recommendation algorithm. It will match the branches with surplus stocks and the branches with shortage stocks, then examine slow mov-

ing branch stocks that are still in excess and look for markdowns, and then suggest a reorder. If sales rate of an item indicates that it will run out of stock before it can be sold in time, the Markdown algorithm will start to alert so that corrective action can be taken as soon as possible. Usually there are only a few branches in a business, making pairing a cost-effective approach. The primary keys will be used to index the database while the latest predictions will be cached for serving instead of recomputing and uploads and modeling will be performed in the background.

6.8 Conclusion

In this chapter, the high level as well as the low level design of StockSense was explained. Separate modules have been defined for tenant management, point-of-sale/inventory management, data ingestion, forecasting, recommendation engine, and dashboard reporting within the suggested architecture. Tenant isolation, role-based access control, data validation, background processing, and database indexing constitute the base of the architecture for its security and scalability. The design also supports the project's main decision flow: forecast demand, redistribute existing stock where possible, recommend markdowns for remaining slow-moving stock, and reorder only when necessary. The testing and maintenance strategy will be used to ensure the system is correct, performs well, is secure, and will be accurate before it is put into use.

Chapter 7 Implementation and Test Cases

This chapter discusses how the design of StockSense described in the previous chapters is implemented and how the system will be tested. It describes all the modules, the tools used for each module, and the test cases written for each use case in Chapter 4. The content of this chapter is not final and will be updated in FYP-2 as we implement the solution further.

7.1 Implementation

We implement the solution following the iterative plan specified in Section 5.11. We start with the development of the multi-tenant core. Next come the offline forecast experiments, followed by the implementation of the forecast, recommendation and markdown modules. Each module is developed separately, so that changes to one of them can be made independently of the other modules. The current state of each module is described in the table below.

Table 7.1: Implementation Status of StockSense Modules

No.	Module	Main Tools	Status
1	Tenant and User Management	FastAPI, PostgreSQL	Design complete, implementation in FYP-2
2	Store, Product and Inventory Management	FastAPI, PostgreSQL	Design complete, implementation in FYP-2
3	Point of Sale	React, FastAPI	Screen designed, implementation in FYP-2
4	Data Ingestion and Cleaning	pandas, openpyxl	Design complete, implementation in FYP-2
5	Demand Forecasting	LightGBM, statsmodels	Offline experiments in FYP-2
6	Recommendation Engine	Python, SciPy	Design complete, implementation in FYP-2
7	Markdown Optimizer	Python, NumPy	Design complete, implementation in FYP-2
8	Explainable Recommendations	LightGBM contributions, text templates	Design complete, implementation in FYP-2
9	Dashboard and Alerts	React	Screens designed, implementation in FYP-2
10	Subscription Management	FastAPI, PostgreSQL	Design complete, implementation in FYP-2

7.1.1 Development Environment

The backend will be written in Python with FastAPI. The web dashboard and the POS screens will be written in React. All the data will be stored in PostgreSQL, and every business table will have a tenant ID. The editor is Visual Studio Code, version control is done with Git and GitHub, the API is tested with Postman, and the experiments are done in Jupyter Notebook. The team will work in short iterations, and the supervisor will review the results at the end of each iteration.

7.1.2 Tenant and User Management

This module will perform business sign up, login and the management of roles. During sign up, the business gets a tenant, a free subscription and an owner account, all created in a single database transaction. Only password hashes are stored in the database. After login, every request carries the tenant ID and the role, which the backend checks on every read and write operation on business data. A suspended tenant cannot log in.

7.1.3 Store, Product and Inventory Management

The owner can create, modify and archive stores, define products with sizes and variants, and invite store managers. Variants have SKUs which are unique per tenant. The inventory data is stored per product variant per store. A manual correction of the inventory needs a reason, which is logged. A store manager can view and change only his own store.

7.1.4 Point of Sale

Using the POS screen, a store manager can select the product and the size, enter the quantity and make a sale. First, the system checks whether there is sufficient stock. Once the sale is confirmed, the sale information, the sold items and the updated stock levels are stored in one transaction. The goal is to finish this within 2 seconds, according to the requirements.

7.1.5 Data Ingestion and Cleaning

This component accepts CSV and Excel files. The CSV files are parsed using pandas in chunks, while the Excel files are parsed using pandas and openpyxl in batches, so that memory consumption stays low for files of up to 100,000 rows. The owner matches the detected columns to the standard fields (product, size, store, date, quantity) and sets the date format. Then the system validates the structure, values and references, normalizes the names and sizes (e.g.

"Medium" becomes "M"), detects returns and unusual values, and marks possible duplicates without removing them. The upload report shows the accepted and rejected rows with reasons. Only after the owner's approval are the rows imported, in one transaction that is rolled back on any failure.

7.1.6 Demand Forecasting

Forecasting starts with modeling and testing in offline mode using Jupyter Notebook. The steps are as follows: creation of weekly and monthly demand series for every variant per size and store; creation of lag, rolling, calendar, price and lifecycle features; training of one global LightGBM model per tenant with the Tweedie objective function; and comparing the performance with the baselines (naive, seasonal naive, moving average, Holt-Winters and Croston). An upper-quantile model is also trained to derive the safety stock. The model hyperparameters are tuned with the rolling-origin validation procedure, with no data shuffling. Then the model is integrated into a background job that forecasts the demand and writes the results into the Forecast table with the model name. For a new tenant, no model forecast is produced if there is not enough history, and the user is asked to load more data.

7.1.7 Recommendation Engine

For each variant and store, the engine compares the stock on hand with the forecast demand and the safety stock in order to detect surpluses or shortages. The surplus stock is allocated to the shortage stores by solving a small transportation problem, and a transfer is suggested only if the margin exceeds the transfer cost. If a shortage still remains, a reorder recommendation is generated.

7.1.8 Markdown Optimizer

The excess stock is evaluated based on the weeks of cover. If the product will not be sold in time, the optimizer tries discounts of 5%, 10%, up to 50% using the price elasticity. The lowest discount that meets the sell-through goal without going below the cost price is chosen. The estimated profit is stored as the impact.

7.1.9 Explainable Recommendations

All recommendations are stored along with the figures (forecast demand, stock, surplus or shortage, stores, margin, cost and impact) and the key forecast drivers obtained from the LightGBM feature contributions. These are inserted into text templates, so that the owner

reads a short and simple sentence rather than a score.

7.1.10 Dashboard, Alerts and Subscription

The dashboard shows the monthly sales, the items at risk of stockout and dead stock, the recent sales, the branch status and the open recommendations. The owner or manager can approve or reject a recommendation, and an approved redistribution results in the creation of a stock transfer. On the free plan, the forecast and recommendation parts are replaced by an upgrade notice. The subscription component switches the tenant between the two plans, and a failed payment keeps the previous plan.

7.2 Test Case Design and Description

All use cases from Chapter 4 have at least one test case. The tests will be performed in a Linux development environment, where the React dashboard will be open in Google Chrome, and the backend and PostgreSQL will run locally. Sample tenants with two or three branches and sample sales files will be used. Forecast accuracy is not checked here, since it will be measured separately using the evaluation protocol of Section 5.10.
The Result field remains “Not yet executed”. It will be changed to Passed or Failed, with comments, once the tests are performed in FYP-2.

Table 7.2: Test Case 1: Register Business

Test Case ID	1
QA Test Engineer	Muhammad Usman
Reviewed By	Muhammad Rizwan Arshad
Test Date	To be filled
Use Case Reference	UC-01 Register Business
Objective	To check that a new business can register and gets a free-tier tenant.
Pre-Requisite	The email is not already registered.
Steps	1. Open the signup page. 2. Enter business name, owner name, email and password. 3. Submit the form.
Expected Result	A tenant, a free subscription and an owner account are created, and a verification email is sent. A second signup with the same email shows an error.
Result	Not yet executed

Table 7.3: Test Case 2: Login

Test Case ID	2
QA Test Engineer	Muhammad Usman
Reviewed By	Hayyan Haider
Test Date	To be filled
Use Case Reference	UC-02 Login
Objective	To check login with valid and invalid details and with a suspended tenant.
Pre-Requisite	A registered user exists.
Steps	1. Log in with the correct email and password. 2. Log in with a wrong password. 3. Suspend the tenant and try to log in again.
Expected Result	Valid details open the dashboard. A wrong password shows “Incorrect email or password”. A suspended tenant cannot log in.
Result	Not yet executed

Table 7.4: Test Case 3: Manage Stores

Test Case ID	3
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-03 Manage Stores
Objective	To check that the owner can add, edit and remove stores.
Pre-Requisite	The owner is logged in.
Steps	1. Add a store. 2. Edit its name. 3. Remove a store that has sales history.
Expected Result	The store is added and updated. A store with inventory or sales is archived and not deleted.
Result	Not yet executed

Table 7.5: Test Case 4: Manage Store Managers

Test Case ID	4
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Rizwan Arshad
Test Date	To be filled
Use Case Reference	UC-04 Manage Store Managers
Objective	To check that the owner can invite a manager and assign a store.
Pre-Requisite	The owner is logged in and at least one store exists.
Steps	1. Invite a manager by email and choose a store. 2. Invite an email that already has an account.
Expected Result	The manager account is created and an invitation is sent. The second invitation is rejected with an error.
Result	Not yet executed

Table 7.6: Test Case 5: Manage Products and Variants

Test Case ID	5
QA Test Engineer	Muhammad Rizwan Arshad
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-05 Manage Products and Variants
Objective	To check product creation and the unique SKU rule.
Pre-Requisite	The owner is logged in.
Steps	1. Create a product with sizes 40, 41 and 42. 2. Create another variant with an existing SKU.
Expected Result	The product and its variants are saved. The duplicate SKU is rejected.
Result	Not yet executed

Table 7.7: Test Case 6: Adjust Inventory

Test Case ID	6
QA Test Engineer	Muhammad Rizwan Arshad
Reviewed By	Hayyan Haider
Test Date	To be filled
Use Case Reference	UC-06 Adjust Inventory
Objective	To check manual stock correction and that a negative quantity is refused.
Pre-Requisite	The variant exists at the store.
Steps	1. Enter a new quantity with a reason. 2. Enter a negative quantity.
Expected Result	The first change is saved and logged. The negative value is rejected and the stock stays the same.
Result	Not yet executed

Table 7.8: Test Case 7: Record Sale (POS)

Test Case ID	7
QA Test Engineer	Muhammad Usman
Reviewed By	Hayyan Haider
Test Date	To be filled
Use Case Reference	UC-07 Record Sale (POS)
Objective	To check that a sale reduces inventory, and that low stock gives a warning.
Pre-Requisite	A store manager is logged in and stock is available.
Steps	1. Select a product and size and sell 1 unit. 2. Try to sell more units than are in stock.
Expected Result	The sale is saved and the stock goes down by one. The second sale shows an insufficient stock warning.
Result	Not yet executed

Table 7.9: Test Case 8: Upload Historical Sales Data

Test Case ID	8
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-08 Upload Historical Sales Data
Objective	To check upload, column mapping, cleaning and the upload report.
Pre-Requisite	The owner is logged in. A CSV file with a few wrong rows is ready.
Steps	1. Upload the file. 2. Map the columns. 3. Check the report. 4. Approve the import.
Expected Result	Good rows are cleaned and imported. Wrong rows are listed with reasons. Nothing is saved before approval.
Result	Not yet executed

Table 7.10: Test Case 9: Upload of an Unsupported or Broken File

Test Case ID	9
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Rizwan Arshad
Test Date	To be filled
Use Case Reference	UC-08 Upload Historical Sales Data
Objective	To check that bad files do not damage existing data.
Pre-Requisite	The tenant already has saved sales.
Steps	1. Upload a .pdf file. 2. Upload a damaged Excel file. 3. Force a failure in the middle of an import.
Expected Result	The file is rejected with a simple reason, and a failed import is rolled back. The existing data is unchanged.
Result	Not yet executed

Table 7.11: Test Case 10: View Dashboard and Alerts

Test Case ID	10
QA Test Engineer	Muhammad Rizwan Arshad
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-09 View Dashboard and Alerts
Objective	To check the dashboard for free and paid tenants and for a new tenant.
Pre-Requisite	One free tenant and one paid tenant exist.
Steps	1. Open the dashboard as a paid owner. 2. Open it as a free owner. 3. Open it for a tenant with no sales.
Expected Result	The paid tenant sees sales, inventory, forecasts and alerts. The free tenant sees sales and inventory with an upgrade message. A new tenant is asked to add sales data.
Result	Not yet executed

Table 7.12: Test Case 11: View Demand Forecast

Test Case ID	11
QA Test Engineer	Muhammad Usman
Reviewed By	Hayyan Haider
Test Date	To be filled
Use Case Reference	UC-10 View Demand Forecast
Objective	To check the forecast display and the rules for the free tier and short history.
Pre-Requisite	A paid tenant with enough sales history exists.
Steps	1. Select a product, size and store. 2. Choose weekly and then monthly view. 3. Repeat for a tenant with too little history. 4. Repeat for a free tenant.
Expected Result	The forecast is shown next to past sales. A short history shows a “not enough data” message. A free tenant sees an upgrade message.
Result	Not yet executed

Table 7.13: Test Case 12: Redistribution Recommendation

Test Case ID	12
QA Test Engineer	Muhammad Rizwan Arshad
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-11 View and Act on Redistribution Recommendation
Objective	To check that a surplus store and a shortage store give a transfer suggestion that can be approved or dismissed.
Pre-Requisite	One branch has surplus stock and another has a forecast shortage of the same variant.
Steps	1. Run the recommendation job. 2. Open the recommendation. 3. Approve it. 4. Repeat with another one and dismiss it.
Expected Result	A redistribution is suggested with quantity, explanation and impact. Approval creates a stock transfer and both stores are notified. A dismissed one is closed. If the source stock has changed, a warning is shown.
Result	Not yet executed

Table 7.14: Test Case 13: Markdown Recommendation

Test Case ID	13
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Rizwan Arshad
Test Date	To be filled
Use Case Reference	UC-12 View Markdown Recommendation
Objective	To check that slow-moving stock that cannot be moved gets a discount suggestion.
Pre-Requisite	A variant has weeks of cover above the selling time and no store needs it.
Steps	1. Run the optimizer. 2. Open the markdown recommendation. 3. Accept it.
Expected Result	A discount is suggested that does not go below the cost price. Accepting it updates the price.
Result	Not yet executed

Table 7.15: Test Case 14: Reorder Recommendation

Test Case ID	14
QA Test Engineer	Muhammad Usman
Reviewed By	Hayyan Haider
Test Date	To be filled
Use Case Reference	UC-13 View Reorder Recommendation
Objective	To check that a reorder is suggested only when no other branch has surplus.
Pre-Requisite	A forecast shortage exists and no other branch has surplus.
Steps	1. Run the recommendation job. 2. Open the Reorder tab. 3. Mark one as actioned.
Expected Result	A reorder quantity is shown with its reason. The status changes to Actioned.
Result	Not yet executed

Table 7.16: Test Case 15: Manage Subscription

Test Case ID	15
QA Test Engineer	Muhammad Rizwan Arshad
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-14 Manage Subscription
Objective	To check upgrade, downgrade and failed payment.
Pre-Requisite	The owner is logged in.
Steps	1. Upgrade to the paid plan. 2. Try an upgrade where the payment fails. 3. Downgrade to free.
Expected Result	Paid features turn on after a successful upgrade. A failed payment keeps the old plan. A downgrade turns paid features off but keeps all sales and inventory data.
Result	Not yet executed

Table 7.17: Test Case 16: Manage Tenants

Test Case ID	16
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-15 Manage Tenants
Objective	To check that the administrator can suspend and reactivate a tenant.
Pre-Requisite	The administrator is logged in.
Steps	1. Suspend a tenant. 2. Try to log in as that tenant's user. 3. Reactivate the tenant.
Expected Result	Users of a suspended tenant cannot log in. After reactivation they can log in again.
Result	Not yet executed

7.2.1 Non-Functional Test Cases

Table 7.18: Test Case 17: Tenant Data Isolation

Test Case ID	17
QA Test Engineer	Muhammad Usman
Reviewed By	Hayyan Haider
Test Date	To be filled
Use Case Reference	FR-11, Security (Section 4.4.3)
Objective	To check that one business can never see the data of another.
Pre-Requisite	Two tenants with different data exist.
Steps	1. Log in as Tenant A. 2. Call the API with the IDs of Tenant B records. 3. Log in as a store manager and open another store's data.
Expected Result	All such requests are refused. No data of Tenant B or of another store is returned.
Result	Not yet executed

Table 7.19: Test Case 18: POS Speed

Test Case ID	18
QA Test Engineer	Muhammad Rizwan Arshad
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	Performance (Section 4.4.1)
Objective	To check that a POS sale is saved in under 2 seconds.
Pre-Requisite	A tenant with a normal amount of data.
Steps	Record 50 sales and note the time taken for each.
Expected Result	Each sale is saved in under 2 seconds.
Result	Not yet executed

Table 7.20: Test Case 19: Large File Upload

Test Case ID	19
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Rizwan Arshad
Test Date	To be filled
Use Case Reference	Performance (Section 4.4.1)
Objective	To check that a file with 100,000 rows is processed in under 5 minutes.
Pre-Requisite	A test CSV and a test Excel file with 100,000 rows each.
Steps	Upload both files and note the processing time and memory use.
Expected Result	Both files are processed in under 5 minutes without the system crashing.
Result	Not yet executed

Table 7.21: Test Case 20: Password and Connection Security

Test Case ID	20
QA Test Engineer	Muhammad Usman
Reviewed By	Muhammad Rizwan Arshad
Test Date	To be filled
Use Case Reference	Security (Section 4.4.3)
Objective	To check that passwords are hashed and the traffic uses HTTPS.
Pre-Requisite	A test database and a deployed test server.
Steps	1. Look at the saved password field in the database. 2. Open the site over plain HTTP.
Expected Result	Only hashed passwords are stored. Plain HTTP is not accepted.
Result	Not yet executed

7.3 Test Metrics

The tables below will be filled in after the test cases are executed in FYP-2. Only the planned numbers are filled in at this stage.

Table 7.22: Functional Test Case Metrics

Metric	Value
Number of Test Cases	16
Number of Test Cases Passed	To be filled
Number of Test Cases Failed	To be filled
Test Case Defect Density	To be filled

Table 7.23: Non-Functional Test Case Metrics

Metric	Value
Number of Test Cases	4
Number of Test Cases Passed	To be filled
Number of Test Cases Failed	To be filled
Test Case Defect Density	To be filled

7.4 Conclusion

The implementation process of all the StockSense modules and the testing of these modules were discussed in this chapter. Each module has its own toolset and function, and all the use cases specified in Chapter 4 have test cases. In addition to that, four extra test cases cover the major requirements that regular functional tests do not cover. They include tenant isolation, speed of POS transactions, file uploads and security. All the testing results will be included in FYP-2 once the system is developed. If a test fails, it will be fixed and repeated.

Chapter 9 Experimental Results and Discussion

In FYP-1 our work went mostly into the requirements, methodology and design of StockSense, so the forecasting and recommendation experiments have been designed but not run yet. This chapter describes each planned experiment: the data it uses, how it is set up, and the table its results will go into. The result fields are marked "To be filled" for now and will be completed in FYP-2, following the iterative plan in Section 5.11.

9.1 Objectives of the Experiments

The experiments are meant to answer the following questions:

1. Is a global LightGBM model more accurate than simple statistical baselines at forecasting product-size-store demand?
2. When the sales history is short, does one global model per tenant do better than a separate model for each series?
3. How much sales history does a new tenant need before forecasts can be trusted?
4. Which loss function (Tweedie, Poisson or squared error) works best for sparse retail demand?
5. Do the redistribution and markdown recommendations reduce dead stock and lost sales compared with doing nothing?

9.2 Experimental Setup

9.2.1 Datasets

- Public retail data: The M5 dataset [7] has daily item-store sales, prices and calendar events from Walmart stores. It supports evaluation at the store and hierarchy levels, so we use it for early model development. It has no size-level sales.
- Retailer data: Sales data from a small multi-branch clothing or footwear retailer will be used if we can get access to it. Size-level behaviour can only be tested on data like this.
- Sample tenant data: Small sample tenants with two or three branches each are used to test the upload pipeline and the recommendation logic end to end.

9.2.2 Data Preparation

The data is cleaned and aggregated into weekly and monthly series for each variant and store, as described in Section 5.4. Each series is labelled smooth, erratic, intermittent or lumpy using

the ADI and CV^2 thresholds of 1.32 and 0.49, so accuracy can be reported separately for each demand class. Periods in which a variant was out of stock for most of the time are treated as censored and left out of the training loss.

9.2.3 Environment and Tools

The experiments run offline in Jupyter Notebook with Python, pandas, NumPy, scikit-learn, LightGBM and statsmodels. Random seeds are fixed so the results can be reproduced.

9.2.4 Evaluation Protocol

All splits are chronological. Hyperparameters are tuned with rolling-origin validation on an expanding window, and the final comparison uses the most recent period, which no model sees during training or tuning. Accuracy is measured with RMSE, MAE, WAPE and non-zero MAPE as defined in Section 5.10. For MAPE we also report the share of periods left out because actual demand was zero.

9.3 Exploratory Analysis of the Demand Data

Before training any model, we check how sparse the demand series are. If a large share of series turn out to be intermittent or lumpy, baselines built for that kind of demand, such as Croston's method, become more important in the comparison.

Table 9.1: Distribution of Demand Classes

Demand Class	Number of Series	Share of Series (%)
Smooth	To be filled	To be filled
Erratic	To be filled	To be filled
Intermittent	To be filled	To be filled
Lumpy	To be filled	To be filled

9.4 Experiment 1: LightGBM Against Statistical Baselines

Aim: To test the main model-selection claim from Chapter 3, that the global LightGBM model is more accurate than simple baselines.

Setup: The LightGBM model, trained with the Tweedie objective, and every baseline are evaluated on the same chronological test period with a weekly horizon.

Table 9.2: Forecast Accuracy of LightGBM and Baselines (Weekly)

Model	RMSE	MAE	WAPE (%)	MAPE (%)
Naive (last value)	To be filled	To be filled	To be filled	To be filled
Seasonal naive	To be filled	To be filled	To be filled	To be filled
Moving average	To be filled	To be filled	To be filled	To be filled
Holt–Winters	To be filled	To be filled	To be filled	To be filled
Croston's method	To be filled	To be filled	To be filled	To be filled
LightGBM (global)	To be filled	To be filled	To be filled	To be filled

We will also break these results down by demand class and by fast- and slow-moving items, because an overall average can hide poor accuracy on sparse series.

9.5 Experiment 2: Global Model Against Separate Models

Aim: Montero-Manso and Hyndman [10] and Bandara et al. [6] suggest that one shared model can beat per-series models when each series is short. This experiment checks whether that holds for StockSense's product-size-store series.

Setup: One global model per tenant and one model per series are trained on the same data and evaluated on the same test period.

Table 9.3: Global Model Against Separate Models

Approach	RMSE	WAPE (%)	Training Time
One global model per tenant	To be filled	To be filled	To be filled
One model per series	To be filled	To be filled	To be filled

9.6 Experiment 3: Amount of Training History

Aim: To find the minimum sales history a new tenant needs before model forecasts are shown (Section 5.6).

Setup: The model is trained on only the last 4, 8 and 12 weeks of history and evaluated on the same test period each time.

Table 9.4: Forecast Accuracy Against Training History

History Used	RMSE	WAPE (%)	Better than Best Baseline?
4 weeks	To be filled	To be filled	To be filled
8 weeks	To be filled	To be filled	To be filled
12 weeks	To be filled	To be filled	To be filled

The system will use the shortest history at which LightGBM clearly beats the best baseline as its minimum history threshold.

9.7 Experiment 4: Loss Function Comparison

Aim: Demand is non-negative and full of zeros, which is why we chose the Tweedie objective. This experiment checks that choice.

Setup: LightGBM is trained with the Tweedie objective, the Poisson objective and squared error, keeping all other settings the same.

Table 9.5: Comparison of LightGBM Objective Functions

Objective	RMSE	MAE	WAPE (%)
Tweedie	To be filled	To be filled	To be filled
Poisson	To be filled	To be filled	To be filled
Squared error	To be filled	To be filled	To be filled

9.8 Experiment 5: Recommendation Backtest

Aim: To estimate what the redistribution, markdown and reorder recommendations are worth, by replaying past sales as described in Section 5.10.

Setup: At each cut-off date, the engine produces recommendations from the data available up to that date. These are applied to a simulated stock record, and the following weeks are replayed with the actual sales.

Table 9.6: Recommendation Backtest Results

Measure	No Action	With StockSense	Change
Dead-stock units	To be filled	To be filled	To be filled
Lost sales (units)	To be filled	To be filled	To be filled
Sell-through rate (%)	To be filled	To be filled	To be filled
Number of transfers	–	To be filled	–
Estimated margin impact	–	To be filled	–

9.9 Discussion

Once the results are in, we will look first at whether LightGBM is accurate enough over the baselines to be worth the extra complexity. This matters most for intermittent and lumpy series, where simple methods often do well. We will then look at how much history a new tenant needs, since that decides how long onboarding takes in UC-10. Finally, we will check whether redistribution cuts dead stock and lost sales more than markdowns alone, which is the main claim behind StockSense.

The M5 results [7] and the small-business study by Purnamasari et al. [8] lead us to expect gradient-boosted models to beat the statistical baselines, but our own experiments still have to confirm this.

9.9.1 Threats to Validity

- No size-level data in M5: Conclusions about sizes will come only from real retailer data, since M5 does not record sizes.
- Censored demand: Sales recorded during stock-outs understate true demand, which can bias both the forecasts and the backtest.
- Simulation assumptions: The backtest assumes fixed transfer lead times and a constant price elasticity for markdowns, and neither may hold in practice.
- Retailer differences: Results from one retailer or dataset may not carry over to every tenant.

9.10 Conclusion

This chapter set out the experiments for StockSense: a demand sparsity analysis, a comparison of LightGBM with statistical baselines, global against separate models, a minimum-history test,

a loss-function comparison, and a historical backtest of the recommendations. Since the setup and result tables are fixed now, the FYP-2 experiments can be run and reported the same way. All results will be filled in and discussed in FYP-2.

Chapter 10 Conclusion and Future Work

This chapter summarizes the work completed in FYP-1 and presents the plan for the work to be completed in FYP-2.

10.1 Conclusion

StockSense is recommended as a multi-tenant SaaS solution to assist and enable small and medium sized retailers to manage their product, variants, inventory and stores via a single solution. It incorporates a free point-of-sale and inventory layer along with historical sales upload, product-size-location demand forecasting, stock redistribution, markdown optimization, reorder suggestions and explanations of the recommended actions. The project vision, business opportunity, related work, requirements, methodology, system architecture, database design, security controls, testing strategy and maintenance approach have been presented in the report. The proposed design addresses the main inventory problems faced by the target retailers. It enables them to see the difference in the demand of products, variants and branches through sales information. It prioritizes moving available stock from a branch with surplus stock to a branch with shortage of stock before suggesting markdowns or a new order. The platform also retains all of the reasons and approximate financial consequences of each suggestion so that business owners can understand and audit the system's decisions.

The work completed in this report includes the definition of the system scope, research on competing platforms, a literature review of previous work, functional and non-functional requirements, use cases, data ingestion process, forecasting and recommendation workflow, evaluation protocol, and high-level and low-level design. The planned implementation will use a React dashboard, a Python REST backend, PostgreSQL, pandas and openpyxl for data processing, and LightGBM with statistical baselines for forecasting. In FYP-2, the backend, database, authentication, tenant isolation, dashboard, POS workflows, and CSV and Excel upload pipeline will be implemented and integrated. The models, the recommendations on redistribution, markdown optimization, reorder suggestions, and the explanation features will then be designed.The models, recommendation for redistribution, markdown optimization, reorder suggestions and explanation features will then be designed. The whole system will be tested through unit, integration, and end-to-end testing, evaluated using forecasting metrics and historical data simulation, and prepared for documentation, deployment, and final demonstration.

10.2 Future Work

Future work will focus on solving the cold-start problem for businesses with no past sales data, so they can receive useful insights from the first day of their business. The system will also be made compatible with a wide range of POS systems and will include seasonal and promotional features. These features will consider changes in demand during periods such as holidays, festivals, or different seasons, as well as the effect of discounts and special offers on sales. This will help the system provide more accurate forecasts and recommendations.

Bibliography

[1] P. K. Singh, Y. Gupta, N. Jha, and A. Rajan, "Fashion retail: Forecasting demand for new items," arXiv preprint arXiv:1907.01960, 2019. [Online]. Available: https://arxiv.org/abs/1907.01960

[2] A. L. D. Loureiro, V. L. Miguéis, and L. F. M. da Silva, "Exploring the use of deep neural networks for sales forecasting in fashion retail," Decision Support Systems, vol. 114, pp. 81–93, 2018.

[3] K. J. Ferreira, B. H. A. Lee, and D. Simchi-Levi, "Analytics for an online retailer: Demand forecasting and price optimization," Manufacturing & Service Operations Management, vol. 18, no. 1, pp. 69–88, 2016.

[4] F. Caro and J. Gallien, "Clearance pricing optimization for a fast-fashion retailer," Operations Research, vol. 60, no. 6, pp. 1404–1422, 2012.

[5] C. Paterson, G. Kiesmüller, R. Teunter, and K. Glazebrook, "Inventory models with lateral transshipments: A review," European Journal of Operational Research, vol. 210, no. 2, pp. 125–136, 2011.

[6] K. Bandara, C. Bergmeir, and S. Smyl, "Forecasting across time series databases using recurrent neural networks on groups of similar series: A clustering approach," Expert Systems with Applications, vol. 140, p. 112896, 2020.

[7] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," International Journal of Forecasting, vol. 38, no. 4, pp. 1346–1364, 2022.

[8] D. I. Purnamasari, V. A. Permadi, A. Saepudin, and R. P. Agusdin, "Demand forecasting for improved inventory management in small and medium-sized businesses," Jurnal Nasional Pendidikan Teknik Informatika (JANAPATI), vol. 12, no. 1, pp. 56–66, 2023.

[9] R. Fildes, S. Ma, and S. Kolassa, "Retail forecasting: Research and practice," International Journal of Forecasting, vol. 38, no. 4, pp. 1283–1318, 2022.

[10] P. Montero-Manso and R. J. Hyndman, "Principles and algorithms for forecasting groups of time series: Locality and globality," International Journal of Forecasting, vol. 37, no. 4, pp. 1632–1653, 2021.

[11] M. L. Fisher and A. Raman, "Reducing the cost of demand uncertainty through accurate response to early sales," Operations Research, vol. 44, no. 1, pp. 87–99, 1996.

[12] S. Thomassey, "Sales forecasts in clothing industry: The key success factor of the supply chain management," International Journal of Production Economics, vol. 128, no. 2, pp. 470–483, 2010.

[13] M. Seyedan and F. Mafakheri, "Predictive big data analytics for supply chain demand forecasting: methods, applications, and research opportunities," Journal of Big Data, vol. 7, p. 53, 2020.

[14] F. Petropoulos et al., "Forecasting: theory and practice," International Journal of Forecasting, vol. 38, no. 3, pp. 705–871, 2022.

[15] D. Bertsimas and N. Kallus, "From predictive to prescriptive analytics," Management Science, vol. 66, no. 3, pp. 1025–1044, 2020.

[16] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," Nature Machine Intelligence, vol. 1, no. 5, pp. 206–215, 2019.

[17] D. Corsten and T. Gruen, "Desperately seeking shelf availability: an examination of the extent, the causes, and the efforts to address retail out-of-stocks," International Journal of Retail & Distribution Management, vol. 31, no. 12, pp. 605–617, 2003.